

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC NHA TRANG
KHOA CÔNG NGHỆ THÔNG TIN**



ĐỒ ÁN TỐT NGHIỆP

**ỨNG DỤNG PHÁT HIỆN VÀ PHÂN LOẠI KÍCH THƯỚC TÔM
TRONG CHẾ BIẾN THỦY SẢN**

Giảng viên hướng dẫn: ThS. Nguyễn Hải Triều
Sinh viên thực hiện: Mai Ngọc Hoàng Long
Mã số sinh viên: 64131212

Khánh Hòa – 2026

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC NHA TRANG
KHOA CÔNG NGHỆ THÔNG TIN**



ĐỒ ÁN TỐT NGHIỆP

**ỨNG DỤNG PHÁT HIỆN VÀ PHÂN LOẠI KÍCH THƯỚC TÔM
TRONG CHẾ BIẾN THỦY SẢN**

GVHD: ThS. Nguyễn Hải Triều
SVTH: Mai Ngọc Hoàng Long
MSSV: 64131212

Khánh Hòa – Tháng 6/2026

TRƯỜNG ĐẠI HỌC NHA TRANG

Khoa Công Nghệ Thông Tin

PHIẾU ĐĂNG KÝ NHẬN ĐỒ ÁN TỐT NGHIỆP

1. Họ và tên sinh viên/nhóm sinh viên đăng ký đề tài (sĩ số trong nhóm: 01)

(1) Mai Ngọc Hoàng Long MSSV: 64131212 khóa: 64
(2).....MSSV:.....khóa:.....
(3).....MSSV:.....khóa:.....
(4).....MSSV:.....khóa:.....
(5).....MSSV:.....khóa:.....
Ngành: Công Nghệ Thông Tin Khoa Công Nghệ Thông Tin

2. Tên đề tài đăng ký:

Ứng dụng phát hiện và phân loại kích thước trong chế biến thủy sản

3. Cán bộ hướng dẫn:

(1) ThS. Nguyễn Hải Triều
(2).....

Sinh viên đã hiểu rõ yêu cầu của đề tài, cam kết thực hiện và hoàn thành theo đúng nội dung, thời hạn đề ra.

Khánh Hòa, ngày... tháng ... năm

Ý kiến cán bộ hướng dẫn

(Ký và ghi rõ họ tên)

Sinh viên

(Ký và ghi rõ họ tên)

Trưởng Bộ môn duyệt

(Ký và ghi rõ họ tên)

Khoa Công Nghệ Thông Tin

Ngành: Công Nghệ Thông Tin

<i>Lần KT</i>	<i>Ngày</i>	<i>Nội dung</i>	<i>Nhận xét của GVHD</i>
1	21/03/2026	Tìm hiểu về đề tài, thu thập dữ liệu, huấn luyện mô hình, xây dựng chức năng đo độ dài tôm với đầu vào là ảnh.	
2	04/04/2026	Thu thập dữ liệu, huấn luyện lại mô hình, tối ưu tốc độ chức năng đo độ dài với đầu vào là ảnh, xây dựng chức năng đo độ dài tôm với đầu vào là video.	
3	18/04/2026	Tối ưu tốc độ thực thi của hệ thống và hiển thị ảnh kết quả.	
4	09/05/2026	Xây dựng giao diện UI cho ứng dụng.	
5	30/05/2026	Tối ưu tốc độ thực thi hệ thống và viết báo cáo.	
Kiểm tra giữa tiến độ của Trưởng Bộ môn			
Ngày kiểm tra:		Đánh giá công việc hoàn thành:.....%:	Ký tên
.....		Được tiếp tục: ☐ Không tiếp tục: ☐	
<i>Lần KT</i>	<i>Ngày</i>	<i>Nội dung</i>	<i>Nhận xét của GVHD</i>
6			
7			
8			
9			
...			

Nhận xét chung (sau khi sinh viên hoàn thành ĐA/KL/CĐTN):

.....
.....
.....

Điểm hình thức:...../10 Điểm nội dung:...../10 **Điểm tổng kết:**...../10

Đối với ĐA/KLTN: Kết luận sinh viên: Được bảo vệ: ☐ Không được bảo vệ: ☐

Khánh Hòa, ngày.....tháng.....năm.....

Cán bộ hướng dẫn

(Ký và ghi rõ họ tên)

Khoa Công Nghệ Thông Tin

Cơ quan công tác: Trường Đại Học Nha Trang

Cán bộ chấm phản biện
(Ký và ghi rõ họ tên)

II. Phần nhận xét cụ thể (dựa theo phiếu chấm điểm và khung tiêu chí đánh giá theo Rubric)

II.1. Hình thức thuyết minh (tỉ trọng 30%)

** **Trình bày** (Rõ ràng, mạch lạc? Biểu bảng, hình vẽ trình bày rõ ràng, đúng quy cách? ...)*

.....

** **Bố cục và lập luận** (Bố cục hợp lý? Tỉ trọng giữa các phần? Cơ sở lập luận?...)*

.....

** **Văn phong** (Gọn gàng, súc tích hay rườm rà, khó hiểu? Lỗi văn phạm và chính tả? ...)*

.....

II.2. Nội dung thuyết minh (tỉ trọng 30%)

** **Mục tiêu nghiên cứu** (Trình bày rõ ràng? Ý nghĩa khoa học và thực tiễn? Tính khả thi?...)*

.....

** **Tổng quan tài liệu** (Phân tích và đánh giá? Độ tin cậy và chất lượng nguồn tài liệu? ...)*

.....

** **Phương pháp nghiên cứu** (Hiện đại? Phù hợp với mục tiêu và nội dung nghiên cứu? Mô tả? Đánh giá và so sánh với các phương pháp khác? ...)*

.....

II.3. Kết quả nghiên cứu (tỉ trọng 20%)

** **Kết quả đạt được** (Độ tin cậy? Tính sáng tạo? Giá trị khoa học và thực tiễn?...)*

.....

** **Kết luận** (Đáp ứng mục tiêu nghiên cứu? Quan điểm của cá nhân? ...)*

.....

4. MỨC ĐỘ TRÍCH DẪN VÀ SAO CHÉP (tỉ trọng 20%)

** **Mức độ trích dẫn** (Đúng quy định? Trung thực, đầy đủ, rõ ràng? Sắp xếp tài liệu tham khảo?...)*

.....

** **Mức độ sao chép** (Tỉ lệ sao chép? Hình thức sao chép?...)*

.....

Khoa Công Nghệ Thông Tin

(Ký và ghi rõ họ tên)

Mai Ngoc Hoang Long

MNHLong_final.docx

-  Check ĐA 02
-  ĐATN 2026
-  Đào tạo Đại học

Thông tin chi tiết về tài liệu

ID bài nộp

trn:oid:::1:3587843281

Ngày nộp

09:01 GMT+7 6 thg 6, 2026

Ngày tải xuống

09:10 GMT+7 6 thg 6, 2026

Tên tệp

MNHLong_final.docx

Kích cỡ tệp

21.8 MB

106 Trang

20.502 Từ

78.910 Ký tự

8% Tính tương đồng nói chung

Tổng cộng của tất cả các kết quả trùng khớp, bao gồm cả các nguồn trùng lặp, cho mỗi c...

Đã lọc khỏi Báo cáo

- Mục lục tham khảo
- Văn bản được trích dẫn

Nguồn hàng đầu

- 5%  Nguồn Internet
- 3%  Ấn bản
- 3%  Bài tập được nộp (bài của học sinh)

Cờ chỉ báo về tính toàn vẹn

0 cờ chỉ báo về tính toàn vẹn để đánh giá

Không phát hiện thấy văn bản nào nghi ngờ là ngụy tạo.

Các thuật toán trong hệ thống của chúng tôi xem xét kỹ lưỡng một tài liệu để phát hiện sự không nhất quán khiến cho tài liệu đó khác với một bài nộp bình thường. Nếu nhận thấy điều gì lạ, chúng tôi sẽ gắn cờ để bạn xem xét.

Cờ không nhất thiết là dấu hiệu chỉ báo có vấn đề. Tuy nhiên, bạn nên chú ý vào phần đó để xem xét thêm.

LỜI CAM ĐOAN CỦA SINH VIÊN VỀ LIÊM CHÍNH HỌC THUẬT

Em xin cam đoan rằng toàn bộ nội dung được trình bày trong báo cáo với đề tài “Ứng dụng phát hiện và phân loại kích thước tôm trong chế biến thủy sản” là kết quả của quá trình tự tìm hiểu, nghiên cứu và thực hiện của bản thân em dưới sự hướng dẫn của giảng viên hướng dẫn.

Các nội dung liên quan đến cơ sở lý thuyết, quá trình xây dựng hệ thống, dữ liệu thực nghiệm, kết quả đánh giá mô hình, hình ảnh minh họa và chương trình ứng dụng được trình bày trong báo cáo đều đảm bảo tính trung thực, khách quan và có nguồn gốc rõ ràng. Những tài liệu, công trình nghiên cứu, bài báo khoa học, website và các nguồn tham khảo được sử dụng trong quá trình thực hiện đề tài đều đã được trích dẫn và liệt kê đầy đủ trong phần Tài liệu tham khảo theo đúng quy định.

Em cam kết không sao chép, đạo văn hoặc sử dụng trái phép kết quả nghiên cứu, mã nguồn, dữ liệu hay sản phẩm trí tuệ của cá nhân, tổ chức khác mà không ghi nhận nguồn tham khảo. Các kết quả thực nghiệm trong báo cáo phản ánh đúng quá trình triển khai, huấn luyện, kiểm thử và đánh giá hệ thống phát hiện, đo lường và phân loại kích thước tôm.

Trong quá trình thực hiện đề tài, mặc dù em đã cố gắng nghiên cứu và hoàn thiện hệ thống với tinh thần nghiêm túc, tuy nhiên do còn hạn chế về kiến thức chuyên môn, kinh nghiệm thực tiễn và điều kiện thực nghiệm nên báo cáo khó tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý, nhận xét và đánh giá từ quý thầy cô để đề tài được hoàn thiện hơn.

Em xin hoàn toàn chịu trách nhiệm trước Nhà trường, Khoa và Giảng viên hướng dẫn nếu phát hiện bất kỳ sai phạm nào liên quan đến tính trung thực, liêm chính học thuật hoặc vấn đề bản quyền trong nội dung báo cáo này.

Khánh Hòa, ngày.....tháng.....năm.....

Sinh viên thực thiện

LỜI CẢM ƠN

Trong quá trình thực hiện báo cáo với đề tài “Ứng dụng phát hiện và phân loại kích thước tôm trong chế biến thủy sản”, em đã nhận được nhiều sự quan tâm, giúp đỡ và hướng dẫn tận tình từ quý thầy cô, gia đình và bạn bè. Đây là nguồn động viên quan trọng giúp em hoàn thành đề tài này.

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc đến Ban Giám hiệu Trường Đại học Nha Trang đã tạo điều kiện thuận lợi về môi trường học tập, cơ sở vật chất và chương trình đào tạo trong suốt quá trình em học tập tại trường.

Em xin chân thành cảm ơn quý thầy cô Khoa Công nghệ Thông tin đã tận tình giảng dạy, truyền đạt những kiến thức chuyên môn và kinh nghiệm quý báu. Những kiến thức đó là nền tảng quan trọng giúp em thực hiện đề tài, đặc biệt trong các nội dung liên quan đến xử lý ảnh, trí tuệ nhân tạo và xây dựng ứng dụng.

Đặc biệt, em xin gửi lời cảm ơn chân thành đến giảng viên hướng dẫn đã dành thời gian định hướng, góp ý và hỗ trợ em trong suốt quá trình nghiên cứu, xây dựng và hoàn thiện báo cáo. Những ý kiến đóng góp của thầy/cô đã giúp em hoàn thiện đề tài một cách tốt hơn.

Em cũng xin cảm ơn gia đình và bạn bè đã luôn động viên, chia sẻ và hỗ trợ em trong quá trình học tập cũng như trong thời gian thực hiện đề tài.

Mặc dù em đã cố gắng hoàn thành đề tài với tinh thần nghiêm túc, nhưng do kiến thức và kinh nghiệm thực tiễn còn hạn chế nên báo cáo khó tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp từ quý thầy cô để đề tài được hoàn thiện hơn.

Cuối cùng, em xin kính chúc quý thầy cô luôn dồi dào sức khỏe, hạnh phúc và đạt nhiều thành công trong công tác giảng dạy, nghiên cứu.

Em xin chân thành cảm ơn!

TÓM TẮT ĐỒ ÁN

Đề tài “Ứng dụng phát hiện và phân loại kích thước tôm trong chế biến thủy sản” tập trung xây dựng ứng dụng hỗ trợ tự động phát hiện, đo lường và phân loại kích thước tôm từ hình ảnh hoặc video. Hệ thống hướng đến việc giảm phụ thuộc vào thao tác thủ công, nâng cao tốc độ xử lý và hỗ trợ dây chuyền chế biến thủy sản trong công đoạn phân loại sản phẩm.

Khóa luận nghiên cứu các kiến thức nền tảng về xử lý ảnh, thị giác máy tính và mô hình YOLOv8 để phát hiện đối tượng, tách vùng tôm và xác định kích thước. Ứng dụng cho phép nhận dữ liệu đầu vào là ảnh hoặc video, xử lý qua các bước phát hiện, đo chiều dài, phân loại kích thước theo nhóm và xuất kết quả phục vụ thống kê, đánh giá.

Kết quả đạt được là một hệ thống thử nghiệm có khả năng phát hiện và phân loại kích thước tôm tự động, góp phần minh chứng tính khả thi của việc ứng dụng trí tuệ nhân tạo trong chế biến thủy sản. Đề tài cũng tạo cơ sở để tiếp tục mở rộng dữ liệu, cải thiện độ chính xác và phát triển hệ thống theo hướng phù hợp hơn với môi trường sản xuất thực tế.

MỤC LỤC

CHƯƠNG 1. TỔNG QUAN VỀ VẤN ĐỀ NGHIÊN CỨU	1
1.1 Đặt vấn đề.....	1
1.2 Phương pháp nghiên cứu	1
1.3 Dữ liệu nghiên cứu	1
1.4 Mục tiêu và phạm vi nghiên cứu	2
1.5 Nội dung thực hiện	2
1.6 Bố cục đồ án	2
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT.....	3
2.1 Ảnh kỹ thuật số và dữ liệu video.....	3
2.1.1 Pixel, độ phân giải ảnh và hệ tọa độ ảnh	3
2.1.2 Ảnh màu và ảnh nhị phân.....	3
2.2 Mạng nơ-ron tích chập trong thị giác máy tính.....	3
2.2.1 Convolutional Neural Network và đặc trưng ảnh	3
2.2.2 Convolutional layer, activation function và Batch Normalization.....	3
2.2.3 Multiscale Features trong object detection và instance segmentation	4
2.3 Object detection và instance segmentation	4
2.3.1 Object detection và bounding box.....	4
2.3.2 Instance segmentation và segmentation mask.....	4
2.3.3 Confidence, IoU và Non-Maximum Suppression	5
2.4 YOLOv8 trong phát hiện và phân đoạn đối tượng.....	5
2.4.1 Kiến trúc tổng quan của YOLOv8.....	5
2.4.2 YOLOv8 Detect.....	6
2.4.3 YOLOv8 Segment	7
2.4.4 Vai trò kết hợp giữa hai mô hình trong bài toán đo tâm.....	8
2.5 Multi-Object Tracking và ByteTrack	8
2.5.1 Multi-Object Tracking.....	8
2.5.2 Tracking-by-detection.....	9
2.5.3 Liên kết dữ liệu trong ByteTrack	9
2.6 Xử lý segmentation mask và trích xuất hình dạng tâm.....	9
2.6.1 Connected component	9
2.6.2 Skeleton và medial axis	9
2.6.3 Đồ thị tám lân cận và BFS trong tìm đường dài nhất.....	10

2.7 Quy đổi chiều dài và hiệu chuẩn hệ số quy đổi.....	10
2.7.1 Chiều dài theo pixel và chiều dài thực	10
2.7.2 Hệ số quy đổi trong quy đổi đơn vị đo	11
2.7.3 Phương pháp bình phương tối thiểu thông thường trong hiệu chuẩn.....	11
2.8 Chỉ số đánh giá mô hình và sai số đo lường.....	12
2.8.1 Precision, Recall và F1-score	12
2.8.2 AP, mAP50 và mAP50-95	13
2.8.3 MAE, RMSE, MAPE	13
2.8.4 FPS và thời gian xử lý	14
CHƯƠNG 3. CÔNG CỤ VÀ NỀN TẢNG SỬ DỤNG	15
3.1 Python và Conda.....	15
3.2 Roboflow	15
3.3 Kaggle.....	15
3.4 Ultralytics YOLOv8	15
3.5 OpenVINO.....	16
3.6 OpenCV	16
3.7 Flask và giao diện web	16
3.8 Các thư viện hỗ trợ xử lý dữ liệu.....	16
CHƯƠNG 4. HIỆN TRẠNG.....	18
4.1 Hiện trạng phân loại tôm trong chế biến thủy sản.....	18
4.1.1 Nhu cầu phân loại kích cỡ tôm.....	18
4.1.2 Phân loại thủ công	18
4.1.3 Phân loại bán tự động theo trọng lượng	18
4.1.4 Phân loại tự động bằng cơ cấu cơ khí	19
4.1.5 Phân loại tự động dựa trên hình ảnh và cảm biến	20
4.1.6 Định hướng đề tài từ hiện trạng phân loại.....	21
CHƯƠNG 5. DATASET, HUẤN LUYỆN VÀ ĐÁNH GIÁ MÔ HÌNH.....	22
5.1 Xây dựng dataset	22
5.1.1 Nguồn dữ liệu	22
5.1.2 Tiêu chí chọn lọc ảnh.....	22
5.1.3 Gán nhãn dữ liệu.....	22
5.1.4 Cấu trúc nhãn cho phát hiện và phân đoạn.....	23
5.1.5 Tăng cường dữ liệu.....	24

5.1.6 Chia dataset	25
5.1.7 Test set đánh giá mô hình	25
5.2 Huấn luyện mô hình	27
5.2.1 Cấu hình huấn luyện trên Kaggle	27
5.2.2 Kết quả huấn luyện mô hình phát hiện.....	27
5.2.3 Kết quả huấn luyện mô hình phân đoạn	30
5.3 Đánh giá mô hình	37
5.3.1 Đánh giá mô hình phát hiện trên test set	37
5.3.2 Đánh giá mô hình phân đoạn trên test set	41
5.3.3 So sánh và lựa chọn mô hình sử dụng.....	47
CHƯƠNG 6. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....	49
6.1 Tổng quan hệ thống đo và phân loại kích cỡ tôm	49
6.1.1 Mục tiêu của hệ thống	49
6.1.2 Dữ liệu đầu vào và đầu ra.....	49
6.1.3 Bảng tham số cấu hình	49
6.1.4 Các thành phần chính của hệ thống.....	51
6.2 Kiến trúc xử lý tổng thể của hệ thống	53
6.2.1 Luồng xử lý từ dữ liệu đầu vào đến kết quả đầu ra.....	53
6.2.2 Kiến trúc pipeline đa luồng	53
6.2.3 Lưu đồ pipeline F1 đến F6	54
6.2.4 Cơ chế ghi log và kiểm soát lỗi	54
6.3 Thiết kế các flow xử lý	55
6.3.1 F1 - Đọc dữ liệu đầu vào	55
6.3.2 F2 - Phát hiện và tracking.....	56
6.3.3 F3 - Chọn frame đại diện.....	58
6.3.4 F4 - Segmentation vùng tôm	59
6.3.5 F5 - Đo chiều dài bằng skeleton.....	60
6.3.6 F6 - Quy đổi, phân loại và lưu kết quả.....	61
6.4 Các lựa chọn thiết kế chính	62
6.4.1 Kết hợp YOLOv8 Detect và YOLOv8 Segment.....	62
6.4.2 Chọn frame đại diện trước khi segmentation	62
6.4.3 Đo chiều dài trên skeleton thay vì bounding box.....	62
6.4.4 Hiệu chuẩn scale factor bằng nhiều mẫu.....	62

6.4.5 Tổ chức kết quả theo run và hỗ trợ export	63
CHƯƠNG 7. XÂY DỰNG ỨNG DỤNG VÀ KIỂM THỬ VẬN HÀNH	64
7.1 Xây dựng pipeline xử lý	64
7.1.1 Tổ chức chương trình chính	64
7.1.2 Cài đặt F1 đọc dữ liệu đầu vào	64
7.1.3 Cài đặt F2 phát hiện và tracking	65
7.1.4 Cài đặt F3 chọn frame đại diện	67
7.1.5 Cài đặt F4 segmentation vùng tôm	70
7.1.6 Cài đặt F5 đo chiều dài bằng skeleton	72
7.1.7 Cài đặt F6 quy đổi và lưu kết quả	74
7.1.8 Cấu hình suy luận bằng OpenVINO	76
7.2 Xây dựng giao diện web	77
7.2.1 Flask và API vận hành	77
7.2.2 Giao diện nạp dữ liệu và chạy pipeline	77
7.2.3 Giao diện cấu hình tham số xử lý	78
7.2.4 Giao diện kết quả và hiệu chuẩn scale factor	79
7.2.5 Giao diện log, ảnh minh họa và bảng kích cỡ	82
7.3 Kiểm thử kết quả đo chiều dài	84
7.3.1 Tập hiệu chuẩn scale factor	84
7.3.2 Các tập đánh giá	85
7.3.3 Kết quả chi tiết theo từng tập kiểm thử	87
7.3.4 Nhận xét chung về hệ thống	95
7.4 Kiểm thử hiệu năng xử lý video	95
7.4.1 FPS xử lý của hai tập video	95
7.4.2 So sánh giữa hai video	96
CHƯƠNG 8. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	97
8.1 Kết quả đạt được	97
8.2 Hạn chế của hệ thống hiện tại	97
8.3 Hướng phát triển	98
TÀI LIỆU THAM KHẢO	100

DANH MỤC HÌNH VẼ, ĐỒ THỊ

Hình 2.1. Kiến trúc YOLOv8 Detect [6]	6
Hình 2.2. Kiến trúc YOLOv8 Segment [7]	7
Hình 4.1. Máy phân cỡ theo trọng lượng [27].....	19
Hình 4.2. Máy phân cỡ tôm bằng con lăn [28].....	20
Hình 4.3. Máy phân cỡ tôm thông minh dùng camera và laser [31].....	21
Hình 5.1. Minh họa vùng gán nhãn tôm.....	23
Hình 5.2. Ảnh trong dataset huấn luyện YOLOv8n detection	23
Hình 5.3. Ảnh trong dataset huấn luyện YOLOv8n segmentation.....	24
Hình 5.4. Mẫu trong test set với tôm nằm thẳng có thước bên cạnh.....	26
Hình 5.5. Mẫu trong test set với tôm nằm cong	26
Hình 5.6. Mẫu trong test set với nhiều con tôm nằm cạnh nhau	26
Hình 5.7. Biểu đồ kết quả huấn luyện YOLOv8n detection	28
Hình 5.8. Đường cong Precision-Recall của YOLOv8n detection trong quá trình huấn luyện	29
Hình 5.9. Đường cong F1-score của YOLOv8n detection.....	29
Hình 5.10. Confusion matrix của YOLOv8n detection.....	30
Hình 5.11. Biểu đồ kết quả huấn luyện YOLOv8n segmentation	31
Hình 5.12. Đường cong Box Precision-Recall của YOLOv8n segmentation	32
Hình 5.13. Đường cong Box Precision theo confidence của YOLOv8n segmentation	32
Hình 5.14. Đường cong Box Recall theo confidence của YOLOv8n segmentation.....	33
Hình 5.15. Đường cong Mask Precision-Recall của YOLOv8n segmentation.....	34
Hình 5.16. Đường cong Mask Precision theo confidence của YOLOv8n segmentation	34
Hình 5.17. Đường cong Mask Recall theo confidence của YOLOv8n segmentation	35
Hình 5.18. Đường cong Mask F1 của YOLOv8n segmentation	36
Hình 5.19. Confusion matrix của YOLOv8n segmentation	36
Hình 5.20. Đường cong Precision-Recall của YOLOv8n detection trên test set	38
Hình 5.21. Đường cong F1-score của YOLOv8n detection trên test set.....	38
Hình 5.22. Đường cong Precision theo confidence của YOLOv8n detection trên test set.....	39
Hình 5.23. Đường cong Recall theo confidence của YOLOv8n detection trên test set.....	40
Hình 5.24. Confusion matrix của YOLOv8n detection trên test set	40

Hình 5.25. Đường cong Box Precision-Recall của YOLOv8n segmentation trên test set	42
Hình 5.26. Đường cong Box Precision theo confidence của YOLOv8n segmentation trên test set	43
Hình 5.27. Đường cong Box Recall theo confidence của YOLOv8n segmentation trên test set	43
Hình 5.28. Đường cong Mask Precision-Recall của YOLOv8n segmentation trên test set	44
Hình 5.29. Đường cong Mask Precision theo confidence của YOLOv8n segmentation trên test set	45
Hình 5.30. Đường cong Mask Recall theo confidence của YOLOv8n segmentation trên test set	45
Hình 5.31. Đường cong Mask F1 của YOLOv8n segmentation trên test set	46
Hình 5.32. Confusion matrix của YOLOv8n segmentation trên test set	47
Hình 6.1. Lưu đồ tổng quan hệ thống đo và phân loại kích cỡ tôm	52
Hình 6.2. Lưu đồ pipeline từ F1 đến F6	54
Hình 6.3. Lưu đồ đọc dữ liệu đầu vào trong F1	55
Hình 6.4. Lưu đồ phát hiện và tracking trong F2	57
Hình 6.5. Lưu đồ chọn frame đại diện trong F3	58
Hình 6.6. Lưu đồ tạo mask trong F4	59
Hình 6.7. Lưu đồ đo chiều dài bằng skeleton trong F5	60
Hình 6.8. Lưu đồ quy đổi, phân loại và lưu kết quả trong F6	61
Hình 7.1. Mã nguồn đọc dữ liệu đầu vào trong F1	65
Hình 7.2. Mã nguồn phát hiện và tracking trong F2	66
Hình 7.3. Mã nguồn kiểm tra vạch tham chiếu trong F3	69
Hình 7.4. Frame ứng viên khi tôm chạm vạch tham chiếu trong F3	70
Hình 7.5. Frame đại diện được chọn trong F3	70
Hình 7.6. Mã nguồn segmentation vùng tôm trong F4	71
Hình 7.7. Kết quả segmentation vùng tôm trong F4	72
Hình 7.8. Mask thu được sau segmentation trong F4	72
Hình 7.9. Mã nguồn đo chiều dài bằng skeleton trong F5	73
Hình 7.10. Skeleton được tạo từ mask trong F5	73
Hình 7.11. Đường dài nhất trên skeleton trong F5	74
Hình 7.12. Cấu trúc thư mục output sau khi pipeline hoàn thành	75
Hình 7.13. Cấu trúc file JSON kết quả sau khi xử lý	76

Hình 7.14. Kết quả đo chiều dài và phân loại trong F6.....	76
Hình 7.15. Giao diện tab dữ liệu dùng để nạp đầu vào và chạy pipeline	78
Hình 7.16. Giao diện tab cấu hình tham số xử lý	79
Hình 7.17. Giao diện tab kết quả hỗ trợ xem kết quả và hiệu chuẩn scale factor	81
Hình 7.18. Giao diện tính scale factor từ dữ liệu đo thực tế	82
Hình 7.19. Giao diện tab log dùng để theo dõi trạng thái xử lý	83
Hình 7.20. Giao diện xem ảnh minh họa kết quả xử lý.....	83
Hình 7.21. Giao diện tab kích cỡ dùng để cấu hình nhóm phân loại	84
Hình 7.22. Ảnh đại diện dataset hiệu chuẩn scale factor	85
Hình 7.23. Ảnh đại diện nhóm tôm thẳng một con	86
Hình 7.24. Ảnh đại diện nhóm tôm cong một con	86
Hình 7.25. Ảnh đại diện nhóm nhiều con nằm cạnh nhau	86

DANH MỤC BẢNG BIỂU

Bảng 5.1. Cấu hình augmentation	24
Bảng 5.2. Kết quả đánh giá YOLOv8n detection trên test set	37
Bảng 5.3. Kết quả bounding box của YOLOv8n segmentation trên test set	41
Bảng 5.4. Kết quả segmentation mask của YOLOv8n segmentation trên test set	41
Bảng 6.1. Các tham số cấu hình chính của hệ thống	49
Bảng 6.2. Các tham số cấu hình phân loại kích cỡ tôm	51
Bảng 7.1. Chỉ số tổng hợp theo từng tập kiểm thử	87
Bảng 7.2. Kết quả đo chi tiết tập tôm thẳng một con	88
Bảng 7.3. Kết quả đo chi tiết tập tôm cong một con	89
Bảng 7.4. Kết quả đo chi tiết tập nhiều con thứ nhất	91
Bảng 7.5. Kết quả đo chi tiết tập nhiều con thứ hai	91
Bảng 7.6. Kết quả đo chi tiết tập nhiều con thứ ba	92
Bảng 7.7. Kết quả đo chi tiết tập nhiều con thứ tư	92
Bảng 7.8. Kết quả đo chi tiết tập nhiều con thứ năm	93
Bảng 7.9. Kết quả đo chi tiết tập video thứ nhất	93
Bảng 7.10. Kết quả đo chi tiết tập video thứ hai	94
Bảng 7.11. Hiệu năng xử lý của hai tập video	95

LỜI MỞ ĐẦU

Trong quá trình học tập và nghiên cứu tại Trường Đại học Nha Trang, đồ án tốt nghiệp là cơ hội quan trọng để sinh viên hệ thống hóa kiến thức, rèn luyện kỹ năng chuyên môn và vận dụng những nội dung đã học vào giải quyết một vấn đề thực tiễn. Xuất phát từ nhu cầu tự động hóa trong lĩnh vực chế biến thủy sản, đề tài “Ứng dụng phát hiện và phân loại kích thước tôm trong chế biến thủy sản” được lựa chọn nhằm xây dựng một hệ thống hỗ trợ phát hiện, đo lường và phân loại kích thước tôm từ hình ảnh hoặc video.

Trong ngành chế biến thủy sản, tôm là sản phẩm có giá trị kinh tế cao và thường cần được phân loại theo kích thước để phục vụ sản xuất, đóng gói và xuất khẩu. Tuy nhiên, việc phân loại thủ công còn phụ thuộc nhiều vào kinh nghiệm của người lao động, dễ tốn thời gian và có thể phát sinh sai số. Vì vậy, việc ứng dụng thị giác máy tính và mô hình học sâu như YOLOv8 vào bài toán phát hiện, đo chiều dài và phân loại kích thước tôm có ý nghĩa thực tiễn trong việc nâng cao hiệu quả xử lý và hỗ trợ quá trình tự động hóa.

Đồ án tập trung nghiên cứu cơ sở lý thuyết về xử lý ảnh, phát hiện đối tượng, phân đoạn đối tượng và các phương pháp đo kích thước từ ảnh. Trên cơ sở đó, hệ thống được xây dựng để tiếp nhận ảnh hoặc video đầu vào, phát hiện tôm, trích xuất vùng đối tượng, đo chiều dài theo pixel, quy đổi sang đơn vị thực và phân loại theo nhóm kích thước. Kết quả thực hiện là nền tảng ban đầu cho việc tiếp tục mở rộng dữ liệu, cải thiện độ chính xác và phát triển hệ thống phù hợp hơn với môi trường sản xuất thực tế.

Do giới hạn về thời gian, dữ liệu và kinh nghiệm thực tiễn, đồ án khó tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý, nhận xét từ quý thầy cô để có thể tiếp tục hoàn thiện đề tài và nâng cao kiến thức chuyên môn của bản thân.

Repo dự án: <https://github.com/Maingocholang/DoKichThuocTom>

CHƯƠNG 1. TỔNG QUAN VỀ VẤN ĐỀ NGHIÊN CỨU

1.1 ĐẶT VẤN ĐỀ

Tôm là một trong những mặt hàng có giá trị kinh tế cao của ngành chế biến thủy sản. Trong quá trình sản xuất, tôm cần được phân loại theo kích thước để bảo đảm sự đồng đều của sản phẩm, phục vụ đóng gói và đáp ứng yêu cầu của thị trường.

Việc phân loại tôm hiện nay vẫn phụ thuộc nhiều vào con người hoặc các thiết bị chuyên dụng. Phương pháp thủ công tương đối đơn giản nhưng cần nhiều thời gian, chi phí nhân công và có thể xảy ra sai lệch khi số lượng tôm lớn. Các thiết bị phân loại tự động giúp tăng năng suất nhưng thường cần đầu tư và điều chỉnh để phù hợp với điều kiện sản xuất.

Sự phát triển của trí tuệ nhân tạo và thị giác máy tính tạo điều kiện ứng dụng hình ảnh vào quá trình nhận biết và phân loại sản phẩm. Trên cơ sở đó, đề tài ứng dụng mô hình YOLOv8 để phát hiện, đo chiều dài và phân loại kích cỡ tôm từ hình ảnh hoặc video.

1.2 PHƯƠNG PHÁP NGHIÊN CỨU

Kích thước tôm có thể được xác định bằng nhiều phương pháp như đo thủ công, cân khối lượng, sử dụng cơ cấu cơ khí hoặc xử lý hình ảnh. Mỗi phương pháp có ưu điểm và hạn chế riêng. Đo thủ công dễ thực hiện nhưng phụ thuộc vào người đo. Phương pháp cân hoặc cơ khí phù hợp với quy mô sản xuất nhưng cần thiết bị chuyên dụng. Xử lý hình ảnh có khả năng làm việc với nhiều nguồn dữ liệu và thuận lợi cho việc thống kê, lưu trữ kết quả.

Đề tài lựa chọn phương pháp xử lý hình ảnh kết hợp học sâu vì phù hợp với mục tiêu xây dựng ứng dụng trên máy tính. YOLOv8 được sử dụng để nhận biết tôm trong dữ liệu đầu vào. Kết quả nhận biết được kết hợp với các phương pháp xử lý ảnh để ước lượng chiều dài và phân loại tôm theo các nhóm kích cỡ.

Phương pháp nghiên cứu được thực hiện theo hướng ứng dụng và thực nghiệm. Các công việc chính gồm tìm hiểu cơ sở lý thuyết, chuẩn bị dữ liệu, huấn luyện mô hình, xây dựng ứng dụng và kiểm tra kết quả. Quá trình thử nghiệm giúp đánh giá khả năng nhận biết, đo chiều dài và vận hành của ứng dụng.

1.3 DỮ LIỆU NGHIÊN CỨU

Dữ liệu sử dụng trong đề tài gồm hình ảnh và video có chứa tôm. Dataset do đề tài tự xây dựng trong điều kiện mô phỏng có kiểm soát, sau đó được chọn lọc và gán nhãn để phục vụ quá trình huấn luyện và đánh giá mô hình. Các mẫu được lựa chọn cần thể hiện rõ đối tượng và có chất lượng đủ để quan sát.

Dữ liệu sau khi chuẩn bị được chia thành các nhóm phục vụ huấn luyện, kiểm tra và đánh giá. Việc phân chia giúp theo dõi quá trình học của mô hình và kiểm tra khả năng làm việc trên các mẫu không được sử dụng trong quá trình huấn luyện.

1.4 MỤC TIÊU VÀ PHẠM VI NGHIÊN CỨU

Mục tiêu của đề tài là xây dựng ứng dụng có khả năng phát hiện, đo chiều dài và phân loại kích cỡ tôm từ hình ảnh hoặc video. Ứng dụng hỗ trợ thống kê số lượng tôm theo từng nhóm kích cỡ, lưu trữ kết quả, lưu ảnh minh họa và cung cấp thông tin phục vụ kiểm tra chất lượng.

Đề tài đồng thời làm rõ quy trình chuẩn bị dữ liệu, huấn luyện mô hình, xử lý ảnh và triển khai ứng dụng trong một bài toán thực tế. Cách tiếp cận này giúp liên kết phần mô hình học sâu với phần phần mềm vận hành, từ đó tạo ra kết quả có thể lưu trữ và kiểm tra lại.

Phạm vi nghiên cứu tập trung vào xây dựng và thử nghiệm ứng dụng trên dữ liệu đã chuẩn bị. Sản phẩm được phát triển trong phạm vi thử nghiệm hiện tại và chưa triển khai trực tiếp trên dây chuyền chế biến thủy sản hoàn chỉnh.

1.5 NỘI DUNG THỰC HIỆN

Đề tài tập trung tìm hiểu các kiến thức liên quan đến xử lý ảnh, thị giác máy tính và mô hình YOLOv8. Trên nền tảng đó, dữ liệu ảnh tôm được thu thập, chọn lọc, gán nhãn và sử dụng để huấn luyện mô hình.

Sau khi có mô hình phù hợp, ứng dụng được xây dựng để tiếp nhận hình ảnh hoặc video, xử lý dữ liệu và hiển thị kết quả phân loại. Các chức năng thống kê, lưu trữ và xuất kết quả cũng được thực hiện để hỗ trợ việc theo dõi và đánh giá.

1.6 BỐ CỤC ĐỒ ÁN

Đồ án gồm 8 chương:

- Chương 1. Tổng quan về vấn đề nghiên cứu.
- Chương 2. Cơ sở lý thuyết.
- Chương 3. Công cụ và nền tảng sử dụng.
- Chương 4. Hiện trạng phân loại tôm trong chế biến thủy sản.
- Chương 5. Dataset, huấn luyện và đánh giá mô hình.
- Chương 6. Phân tích và thiết kế hệ thống.
- Chương 7. Xây dựng ứng dụng và kiểm thử vận hành.
- Chương 8. Kết luận và hướng phát triển.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1 ẢNH KỸ THUẬT SỐ VÀ DỮ LIỆU VIDEO

2.1.1 Pixel, độ phân giải ảnh và hệ tọa độ ảnh

Ảnh kỹ thuật số được biểu diễn dưới dạng một ma trận các pixel. Mỗi pixel lưu thông tin cường độ sáng hoặc màu tại một vị trí cụ thể trong ảnh. Độ phân giải ảnh thể hiện số lượng pixel theo chiều rộng và chiều cao, qua đó quyết định mức chi tiết hình ảnh có thể lưu giữ. Khi xử lý ảnh bằng máy tính, hệ tọa độ ảnh thường lấy góc trên bên trái làm gốc tọa độ, trục ngang biểu diễn vị trí cột và trục dọc biểu diễn vị trí hàng trong ma trận ảnh [1].

Trong các bài toán đo kích thước từ ảnh, pixel là đơn vị đo ban đầu của mọi phép tính hình học. Vị trí bounding box, segmentation mask, skeleton và đường trục đều được xác định trên lưới pixel. Vì vậy, chiều dài đo được trong ảnh chưa thể dùng trực tiếp như đơn vị vật lý nếu chưa có hệ số quy đổi giữa pixel và kích thước thực [1].

2.1.2 Ảnh màu và ảnh nhị phân

Trong đề tài, dữ liệu đầu vào được xử lý chủ yếu dưới dạng ảnh màu và ảnh nhị phân. Ảnh màu lưu nhiều channel tại mỗi pixel, nhờ đó giữ lại thông tin màu sắc, biên dạng và bối cảnh của từng frame hoặc ảnh tĩnh. Dạng ảnh này phù hợp cho các bước phát hiện đối tượng và phân đoạn bằng mô hình học sâu, vì mô hình cần khai thác cả đặc trưng hình dạng lẫn đặc trưng màu từ vùng quan sát [1].

Ảnh nhị phân là dạng ảnh trong đó mỗi pixel chỉ thuộc một trong hai nhóm, thường là vùng đối tượng và vùng nền. Trong bài toán phân đoạn, segmentation mask có thể được xem như ảnh nhị phân, trong đó vùng có giá trị dương biểu diễn phần tâm được mô hình dự đoán. Khi mask đã được đưa về dạng nhị phân, các bước connected component, xử lý hình thái, tạo skeleton và tính chiều dài đường trục có thể được thực hiện trực tiếp trên vùng đối tượng đã tách khỏi nền [2].

2.2 MẠNG NƠ-RON TÍCH CHẬP TRONG THỊ GIÁC MÁY TÍNH

2.2.1 Convolutional Neural Network và đặc trưng ảnh

Convolutional Neural Network (CNN) là kiến trúc học sâu được thiết kế phù hợp với dữ liệu có cấu trúc lưới như ảnh. Thay vì xử lý ảnh như một vector phẳng, CNN khai thác quan hệ không gian cục bộ giữa các pixel thông qua các phép convolution. Cách tổ chức này giúp mô hình học được các đặc trưng thị giác từ đơn giản đến phức tạp, từ cạnh và vùng màu đến hình dạng đối tượng [3].

Trong thị giác máy tính, đặc trưng ảnh không nhất thiết phải được thiết kế thủ công. CNN có thể học đặc trưng trực tiếp từ dữ liệu thông qua quá trình training. Các layer đầu thường phản ứng với mẫu cục bộ đơn giản, còn các layer sâu hơn có khả năng biểu diễn cấu trúc hình dạng và ngữ cảnh ở mức trừu tượng hơn [3].

2.2.2 Convolutional layer, activation function và Batch Normalization

Convolutional layer sử dụng nhiều filter để trượt trên ảnh hoặc bản đồ đặc trưng đầu vào. Mỗi filter học một kiểu đặc trưng riêng và tạo ra một bản đồ đặc trưng tương

ứng. Sau phép convolution, activation function đưa tính phi tuyến vào mô hình, giúp mạng biểu diễn được các quan hệ phức tạp hơn so với tổ hợp tuyến tính đơn thuần [3].

Batch Normalization chuẩn hóa phân phối đầu vào của layer theo từng mini-batch trong quá trình training. Cơ chế này giúp giảm sự thay đổi phân phối nội bộ giữa các layer, hỗ trợ quá trình tối ưu ổn định hơn và cho phép dùng learning rate phù hợp hơn trong nhiều kiến trúc học sâu [4].

Trong phạm vi đề tài, convolutional layer và Batch Normalization được trình bày như nền tảng để hiểu cách YOLOv8 trích xuất đặc trưng ảnh. Nội dung tính toán chi tiết của các layer này không phải trọng tâm, vì mục tiêu chính của hệ thống là phát hiện tôm, tách vùng thân tôm và đo chiều dài theo ảnh.

2.2.3 Multiscale Features trong object detection và instance segmentation

Đối tượng trong ảnh có thể xuất hiện ở nhiều kích thước khác nhau. Một mô hình phát hiện hoặc phân đoạn cần xử lý được cả vùng nhỏ, vùng vừa và vùng lớn. Đặc trưng đa tỉ lệ giúp mô hình kết hợp thông tin ở nhiều mức phân giải, qua đó cải thiện khả năng định vị đối tượng khi kích thước và tư thế thay đổi [5].

Trong kiến trúc YOLOv8, đặc trưng đa tỉ lệ được thể hiện qua cách backbone tạo đặc trưng từ ảnh đầu vào, neck kết hợp các đặc trưng ở nhiều mức phân giải, còn head sử dụng các đặc trưng đó để tạo dự đoán. Các sơ đồ kiến trúc YOLOv8 Detect và YOLOv8 Segment đều cho thấy vai trò của các khối trích xuất, tổng hợp và truyền đặc trưng trước khi sinh kết quả cuối cùng [5], [6], [7].

2.3 OBJECT DETECTION VÀ INSTANCE SEGMENTATION

2.3.1 Object detection và bounding box

Object detection là nhiệm vụ xác định vị trí và lớp của đối tượng trong ảnh. Kết quả thường được biểu diễn bằng bounding box, confidence và nhãn lớp. Bounding box cho biết vùng hình chữ nhật bao quanh đối tượng, giúp hệ thống giới hạn vùng quan tâm trước khi thực hiện các bước xử lý chi tiết hơn [5], [8].

Trong bài toán đo tôm, bounding box đóng vai trò khoanh vùng nhanh cá thể tôm trong ảnh hoặc frame video. Đây là thông tin đủ nhẹ để dùng cho tracking và giới hạn vùng quan tâm, nhưng chưa đủ để đo chiều dài thân tôm chính xác khi tôm cong, nghiêng hoặc có hình dạng không khớp với hình chữ nhật bao quanh [5].

2.3.2 Instance segmentation và segmentation mask

Instance segmentation là nhiệm vụ tách riêng vùng pixel của từng đối tượng cụ thể trong ảnh. Khác với object detection chỉ trả về bounding box, instance segmentation tạo ra segmentation mask bám theo vùng đối tượng. Mask này biểu diễn hình dạng chi tiết hơn, do đó phù hợp với các bài toán cần đo kích thước hoặc phân tích hình học [5].

Với tôm, segmentation mask giúp giữ lại vùng thân cần đo và loại bỏ phần background quanh vùng quan tâm. Khi mask đủ bám sát đối tượng, bước tạo skeleton có cơ sở hình học tốt hơn để ước lượng chiều dài theo đường cong thân thay vì đo theo cạnh của bounding box.

Trong quá trình xử lý, segmentation mask có thể được đưa về dạng nhị phân để phân biệt rõ vùng tôm và background. Dạng biểu diễn này giúp các bước connected

component, medial axis và đo chiều dài hoạt động trên vùng đối tượng đã được tách riêng.

2.3.3 Confidence, IoU và Non-Maximum Suppression

Confidence biểu diễn mức tin cậy của dự đoán đối tượng. Trong quá trình suy luận, các dự đoán có confidence thấp thường bị loại bỏ để giảm nhiễu. Intersection over Union đo mức giao nhau giữa hai vùng dự đoán hoặc giữa dự đoán và nhãn thật, thường được dùng trong đánh giá mô hình và trong các bước lọc dự đoán trùng nhau [8].

IoU giữa hai bounding box A và B được tính bằng công thức:

$$IoU(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

trong đó:

- $IoU(A, B)$ là mức giao nhau trên hợp giữa hai bounding box.
- $A \cap B$ là vùng giao nhau giữa hai bounding box.
- $A \cup B$ là vùng hợp của hai bounding box.
- $||$ là diện tích của vùng được xét.

Non-Maximum Suppression là kỹ thuật loại bỏ các bounding box trùng lặp bằng cách giữ lại dự đoán có confidence cao hơn và loại các dự đoán có IoU vượt ngưỡng với nó. Cơ chế này giúp đầu ra detection gọn hơn, tránh trường hợp cùng một đối tượng được biểu diễn bởi nhiều bounding box gần nhau [9].

2.4 YOLOV8 TRONG PHÁT HIỆN VÀ PHÂN ĐOẠN ĐỐI TƯỢNG

2.4.1 Kiến trúc tổng quan của YOLOv8

YOLOv8 là họ mô hình thị giác máy tính do Ultralytics phát triển, hỗ trợ nhiều tác vụ như object detection và instance segmentation. Kiến trúc YOLOv8 được tổ chức theo ba phần chính gồm backbone, neck và head. Backbone trích xuất đặc trưng từ ảnh đầu vào, neck tổng hợp đặc trưng đa tỉ lệ, còn head tạo dự đoán cuối cùng cho từng tác vụ [5].

Trong hai sơ đồ kiến trúc được sử dụng ở Hình 2.1 và Hình 2.2, Conv là khối xử lý cơ bản để trích xuất đặc trưng cục bộ và thay đổi số channel của bản đồ đặc trưng. C2f tiếp tục truyền và kết hợp đặc trưng qua các nhánh bên trong, còn SPPF mở rộng vùng cảm nhận bằng các tầng pooling trước khi đặc trưng được đưa sang phần tổng hợp [6], [7].

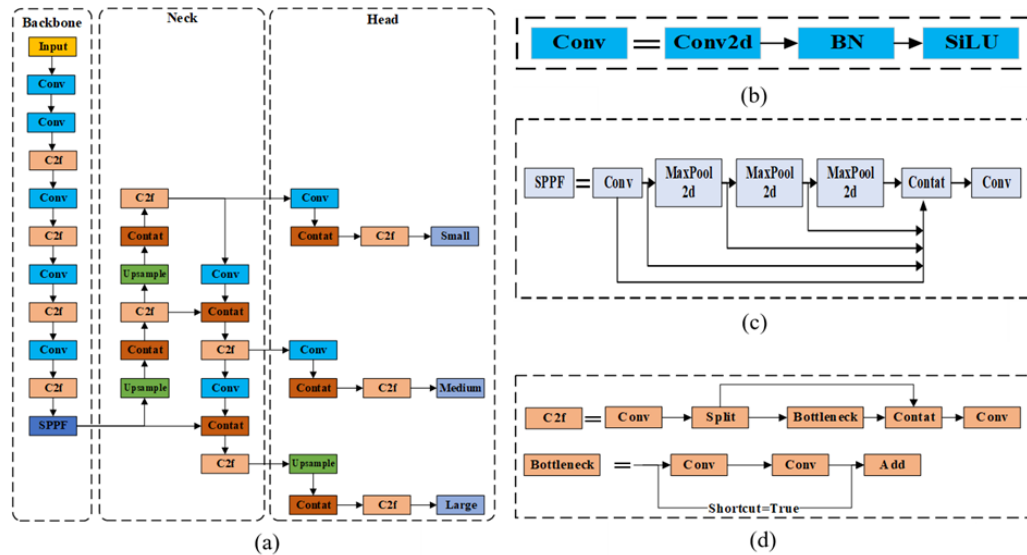
Ở phần neck, Upsample làm tăng kích thước bản đồ đặc trưng để ghép với đặc trưng ở tầng nông hơn, còn Concat nối các đặc trưng theo chiều channel. Cách kết hợp này giúp mô hình sử dụng đồng thời thông tin vị trí từ tầng có độ phân giải cao và thông tin ngữ nghĩa từ tầng sâu. Trong các sơ đồ, những nhánh đầu ra ở nhiều mức đặc trưng giúp mô hình xử lý đối tượng có kích thước quan sát khác nhau, phù hợp với dữ liệu tóm có tư thế, khoảng cách camera và vùng xuất hiện không cố định [6], [7].

2.4.2 YOLOv8 Detect

YOLOv8 Detect là biến thể dùng cho object detection. Mục tiêu của biến thể này là xác định đối tượng có xuất hiện trong ảnh hay không, đối tượng thuộc lớp nào và vùng chứa đối tượng nằm ở đâu. Đầu ra của mô hình gồm bounding box, confidence và lớp đối tượng. Bounding box cho biết vùng hình chữ nhật bao quanh đối tượng, confidence biểu diễn mức tin cậy của dự đoán, còn lớp đối tượng cho biết nhãn được mô hình gán cho vùng đó [5], [6].

Luồng xử lý của YOLOv8 Detect đi từ backbone, qua neck rồi đến Detect head. Backbone tạo đặc trưng ảnh ở nhiều mức trừu tượng, neck tổng hợp đặc trưng đa tỉ lệ, còn Detect head dùng các đặc trưng này để dự đoán vị trí và lớp của đối tượng. Trong sơ đồ YOLOv8 Detect, các nhánh đầu ra Small, Medium và Large thể hiện việc mô hình dự đoán trên nhiều mức đặc trưng khác nhau. Cách tổ chức này giúp mô hình có cơ sở xử lý các đối tượng có kích thước quan sát khác nhau trong ảnh [5], [6].

Hình 2.1 minh họa kiến trúc YOLOv8 Detect ở mức phục vụ trình bày thực nghiệm. Kiến trúc này cho thấy vai trò của các khối trích xuất đặc trưng, tổng hợp đặc trưng và head phát hiện trong việc tạo dự đoán đối tượng [6].



Hình 2.1. Kiến trúc YOLOv8 Detect [6]

Trong Hình 2.1, backbone nhận ảnh đầu vào và tạo các bản đồ đặc trưng qua chuỗi khối Conv, C2f và SPPF. Các tiểu hình bên phải làm rõ cấu trúc của những khối chính. Conv gồm Conv2d, Batch Normalization và SiLU. SPPF gồm Conv, các tầng MaxPool2d, Concat và Conv. C2f gồm Conv, Split, các Bottleneck, Concat và Conv. Nhờ các khối này, backbone vừa trích xuất đặc trưng cục bộ vừa bổ sung thông tin ngữ cảnh rộng hơn trước khi đưa đặc trưng sang neck [6].

Neck trong sơ đồ kết hợp các đặc trưng ở nhiều độ phân giải. Upsample làm tăng kích thước bản đồ đặc trưng để ghép với đặc trưng nông hơn, còn Concat nối các đặc trưng theo chiều channel. Head tạo ba nhánh dự đoán được ghi là Small, Medium và Large, qua đó hỗ trợ phát hiện đối tượng ở nhiều kích thước quan sát khác nhau [6].

Trong bài toán đo tâm, YOLOv8 Detect phù hợp cho bước xác định nhanh vùng chứa tâm. Với ảnh hoặc frame video đầu vào, mô hình chỉ cần trả về vùng bounding box

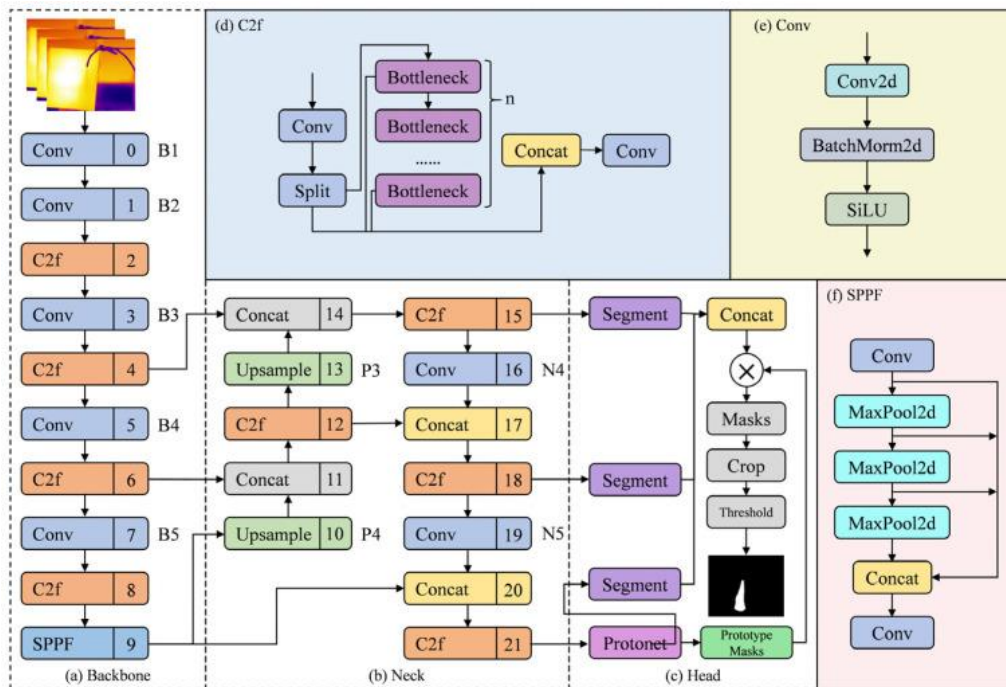
của từng cá thể tôm để hệ thống biết vị trí nào cần xử lý tiếp. Bounding box từ Detect giúp giới hạn phạm vi xử lý cho bước segmentation, đặc biệt khi ảnh có nhiều con tôm hoặc video có nhiều frame. Tuy nhiên, bounding box chỉ mô tả vùng bao ngoài theo hình chữ nhật nên chưa đủ để đo chính xác chiều dài thân tôm cong. Vì vậy, kết quả của YOLOv8 Detect được dùng chủ yếu để định vị, tạo ảnh đã che vùng ngoài bounding box và hỗ trợ bước xử lý tiếp theo, thay vì dùng trực tiếp làm dữ liệu đo chiều dài.

2.4.3 YOLOv8 Segment

YOLOv8 Segment là biến thể dùng cho instance segmentation. Khác với object detection chỉ cần xác định vùng bao quanh đối tượng, instance segmentation cần xác định vùng pixel thuộc về từng đối tượng cụ thể. Vì vậy, ngoài thông tin bounding box, confidence và lớp đối tượng, YOLOv8 Segment còn tạo segmentation mask cho từng instance. Segmentation mask biểu diễn vùng đối tượng ở mức chi tiết hơn bounding box, nên phù hợp với các bài toán cần xử lý hình dạng đối tượng sau khi phát hiện [5], [7].

Hình 2.2 cho thấy YOLOv8 Segment kế thừa luồng trích xuất và tổng hợp đặc trưng của YOLOv8, đồng thời bổ sung nhánh phục vụ tạo mask. Phần backbone và neck vẫn đảm nhiệm việc tạo đặc trưng đa tỉ lệ từ ảnh đầu vào. Phần khác biệt nằm ở head, nơi mô hình vừa dự đoán thông tin gắn với từng instance, vừa tạo thông tin phục vụ sinh mask cho vùng đối tượng [5], [7].

Hình 2.2 minh họa kiến trúc YOLOv8 Segment. Phần Segment head cho phép mô hình tạo mask theo từng instance, nhờ đó đầu ra chứa thông tin hình dạng chi tiết hơn so với bounding box [7].



Hình 2.2. Kiến trúc YOLOv8 Segment [7]

Trong Hình 2.2, backbone và neck của YOLOv8 Segment vẫn giữ vai trò trích xuất và tổng hợp đặc trưng đa tỉ lệ tương tự YOLOv8 Detect. Sơ đồ thể hiện luồng đặc trưng qua các khối Conv, C2f, SPPF, Upsample và Concat. Trong phần neck, các nhân P3 và P4 được đặt cạnh các khối Upsample, còn N4 và N5 được đặt cạnh các khối Conv

trước khi đặc trưng đi sang các nhánh Segment. Điểm khác biệt chính nằm ở head, nơi các khối Segment và nhánh Protonet cùng tham gia tạo thông tin phục vụ segmentation mask [7].

Các khối Segment trong head dự đoán thông tin gắn với từng đối tượng, còn nhánh Protonet tạo Prototype Masks dùng chung. Thông tin từ các nhánh Segment được kết hợp với Prototype Masks để sinh mask cho từng instance. Sau đó, mask đi qua bước Crop theo vùng dự đoán và bước Threshold để thu được segmentation mask cuối cùng. Cơ chế này giúp YOLOv8 Segment tạo được vùng tâm chi tiết hơn bounding box, từ đó cung cấp đầu vào cho connected component, medial axis và đo chiều dài [7].

Trong hệ thống đo tâm, YOLOv8 Segment được dùng để lấy vùng thân tâm sau khi vùng quan tâm đã được giới hạn. Đầu ra mask giúp hệ thống tách phần thân tâm khỏi background, chuyển vùng thân sang dạng nhị phân và tiếp tục xử lý bằng connected component, medial axis và tìm đường trục đại diện. Cách dùng này phù hợp hơn việc đo trực tiếp trên bounding box, vì mask giữ lại hình dạng cong của thân tâm và tạo dữ liệu đầu vào gần với mục tiêu đo chiều dài theo đường thân.

2.4.4 Vai trò kết hợp giữa hai mô hình trong bài toán đo tâm

Trong giai đoạn thử nghiệm ban đầu, hệ thống sử dụng trực tiếp YOLOv8 segmentation phiên bản nano để lấy mask tâm. Phiên bản này có ưu điểm là nhẹ và tốc độ suy luận nhanh, nhưng chất lượng mask chưa ổn định trong các trường hợp tâm cong, nhiều cá thể nằm gần nhau hoặc vùng biên thân chưa rõ. Khi mask thiếu vùng đầu, đuôi hoặc bị dính nhiều, bước skeleton phía sau dễ tạo đường đo sai.

Sau đó, hệ thống thử nâng lên YOLOv8 segmentation phiên bản lớn hơn để cải thiện chất lượng mask. Kết quả phân đoạn tốt hơn, nhưng tốc độ xử lý giảm rõ rệt. Khi áp dụng cho video, mô hình segmentation phải xử lý nhiều frame liên tiếp và còn kết hợp với tracking, nên chi phí xử lý tăng mạnh hơn so với ảnh tĩnh.

Vì vậy, hướng kết hợp hai mô hình được lựa chọn để cân bằng giữa tốc độ và chất lượng đo. YOLOv8 Detect được dùng trước để định vị nhanh vùng chứa tâm bằng bounding box và hỗ trợ tracking trong video. Sau khi đã chọn được vùng hoặc frame cần đo, YOLOv8 Segment mới chạy trên vùng đã được giới hạn để lấy mask chi tiết. Cách tổ chức này giảm số lần và phạm vi chạy segmentation, đồng thời vẫn cung cấp mask đủ chi tiết cho bước skeleton hóa và đo chiều dài thân tâm.

2.5 MULTI-OBJECT TRACKING VÀ BYTETRACK

2.5.1 Multi-Object Tracking

Multi-Object Tracking là nhiệm vụ duy trì định danh của nhiều đối tượng qua chuỗi frame video. Mục tiêu của tracking không chỉ là phát hiện đối tượng ở từng frame, mà còn liên kết các phát hiện thuộc cùng một đối tượng theo thời gian. Kết quả tracking thường tạo ra Track ID để phân biệt các cá thể đang xuất hiện trong video [10].

Trong bài toán xử lý video tâm, tracking giúp hạn chế việc đo lặp lại cùng một con tâm ở nhiều frame liên tiếp. Khi mỗi cá thể có Track ID ổn định, hệ thống có thể chọn một hoặc một số frame đại diện cho cá thể đó thay vì đưa toàn bộ frame sang bước segmentation [10].

2.5.2 Tracking-by-detection

Tracking-by-detection là hướng theo dõi đối tượng dựa trên kết quả detection ở từng frame. Mỗi frame trước hết được xử lý bằng mô hình phát hiện, sau đó các bounding box được liên kết với track đang tồn tại hoặc tạo track mới. Cách tiếp cận này phù hợp khi mô hình detection có độ ổn định đủ tốt và đối tượng cần theo dõi có thể được biểu diễn bằng bounding box [10].

Với video tôm trên băng chuyền, tracking-by-detection phù hợp vì bước detection đã cung cấp bounding box của tôm trong từng frame. Thông tin vị trí này giúp hệ thống xác định sự di chuyển của cá thể qua vùng quan sát, từ đó chọn frame khi tôm nằm ở vùng phù hợp để đo [10].

2.5.3 Liên kết dữ liệu trong ByteTrack

ByteTrack là phương pháp Multi-Object Tracking dựa trên nguyên tắc liên kết dữ liệu giữa các detection box. Điểm đáng chú ý của ByteTrack là không chỉ dùng các detection có confidence cao, mà còn khai thác detection confidence thấp trong bước liên kết để giảm nguy cơ mất track khi đối tượng bị che khuất, mờ hoặc có dự đoán kém ổn định trong một số frame [10].

Cơ chế liên kết hai giai đoạn của ByteTrack giúp duy trì track tốt hơn trong các tình huống detection thay đổi theo thời gian. Đối với bài toán tôm di chuyển trong video, đặc điểm này có ích vì hình dạng quan sát được của tôm có thể thay đổi do tư thế, chuyển động và chất lượng frame [10].

2.6 XỬ LÝ SEGMENTATION MASK VÀ TRÍCH XUẤT HÌNH DẠNG TÔM

2.6.1 Connected component

Connected component là vùng gồm các pixel liên thông trong ảnh nhị phân. Khi segmentation mask có nhiều vùng rời rạc, mỗi vùng được xem như một component riêng. Phân tích connected component cho phép xác định diện tích, bounding box và thông tin thống kê của từng vùng [11].

Trong đề tài, sau khi YOLOv8 Segment tạo mask, hệ thống chọn detection có bounding box lớn nhất và lấy mask tương ứng. Nếu mask gồm nhiều component rời rạc, hệ thống giữ component có diện tích lớn nhất, thường được xem là vùng chính cần đo. Cách làm này giúp loại bỏ các vùng nhỏ không ổn định trước khi tạo skeleton [11].

Sau khi chọn vùng chính, mask được crop theo bounding box segmentation đã mở rộng để tạo vùng xử lý tập trung hơn. Crop mask này là đầu vào cho các bước làm sạch mask và trích xuất hình dạng ở phía sau.

Mask sau khi crop được xử lý bằng các phép toán hình thái. Opening giúp loại bỏ các vùng nhiễu nhỏ còn bám trên mask. Closing giúp lấp các khe hở nhỏ bên trong vùng đối tượng. Với bài toán đo tôm, bước này giúp vùng cần đo liền mạch hơn trước khi tạo skeleton, từ đó giảm khả năng sinh nhánh phụ hoặc đường trục không ổn định.

2.6.2 Skeleton và medial axis

Skeleton là biểu diễn mảnh của vùng đối tượng, giữ lại cấu trúc trung tâm phản ánh hình dạng chính của đối tượng. Với vùng tôm đã được phân đoạn, skeleton giúp

chuyển vùng hai chiều thành một tập pixel dạng đường để phục vụ tính chiều dài theo hình dạng thân [12].

Medial axis là phương pháp tạo skeleton dựa trên trục trung tâm của vùng đối tượng. Phương pháp này thường đi kèm thông tin khoảng cách từ điểm skeleton đến biên vùng, nhờ đó biểu diễn được cấu trúc trung tâm của hình dạng. Trong bài toán đo tâm cong, medial axis phù hợp hơn cách đo trực tiếp theo bounding box vì nó bám theo dạng cong của vùng thân [12].

2.6.3 Đồ thị tám lân cận và BFS trong tìm đường dài nhất

Sau khi có skeleton, mỗi pixel thuộc skeleton có thể được xem như một đỉnh của đồ thị. Hai đỉnh được nối với nhau khi hai pixel tương ứng nằm kề nhau trong vùng tám lân cận, gồm các hướng ngang, dọc và chéo. Cách biểu diễn này giúp chuyển skeleton từ một tập pixel rời rạc thành một đồ thị có thể duyệt để tìm đường trục đại diện [13].

Breadth-First Search (BFS) là thuật toán duyệt đồ thị theo từng lớp khoảng cách từ một đỉnh bắt đầu. Trong đề tài, BFS được dùng hai lần để tìm một đường trục đại diện có chiều dài lớn trên skeleton. Trước hết, hệ thống chọn một điểm bất kỳ trên skeleton và chạy BFS để tìm điểm xa nhất thứ nhất. Từ điểm này, hệ thống chạy BFS lần thứ hai để tìm điểm xa nhất thứ hai. Đường đi giữa hai điểm đó được chọn làm đường đo đại diện cho thân tôm [13].

Sau khi dựng được đường đi, chiều dài pixel được cộng theo từng bước liên tiếp trên đường đó. Mỗi bước ngang hoặc dọc giữa hai pixel lân cận được tính là 1 pixel. Mỗi bước chéo được tính là $\sqrt{2}$ pixel vì tương ứng với đường chéo của một ô pixel đơn vị. Cách tính có thể viết gọn như sau:

$$p = n_1 + \sqrt{2} \times n_2$$

trong đó:

- p là chiều dài đường trục trên ảnh, tính bằng pixel,
- n_1 là số bước ngang hoặc dọc trên đường skeleton,
- n_2 là số bước chéo trên đường skeleton.

2.7 QUY ĐỔI CHIỀU DÀI VÀ HIỆU CHUẨN HỆ SỐ QUY ĐỔI

2.7.1 Chiều dài theo pixel và chiều dài thực

Chiều dài thu được từ đường skeleton là chiều dài được tính trong hệ tọa độ ảnh, với đơn vị ban đầu là pixel. Giá trị này phản ánh độ dài đường trục của tôm trên ảnh, nhưng chưa trực tiếp biểu diễn chiều dài thật ngoài thực tế. Vì vậy, để phục vụ phân loại kích cỡ, hệ thống cần quy đổi chiều dài đo trên ảnh sang đơn vị milimet thông qua một hệ số liên hệ giữa ảnh và kích thước vật lý [1].

Việc quy đổi từ pixel sang milimet phụ thuộc vào điều kiện thu nhận ảnh như độ phân giải, khoảng cách camera, vùng quan sát và cách hiệu chuẩn. Khi các điều kiện này thay đổi, hệ số quy đổi cũng cần được cập nhật để kết quả đo phản ánh đúng kích thước thực của tôm [1], [14].

2.7.2 Hệ số quy đổi trong quy đổi đơn vị đo

Hệ số quy đổi cho biết mỗi pixel trên ảnh tương ứng với bao nhiêu milimet ngoài thực tế. Khi đã có chiều dài đường trục trên ảnh và hệ số quy đổi, chiều dài thực của tôm được tính bằng công thức:

$$L = p \times s$$

trong đó:

- L là chiều dài thực của tôm, tính bằng milimet (mm),
- p là chiều dài đường trục trên ảnh, tính bằng pixel,
- s là hệ số quy đổi, tính bằng milimet trên pixel (mm/px).

Công thức này giả định quan hệ tuyến tính giữa chiều dài trong ảnh và chiều dài thực trong vùng quan sát đã được hiệu chuẩn. Với bối cảnh camera cố định, hệ số quy đổi giúp hệ thống chuyển kết quả đo từ không gian ảnh sang đơn vị có ý nghĩa trong phân loại sản phẩm [14].

2.7.3 Phương pháp bình phương tối thiểu thông thường trong hiệu chuẩn

Khi có nhiều mẫu hiệu chuẩn với chiều dài thật đã biết, hệ số quy đổi có thể được ước lượng bằng Ordinary Least Squares. Mục tiêu của OLS là tìm hệ số sao cho tổng bình phương sai số giữa chiều dài thật và chiều dài dự đoán từ chiều dài đo trên ảnh là nhỏ nhất [14].

Phương pháp này phù hợp với bài toán hiệu chuẩn hệ số quy đổi vì quan hệ giữa chiều dài theo pixel và chiều dài thật được giả định là tuyến tính trong cùng một vùng quan sát. Nếu chỉ dùng một mẫu đơn lẻ, hệ số quy đổi có thể bị lệch khi mẫu đó có mask chưa đủ đầu đuôi hoặc skeleton chưa bám đúng trục thân. OLS tận dụng nhiều mẫu hiệu chuẩn cùng lúc, nhờ đó hệ số thu được ít phụ thuộc vào sai số riêng của từng mẫu hơn. Ngoài ra, mô hình được đặt không có hệ số chặn vì khi chiều dài theo pixel bằng 0 thì chiều dài thật tương ứng cũng phải bằng 0.

Trong trường hợp mô hình quy đổi không có hệ số chặn, quan hệ giữa chiều dài thật và chiều dài pixel có thể biểu diễn như sau:

$$L_i = p_i \times s + e_i$$

trong đó:

- L_i là chiều dài thực của mẫu thứ i , tính bằng milimet (mm),
- p_i là chiều dài skeleton của mẫu thứ i , tính bằng pixel,
- s là hệ số quy đổi cần ước lượng, tính bằng milimet trên pixel (mm/px),
- e_i là sai số đo của mẫu thứ i , tính bằng milimet (mm).

Với mô hình không có hệ số chặn, hệ số quy đổi tối ưu theo OLS được tính như sau:

$$s = \frac{\sum_{i=1}^n p_i L_i}{\sum_{i=1}^n p_i^2}$$

trong đó:

- s là hệ số quy đổi được ước lượng, tính bằng milimet trên pixel (mm/px),
- n là số mẫu dùng để hiệu chuẩn,
- p_i là chiều dài skeleton của mẫu thứ i , tính bằng pixel,
- L_i là chiều dài thực của mẫu thứ i , tính bằng milimet (mm).

OLS phù hợp với hiệu chuẩn hệ số quy đổi vì nó tận dụng nhiều mẫu đo thay vì chỉ dựa vào một mẫu đơn lẻ. Cách này giúp giảm ảnh hưởng của sai số cá biệt trong từng lần đo và tạo hệ số quy đổi ổn định hơn cho cùng một điều kiện camera [14].

2.8 CHỈ SỐ ĐÁNH GIÁ MÔ HÌNH VÀ SAI SỐ ĐO LƯỜNG

2.8.1 Precision, Recall và F1-score

Precision đo tỉ lệ dự đoán đúng trong số các dự đoán được mô hình giữ lại. Chỉ số này phản ánh mức độ ít sinh dự đoán sai của mô hình. Recall đo tỉ lệ đối tượng thật được mô hình phát hiện, phản ánh khả năng hạn chế bỏ sót đối tượng. Hai chỉ số này thường được tính dựa trên TP, FP và FN [8].

Precision và Recall được tính bằng công thức:

$$Precision = \frac{TP}{TP + FP}$$

trong đó:

- *Precision* là tỉ lệ dự đoán đúng trong số các dự đoán được giữ lại,
- TP là số dự đoán đúng,
- FP là số dự đoán sai.

$$Recall = \frac{TP}{TP + FN}$$

trong đó:

- *Recall* là tỉ lệ đối tượng thật được phát hiện,
- TP là số dự đoán đúng,
- FN là số đối tượng bị bỏ sót.

F1-score là trung bình điều hòa giữa Precision và Recall. Chỉ số này hữu ích khi cần nhìn cân bằng giữa khả năng phát hiện đúng và khả năng không bỏ sót. Trong bài toán tìm, Precision thấp có thể tạo nhiều vùng đo sai, còn Recall thấp có thể làm bỏ sót cả thể cần phân loại [8].

F1-score được tính bằng công thức:

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

trong đó:

- $F1$ là F1-score,
- *Precision* là tỉ lệ dự đoán đúng trong số các dự đoán được giữ lại,

- *Recall* là tỉ lệ đối tượng thật được phát hiện.

2.8.2 AP, mAP50 và mAP50-95

Average Precision đo diện tích dưới đường cong Precision-Recall cho một lớp đối tượng. Mean Average Precision là trung bình AP trên các lớp hoặc các điều kiện đánh giá được xét. Với bài toán chỉ có một lớp tôm, mAP vẫn được dùng để phản ánh chất lượng phát hiện hoặc phân đoạn theo chuẩn đánh giá của mô hình [8].

mAP50 tính AP tại ngưỡng IoU 0,50, còn mAP50-95 tính trung bình AP trên dải ngưỡng IoU từ 0,50 đến 0,95. mAP50 thường dễ đạt cao hơn vì yêu cầu khớp vùng thấp hơn, trong khi mAP50-95 nghiêm ngặt hơn và phản ánh tốt hơn chất lượng định vị đối tượng [8].

Trong đề tài, mô hình chỉ học một lớp đối tượng là tôm, nên không cần trình bày công thức trung bình mAP theo nhiều lớp. Các giá trị mAP được dùng theo kết quả đánh giá của Ultralytics để so sánh chất lượng YOLOv8 Detect và YOLOv8 Segment, thay vì tự tính lại diện tích dưới đường cong Precision-Recall trong nội dung chương này [8].

2.8.3 MAE, RMSE, MAPE

Mean Absolute Error (MAE) đo trung bình độ lớn sai số tuyệt đối giữa giá trị đo và giá trị thật. Root Mean Squared Error (RMSE) lấy căn bậc hai của trung bình bình phương sai số, nên nhạy hơn với các sai số lớn. Mean Absolute Percentage Error (MAPE) biểu diễn sai số theo phần trăm, giúp so sánh mức sai lệch tương đối giữa các mẫu có kích thước khác nhau [15].

Mean Absolute Error được tính bằng công thức:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

trong đó:

- *MAE* là sai số tuyệt đối trung bình,
- *n* là số mẫu đánh giá,
- y_i là chiều dài thực của mẫu thứ *i*,
- $\hat{y}_i$ là chiều dài dự đoán hoặc chiều dài đo được của mẫu thứ *i*.

Root Mean Squared Error được tính bằng công thức:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

trong đó:

- *RMSE* là căn sai số bình phương trung bình,
- *n* là số mẫu đánh giá,
- y_i là chiều dài thực của mẫu thứ *i*,

- $\hat{y}_i$ là chiều dài dự đoán hoặc chiều dài đo được của mẫu thứ i .

Mean Absolute Percentage Error được tính bằng công thức:

$$MAPE = \frac{100}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

trong đó:

- $MAPE$ là sai số phần trăm tuyệt đối trung bình,
- n là số mẫu đánh giá,
- y_i là chiều dài thực của mẫu thứ i ,
- $\hat{y}_i$ là chiều dài dự đoán hoặc chiều dài đo được của mẫu thứ i .

2.8.4 FPS và thời gian xử lý

Trong đánh giá hiệu năng xử lý video, FPS là tham số cho biết số frame trong một giây video hoặc số frame hệ thống có thể xử lý trong một giây. FPS nguồn phản ánh tốc độ ghi hình của video đầu vào. Ví dụ, video 30 FPS có 30 frame trong mỗi giây, nên video dài 33 giây có khoảng 990 frame và video dài 26 giây có khoảng 780 frame.

Bên cạnh FPS nguồn, hệ thống có thể được đánh giá bằng FPS xử lý tương đương. Chỉ số này cho biết tốc độ xử lý trung bình của quy trình xử lý trên toàn bộ video và được tính từ số frame ước lượng chia cho thời gian xử lý:

$$FPS_{xl} = \frac{N_f}{T}$$

trong đó:

- FPS_{xl} là tốc độ xử lý tương đương, tính bằng frame trên giây,
- N_f là số frame của video,
- T là thời gian xử lý toàn bộ video, tính bằng giây.

Nếu FPS_{xl} gần với FPS nguồn thì hệ thống có khả năng xử lý gần thời gian thực hơn. Nếu FPS_{xl} thấp hơn FPS nguồn, thời gian xử lý sẽ dài hơn thời lượng phát của video. Trong đề tài này, FPS được dùng cùng với thời gian xử lý để đánh giá khả năng vận hành quy trình xử lý trên dữ liệu video, đặc biệt khi video có độ phân giải 1080 x 1920 và tốc độ ghi hình 30 FPS.

CHƯƠNG 3. CÔNG CỤ VÀ NỀN TẢNG SỬ DỤNG

3.1 PYTHON VÀ CONDA

Python là ngôn ngữ lập trình chính được sử dụng để xây dựng ứng dụng đo chiều dài và phân loại kích cỡ tôm. Ngôn ngữ này hỗ trợ tổ chức các phần xử lý ảnh, suy luận mô hình, đo chiều dài, lưu cấu hình, đọc dữ liệu đầu vào và xuất kết quả trong cùng một hệ thống mã nguồn. Các thư viện được dùng trong đề tài cũng hoạt động trên nền Python, nhờ đó quá trình tích hợp giữa mô hình, xử lý dữ liệu và giao diện web được triển khai thống nhất.

Conda được dùng để quản lý môi trường chạy của đề tài. Việc tách môi trường giúp cố định interpreter, package và phiên bản thư viện cần thiết cho ứng dụng. Cách tổ chức này hạn chế xung đột giữa các thư viện xử lý ảnh, thư viện học sâu và thư viện web, đồng thời giúp quá trình chạy lại ứng dụng ổn định hơn trên máy phát triển [16].

3.2 ROBOFLOW

Roboflow là nền tảng hỗ trợ quản lý dataset cho các bài toán thị giác máy tính. Nền tảng này cung cấp các chức năng liên quan đến tổ chức ảnh, quản lý nhãn, tạo phiên bản dữ liệu và export dataset phục vụ training mô hình [17].

Đối với đề tài, Roboflow được dùng để quản lý hai nhóm dataset chính. Nhóm thứ nhất phục vụ YOLOv8 Detect với nhãn bounding box. Nhóm thứ hai phục vụ YOLOv8 Segment với nhãn segmentation mask. Việc tách riêng hai nhóm dataset giúp quá trình training và đánh giá từng mô hình rõ ràng hơn, đồng thời phù hợp với hướng xử lý gồm phát hiện vùng tôm trước rồi phân đoạn vùng thân tôm sau.

3.3 KAGGLE

Kaggle cung cấp môi trường notebook trực tuyến cho các bài toán phân tích dữ liệu và học máy [18]. Nền tảng này phù hợp với giai đoạn training mô hình vì người dùng có thể chuẩn bị mã huấn luyện, chạy validation và theo dõi kết quả trong cùng một notebook.

Trong quá trình thực hiện đề tài, Kaggle được dùng để huấn luyện và đánh giá hai mô hình YOLOv8n detection và YOLOv8n segmentation. Mô hình sau khi training được đưa về mã nguồn ứng dụng để phục vụ suy luận, đo chiều dài và phân loại kích cỡ tôm. Kaggle chỉ giữ vai trò nền tảng huấn luyện, không tham gia trực tiếp vào ứng dụng xử lý khi người dùng vận hành giao diện web.

3.4 ULTRALYTICS YOLOV8

Ultralytics là bộ công cụ được dùng để làm việc với họ mô hình YOLOv8. Công cụ này hỗ trợ các tác vụ training, validation, prediction và export mô hình, nhờ đó phù hợp với quy trình từ huấn luyện đến tích hợp mô hình vào ứng dụng [5].

Ultralytics được sử dụng cho hai nhiệm vụ chính. YOLOv8 Detect đảm nhiệm phát hiện vùng chứa tôm bằng bounding box. YOLOv8 Segment đảm nhiệm phân đoạn vùng thân tôm bằng segmentation mask sau khi vùng quan tâm đã được giới hạn. Ngoài ra, Ultralytics còn hỗ trợ export mô hình sang định dạng OpenVINO để chuẩn bị cho giai đoạn suy luận trong ứng dụng [19].

3.5 OPENVINO

OpenVINO được dùng làm nền tảng triển khai suy luận sau giai đoạn huấn luyện mô hình. Với mô hình Ultralytics, mô hình có thể được export từ định dạng PyTorch sang định dạng OpenVINO tối ưu hơn cho phần cứng Intel, qua đó phù hợp với nhu cầu chạy suy luận trong môi trường cục bộ [19].

Trong đề tài, YOLOv8 Detect và YOLOv8 Segment được huấn luyện trên môi trường Kaggle Notebook, sau đó được chuyển từ định dạng PyTorch sang OpenVINO để triển khai trên máy cục bộ sử dụng Intel Core i5 và Intel HD Graphics 520. Việc chuyển đổi này giúp pipeline vận hành bằng mô hình OpenVINO thay vì phụ thuộc trực tiếp vào trọng số PyTorch, đồng thời tận dụng tốt hơn nền tảng phần cứng Intel khi xử lý ảnh và video trong quá trình đo chiều dài tôm [19].

3.6 OPENCV

OpenCV là thư viện xử lý ảnh và video được sử dụng rộng rãi trong các hệ thống thị giác máy tính. Thư viện này hỗ trợ thao tác trên ảnh, truy cập dữ liệu pixel, đọc frame từ video và xử lý các dạng biểu diễn ảnh khác nhau [1].

Trong ứng dụng đo tôm, OpenCV được dùng cho các bước đọc ảnh, đọc video, xử lý vùng ảnh và lưu ảnh minh họa kết quả. Các thao tác trên ảnh và dữ liệu pixel được hỗ trợ bởi cấu trúc ảnh dạng ma trận [1]. Ảnh nhị phân được dùng trong các bước xử lý mask và có liên quan đến cơ chế phân ngưỡng ảnh [2]. Thư viện cũng hỗ trợ các hàm phân tích hình dạng như connected component, nhờ đó hệ thống có thể xử lý segmentation mask bị rời rạc trước khi đưa sang bước tạo skeleton và đo chiều dài [11]. Ngoài ra, OpenCV còn được dùng cho các phép morphology open và close để làm sạch mask, cũng như ghi các ảnh debug phục vụ kiểm tra kết quả trung gian.

3.7 FLASK VÀ GIAO DIỆN WEB

Flask là web framework dùng để xây dựng backend web bằng Python. Framework này cung cấp cơ chế tổ chức route, tiếp nhận request và trả response cho trình duyệt, phù hợp với ứng dụng cần kết nối giao diện người dùng với các chức năng xử lý phía sau [20].

Giao diện web của đề tài sử dụng HTML, CSS, JavaScript và Tailwind CSS v4. HTML đảm nhiệm cấu trúc nội dung, JavaScript phục vụ tương tác với backend và cập nhật dữ liệu trên trình duyệt, còn Tailwind CSS hỗ trợ trình bày giao diện bằng utility classes [21]. Nhóm công cụ này giúp người dùng nạp dữ liệu, chỉnh cấu hình, theo dõi log, xem kết quả và xuất báo cáo thông qua giao diện thay vì thao tác trực tiếp bằng lệnh.

3.8 CÁC THƯ VIỆN HỖ TRỢ XỬ LÝ DỮ LIỆU

NumPy được dùng để thao tác trên dữ liệu dạng mảng nhiều chiều, đặc biệt trong các bước xử lý ảnh, segmentation mask và dữ liệu trung gian của pipeline [22]. scikit-image được dùng cho hàm medial_axis, hỗ trợ trích xuất skeleton từ ảnh nhị phân để phục vụ đo chiều dài tôm [12].

Supervision được dùng để tổ chức kết quả phát hiện và hỗ trợ ByteTrack trong xử lý video [23]. Nhờ có cấu trúc lưu kết quả phát hiện và Track ID, ứng dụng có thể theo dõi cùng một con tôm qua nhiều frame trước khi chọn frame đại diện để đo.

OpenPyXL hỗ trợ đọc và ghi dữ liệu XLSX, phù hợp với chức năng xuất kết quả hoặc đọc dữ liệu hiệu chuẩn ở dạng bảng [24]. JSON được dùng để lưu cấu hình và kết quả xử lý theo cấu trúc có thể đọc lại bằng backend [25], còn CSV được dùng cho các thao tác nhập hoặc xuất dữ liệu dạng bảng đơn giản [26].

CHƯƠNG 4. HIỆN TRẠNG

4.1 HIỆN TRẠNG PHÂN LOẠI TÔM TRONG CHẾ BIẾN THỦY SẢN

4.1.1 Nhu cầu phân loại kích cỡ tôm

Phân loại kích cỡ là một bước quan trọng trong chế biến tôm vì sản phẩm sau phân nhóm cần có mức đồng đều nhất định về kích thước. Khi tôm được chia theo các nhóm cỡ rõ ràng, quá trình đóng gói, kiểm tra chất lượng và sắp xếp sản phẩm trở nên thuận lợi hơn. Ở góc độ vận hành, tiêu chí phân loại càng ổn định thì kết quả giữa các mẻ xử lý càng ít phụ thuộc vào cảm quan của từng người lao động.

Trong thực tế, tôm có thể khác nhau về chiều dài, độ cong, bề rộng thân và tư thế quan sát. Những khác biệt này làm cho việc đánh giá kích cỡ bằng mắt dễ bị dao động, đặc biệt khi số lượng sản phẩm lớn hoặc tốc độ xử lý cao. Vì vậy, các hướng tự động hóa phân loại cần hướng tới khả năng tạo ra giá trị đo rõ ràng, giảm phụ thuộc vào kinh nghiệm cá nhân và hỗ trợ quá trình ra quyết định theo ngưỡng cấu hình.

4.1.2 Phân loại thủ công

Phân loại thủ công là cách phân cỡ tôm dựa trên quan sát trực tiếp và kinh nghiệm của công nhân. Người thao tác thường nhìn hình dạng, kích thước thân tôm và độ đồng đều của từng nhóm để tách sản phẩm theo cỡ. Với quy mô nhỏ, cách làm này dễ triển khai vì không đòi hỏi thiết bị chuyên dụng phức tạp.

Hạn chế của phân loại thủ công nằm ở tính ổn định. Khi số lượng tôm tăng, người thao tác phải lặp lại cùng một quyết định trong thời gian dài nên tốc độ, mức tập trung và tiêu chí phân cỡ có thể thay đổi theo ca làm việc. Kết quả phân loại vì vậy phụ thuộc nhiều vào kinh nghiệm cá nhân, trong khi dây chuyền chế biến cần tiêu chí nhất quán hơn để bảo đảm số lượng và chất lượng đầu ra.

4.1.3 Phân loại bán tự động theo trọng lượng

Phân loại bán tự động theo trọng lượng thường dùng thiết bị cân để giảm một phần thao tác thủ công. Tôm được đưa qua bộ cân và giá trị khối lượng được dùng làm căn cứ tách cỡ. Một thiết bị phân cỡ theo trọng lượng được giới thiệu với cơ chế dùng loadcell để xác định khối lượng rồi phân nhóm sản phẩm theo các ngưỡng đã đặt, qua đó minh họa cho hướng phân loại dựa trên khối lượng [27].



Hình 4.1. Máy phân cỡ theo trọng lượng [27]

So với thao tác thủ công, cách phân loại theo trọng lượng giúp quá trình phân cỡ có căn cứ đo lường rõ hơn. Tuy nhiên, khối lượng không biểu diễn trực tiếp hình dạng của từng cá thể tôm. Hai con tôm có khối lượng gần nhau vẫn có thể khác nhau về chiều dài, độ cong hoặc bề rộng thân. Khi bài toán cần phân loại dựa trên kích thước hình học, chỉ dựa vào khối lượng có thể chưa đủ để mô tả đặc trưng cần quan tâm.

4.1.4 Phân loại tự động bằng cơ cấu cơ khí

Phân loại tự động bằng cơ cấu cơ khí thường dựa trên chuyển động của thiết bị để tách sản phẩm theo kích thước. Máy phân cỡ con lăn là một ví dụ minh họa cho hướng này. Thiết bị được giới thiệu với cơ cấu con lăn và khe hở nhằm chia sản phẩm thành các nhóm kích thước, đồng thời hỗ trợ người vận hành quan sát hoặc điều chỉnh cỡ trong quá trình làm việc [28].



Hình 4.2. Máy phân cỡ tôm bằng con lăn [28]

Máy con lăn phù hợp với xử lý theo luồng liên tục, nhưng tiêu chí phân loại gắn chặt với cấu trúc vật lý của thiết bị. Khi muốn thay đổi cách chia nhóm kích cỡ, hệ thống có thể cần điều chỉnh cơ khí hoặc cấu hình vận hành. Cách tiếp cận này có ưu điểm về tốc độ trong môi trường sản xuất, nhưng ít linh hoạt hơn khi tiêu chí phân loại cần thay đổi theo từng lô hàng hoặc yêu cầu kiểm tra khác nhau.

4.1.5 Phân loại tự động dựa trên hình ảnh và cảm biến

Hướng phân loại tự động dựa trên hình ảnh đưa camera và thuật toán xử lý vào quá trình nhận biết từng cá thể. Trong lĩnh vực thủy sản, thị giác máy tính đã được dùng để đo các thuộc tính như chiều dài, diện tích, thể tích, chiều rộng, chiều cao và hình dạng phục vụ phân loại, phân cỡ, đếm và nhận dạng loài [29].

Với tôm, các đặc trưng ảnh như diện tích, chu vi, chiều dài và chiều rộng đã được khai thác để xây dựng mô hình dự đoán khối lượng tôm bóc vỏ [30].

Một thiết bị phân cỡ tôm thông minh được giới thiệu theo hướng kết hợp Artificial Intelligence (AI), camera 3D và laser để hỗ trợ xác định kích thước, trọng lượng hoặc thể tích của từng con tôm trong quá trình phân loại [31].



Hình 4.3. Máy phân cỡ tôm thông minh dùng camera và laser [31]

Các hình ảnh thiết bị trong mục này được dùng nhằm minh họa các hướng phân loại đang được áp dụng hoặc thương mại hóa. Cơ sở khoa học cho hướng xử lý ảnh được tham khảo từ các nghiên cứu về thị giác máy tính trong thủy sản và tôm [29], [30]. Cách phân biệt này giúp nội dung vừa có liên hệ với thiết bị thực tế, vừa tránh dùng nguồn minh họa sản phẩm như nguồn học thuật chính.

Các hệ thống dựa trên hình ảnh cho thấy khả năng khai thác trực tiếp đặc trưng hình học của tôm. Dữ liệu ảnh không chỉ cho biết đối tượng có xuất hiện trong khung hình hay không, mà còn cho phép xác định vùng thân, tách đối tượng khỏi nền và đo các đặc trưng hình dạng. Đây là điểm phù hợp với bài toán cần đo chiều dài từng cá thể trước khi phân loại kích cỡ.

4.1.6 Định hướng đề tài từ hiện trạng phân loại

Các hướng phân loại hiện có cho thấy bài toán phân cỡ tôm có thể được giải quyết theo nhiều mức tự động hóa. Phân loại thủ công dễ triển khai nhưng kết quả phụ thuộc nhiều vào kinh nghiệm và sự ổn định của người lao động. Phân loại theo trọng lượng cho ra giá trị đo cụ thể hơn, tuy nhiên khối lượng không thể hiện trực tiếp chiều dài và hình dạng thân tôm. Trong khi đó, các cơ cấu phân loại tự động có khả năng xử lý liên tục, nhưng cách phân cỡ thường bị giới hạn bởi thiết kế cơ khí và tiêu chí đo cố định.

Với mục tiêu đo chiều dài tôm, hướng xử lý ảnh phù hợp hơn vì camera có thể ghi nhận trực tiếp hình dạng của từng cá thể. Từ ảnh hoặc video đầu vào, hệ thống có thể định vị tôm, tách vùng thân ra khỏi nền và tạo dữ liệu hình học phục vụ bước đo. Các bước đo chiều dài, quy đổi đơn vị và phân loại kích cỡ được trình bày ở các chương sau.

Từ các cơ sở trên, đề tài lựa chọn xây dựng pipeline xử lý ảnh trong điều kiện quan sát có kiểm soát. Cách tiếp cận này phù hợp với yêu cầu chuyển dữ liệu hình ảnh thành kết quả đo lường có thể kiểm tra lại, đồng thời vẫn cho phép mở rộng sang xử lý video. Tuy nhiên, khi triển khai trên dây chuyền thực tế, hệ thống cần tiếp tục được đánh giá thêm về tốc độ xử lý, điều kiện ánh sáng, hiện tượng tôm chổng lặn và độ ổn định của camera.

CHƯƠNG 5. DATASET, HUẤN LUYỆN VÀ ĐÁNH GIÁ MÔ HÌNH

5.1 XÂY DỰNG DATASET

5.1.1 Nguồn dữ liệu

Dataset phục vụ huấn luyện và đánh giá mô hình do đề tài tự xây dựng và gán nhãn. Dữ liệu được thu thập theo hướng mô phỏng môi trường quan sát có kiểm soát, bắt đầu từ hơn 30 con tôm được chụp trên nền trắng. Thiết bị chụp được đặt cách bề mặt đặt mẫu 31 cm và khoảng cách này được giữ cố định trong quá trình thu thập để duy trì tỉ lệ hình ảnh tương đối ổn định giữa các lần chụp.

Việc chọn nền trắng và ánh sáng ổn định nhằm làm nổi bật vùng thân tôm, giảm biến động màu sắc và hạn chế nhiễu từ nền ảnh. Cách thiết lập này không mô phỏng toàn bộ nhà xưởng, mà tập trung vào vùng nhìn của camera khi quan sát tôm trên bề mặt ít thay đổi. Dataset này được dùng để huấn luyện hai nhóm mô hình chính gồm YOLOv8n detection cho nhiệm vụ phát hiện tôm và YOLOv8n segmentation cho nhiệm vụ phân đoạn vùng thân tôm.

Dataset được xây dựng trong điều kiện mô phỏng nên kết quả cần được diễn giải theo đúng phạm vi dữ liệu đã có. Trong môi trường dây chuyền chế biến thực tế, tôm có thể di chuyển liên tục, chồng lấn, bị che khuất, thay đổi hướng đặt và chịu ảnh hưởng bởi ánh sáng công nghiệp. Vì vậy, dataset mô phỏng được dùng như bước xây dựng mô hình cơ sở trước khi mở rộng sang dữ liệu dây chuyền có nhiều biến động hơn.

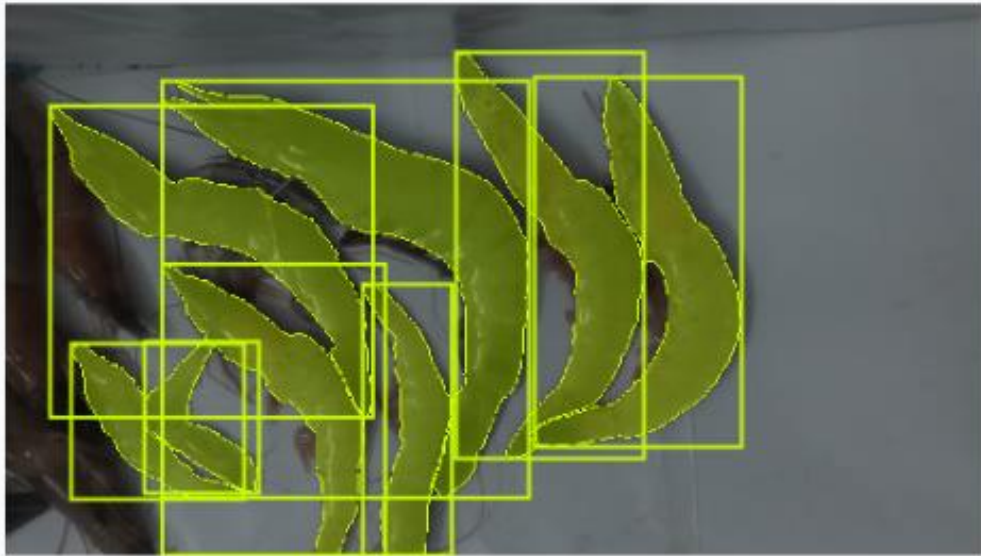
5.1.2 Tiêu chí chọn lọc ảnh

Ảnh được chọn theo tiêu chí ưu tiên vùng thân chính của tôm. Những con tôm nhìn thấy được từ đầu đến đuôi được ưu tiên gán nhãn vì cung cấp đủ thông tin hình dạng cho bài toán đo chiều dài. Các trường hợp tôm bị mất đầu, mất đuôi, bị che khuất nhiều hoặc không đủ thông tin hình dạng không được ưu tiên đưa vào nhãn, vì các mẫu này có thể làm mô hình học sai cấu trúc hình dạng cần đo.

Thân tôm thường có vùng trong mờ, màu không quá đậm và có nhiều chi tiết nhỏ như chân, râu. Nếu nền ảnh phức tạp hoặc vùng thân không rõ, mô hình có thể học nhầm chi tiết nền hoặc chi tiết phụ thành một phần của đối tượng. Vì vậy, quá trình chọn lọc ảnh tập trung vào các mẫu có vùng thân đủ rõ để phục vụ đồng thời nhiệm vụ detection và segmentation.

5.1.3 Gán nhãn dữ liệu

Quá trình gán nhãn được thực hiện theo nguyên tắc giữ vùng thân chính của tôm. Vùng nhãn được xây dựng theo hướng phục vụ đo chiều dài thân tôm thay vì cố mô tả toàn bộ chi tiết nhỏ của đối tượng.



Hình 5.1. Minh họa vùng gán nhãn tôm

Đối với nhãn detection, vùng nhãn cần bao quanh đầy đủ phần đầu, thân và đuôi, nhưng không cố mở rộng theo râu tôm. Đối với nhãn segmentation, vùng nhãn bám theo thân tôm, bỏ qua râu và hạn chế lấy phần chân tôm. Cách gán nhãn này giúp dữ liệu huấn luyện tập trung vào phân hình dạng có ý nghĩa đối với phép đo chiều dài.

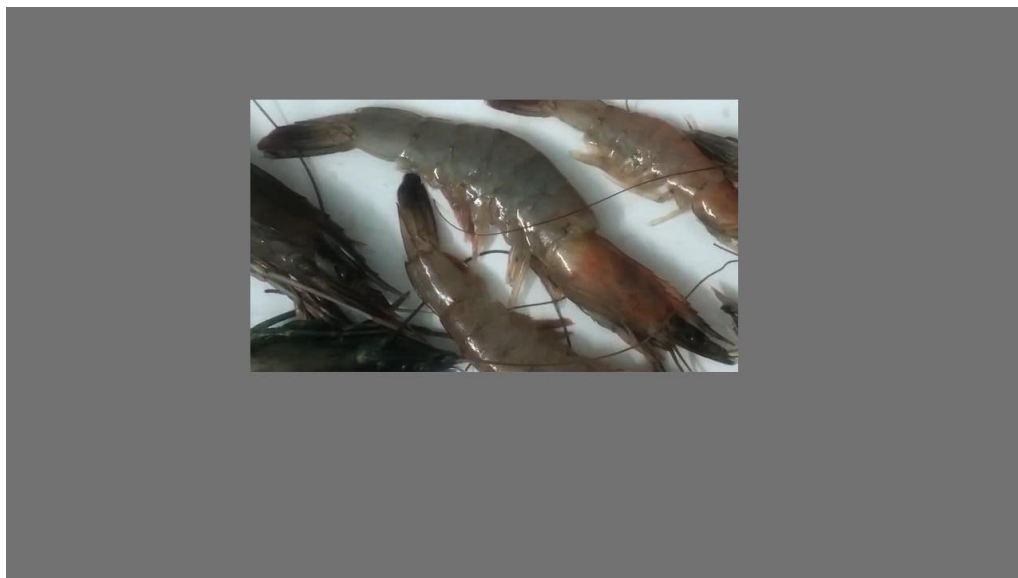
5.1.4 Cấu trúc nhãn cho phát hiện và phân đoạn

Dataset detection được quản lý trên Roboflow [32]. Nhãn của dataset này được biểu diễn bằng bounding box bao quanh vùng thân chính của tôm. Dataset detection giúp mô hình học cách xác định vị trí tôm trong ảnh bằng bounding box.



Hình 5.2. Ảnh trong dataset huấn luyện YOLOv8n detection

Dataset segmentation được hình thành từ tập ảnh ban đầu bằng cách crop bounding box của từng con tôm, sau đó đặt vùng crop trên nền xám. Nhãn của dataset này được biểu diễn bằng segmentation mask bám theo vùng thân tôm. Dataset segmentation được quản lý trên Roboflow [33] và giúp mô hình học vùng thân cần tách theo từng cá thể.



Hình 5.3. Ảnh trong dataset huấn luyện YOLOv8n segmentation

5.1.5 Tăng cường dữ liệu

Cả hai dataset detection và segmentation đều sử dụng cùng nhóm biến đổi augmentation. Các phép biến đổi này giúp mở rộng tính đa dạng của dataset mà không cần thu thêm nhiều ảnh mới.

Bảng 5.1. Cấu hình augmentation

Nhóm augmentation	Cấu hình sử dụng	Ý nghĩa
Flip	Horizontal, Vertical	Giúp mô hình học tôm ở các hướng đối xứng khác nhau
90° Rotate	Clockwise, Counter-Clockwise	Giúp mô hình thích nghi với tôm xoay theo nhiều hướng trong frame
Hue	Between -10° and +10°	Tạo biến đổi nhẹ về sắc độ màu
Saturation	Between -15% and +15%	Mô phỏng thay đổi độ bão hòa màu do điều kiện chụp
Brightness	Between -15% and +15%	Mô phỏng thay đổi độ sáng của ảnh
Exposure	Between -10% and +10%	Mô phỏng thay đổi phơi sáng khi chụp

Nhóm augmentation	Cấu hình sử dụng	Ý nghĩa
Blur	Up to 2.5px	Giúp mô hình chịu được mờ nhẹ do rung hoặc chuyển động
Noise	Up to 0.1% of pixels	Mô phỏng nhiễu nhẹ trên ảnh

Các phép flip và xoay 90 độ phù hợp với dữ liệu tôm vì tôm có thể xuất hiện ở nhiều hướng khác nhau. Các biến đổi màu sắc và ánh sáng giúp mô hình không phụ thuộc quá mức vào một điều kiện chụp duy nhất. Blur và noise giúp tăng khả năng chịu nhiễu trong trường hợp ảnh bị mờ nhẹ hoặc có điểm nhiễu trong quá trình ghi hình.

Augmentation không thay thế hoàn toàn dữ liệu thực tế. Các phép biến đổi này chỉ tạo thêm biến thể từ dữ liệu gốc. Nếu môi trường thật có nền, ánh sáng hoặc tốc độ chuyển động khác quá xa môi trường mô phỏng, mô hình vẫn cần được bổ sung thêm dữ liệu thực tế để tăng khả năng tổng quát.

5.1.6 Chia dataset

Dataset huấn luyện detection dùng cho mô hình YOLOv8n gồm 300 ảnh trong phiên bản dữ liệu trên Roboflow [32]. Phần dữ liệu dùng để training và validation gồm 210 ảnh thuộc training set và 90 ảnh thuộc validation set. Mỗi ảnh thuộc training set được tạo 3 phiên bản augmentation nhằm giúp mô hình học được nhiều biến thể hơn từ số lượng ảnh gốc còn hạn chế.

Dataset huấn luyện segmentation có 728 ảnh thuộc training set và 182 ảnh thuộc validation set. Mỗi ảnh thuộc training set được nhân 3 phiên bản augmentation. So với dataset huấn luyện detection, dataset huấn luyện segmentation có số lượng ảnh cá thể lớn hơn vì mỗi vùng crop tương ứng với một con tôm riêng biệt [33].

Việc tách training set và validation set giúp quá trình huấn luyện có dữ liệu để học tham số và dữ liệu riêng để theo dõi khả năng khái quát của mô hình. Validation set không được dùng như dữ liệu học trực tiếp trong quá trình cập nhật trọng số.

5.1.7 Test set đánh giá mô hình

Test set được xây dựng từ 47 con tôm khác nhau, không trùng với các mẫu dùng để huấn luyện. Các mẫu trong test set được chụp theo nhiều tình huống để kiểm tra khả năng hoạt động của mô hình khi tôm thay đổi tư thế và bối cảnh sắp đặt.



Hình 5.4. Mẫu trong test set với tôm nằm thẳng có thước bên cạnh



Hình 5.5. Mẫu trong test set với tôm nằm cong



Hình 5.6. Mẫu trong test set với nhiều con tôm nằm cạnh nhau

Ba nhóm ảnh trong test set chính gồm tôm nằm thẳng có thước bên cạnh, tôm nằm cong và nhiều con tôm nằm cạnh nhau. Trường hợp tôm nằm thẳng có thước giúp kiểm tra khả năng detection và segmentation trong điều kiện hình dạng rõ. Trường hợp tôm nằm cong phù hợp với bài toán đo chiều dài theo đường thân, vì chiều dài cần đi theo trục cong thay vì đo theo đường thẳng. Trường hợp nhiều con nằm cạnh nhau giúp kiểm tra khả năng tách từng cá thể và hạn chế nhầm lẫn giữa các đối tượng gần nhau.

Ngoài ảnh tĩnh, test set còn có 2 đoạn video quay tôm chạy mô phỏng trên băng chuyền. Dữ liệu video giúp đánh giá thêm khả năng phát hiện trong chuỗi frame, đặc biệt khi đối tượng thay đổi vị trí liên tục. Test set detection và test set segmentation được quản lý riêng trên Roboflow [34], [35].

5.2 HUẤN LUYỆN MÔ HÌNH

5.2.1 Cấu hình huấn luyện trên Kaggle

Hai mô hình được huấn luyện trên Kaggle. Notebook huấn luyện YOLOv8n detection được lưu tại [36]. Notebook huấn luyện YOLOv8n segmentation được lưu tại [37].

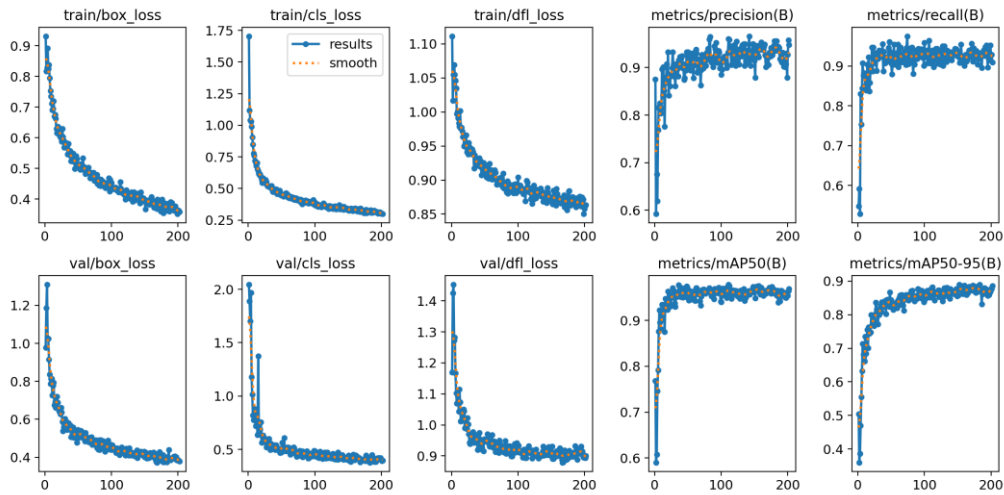
Mô hình detection được huấn luyện bằng YOLOv8n với trọng số PyTorch khởi tạo yolov8n.pt. Mô hình segmentation được huấn luyện bằng biến thể YOLOv8n-seg với trọng số PyTorch khởi tạo yolov8n-seg.pt. Dữ liệu huấn luyện của cả hai mô hình được cấu hình thông qua file data.yaml tương ứng với từng dataset trên Roboflow.

Quá trình huấn luyện chỉ điều chỉnh hai thiết lập chính là số epoch tối đa và patience. Số epoch tối đa được đặt là 500 để mô hình có đủ thời gian học từ dataset đã chuẩn bị. Patience được đặt là 50 để quá trình huấn luyện tự dừng khi kết quả trên validation set không còn cải thiện trong nhiều epoch liên tiếp. Các thiết lập còn lại như batch size, kích thước ảnh đầu vào, optimizer, siêu tham số tối ưu, trọng số loss và augmentation được giữ theo cấu hình training mặc định của Ultralytics. Cách thiết lập này giúp quá trình huấn luyện tập trung vào đánh giá dataset và mô hình, thay vì mở rộng sang bài toán tối ưu siêu tham số.

Trong quá trình huấn luyện, mô hình segmentation còn theo dõi thêm segmentation loss bên cạnh các thành phần loss của bounding box và phân lớp, vì đầu ra của mô hình cần có segmentation mask bám theo vùng thân tôm.

5.2.2 Kết quả huấn luyện mô hình phát hiện

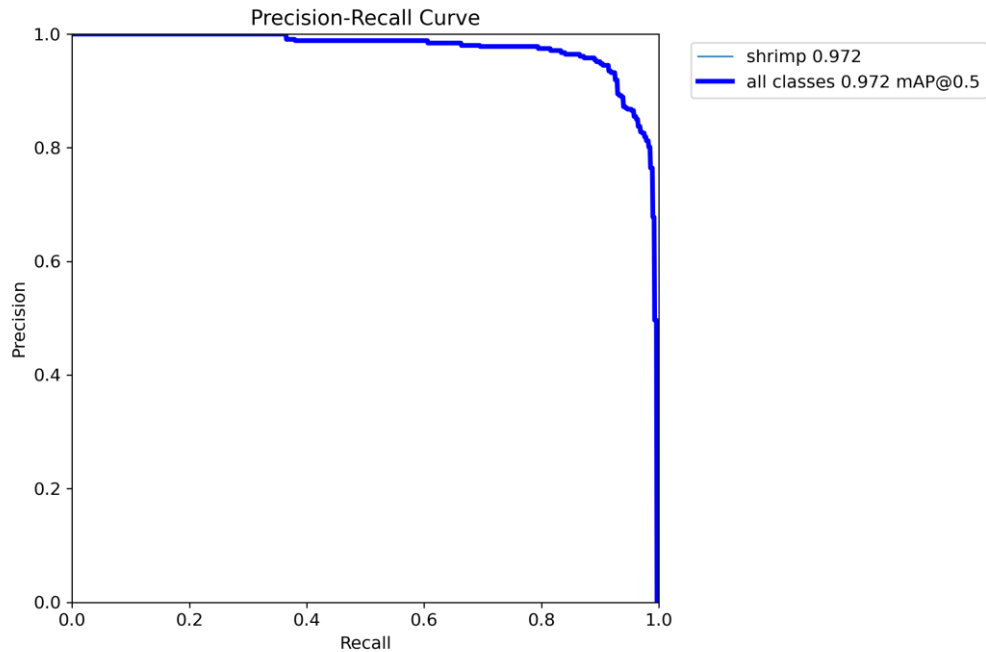
Kết quả huấn luyện YOLOv8n detection được đánh giá thông qua biểu đồ loss, các đường cong theo confidence và confusion matrix. Mô hình được thiết lập tối đa 500 epoch với patience là 50, cho phép quá trình huấn luyện có đủ thời gian đi từ giai đoạn học đặc trưng tổng quát đến giai đoạn tinh chỉnh vị trí bounding box.



Hình 5.7. Biểu đồ kết quả huấn luyện YOLOv8n detection

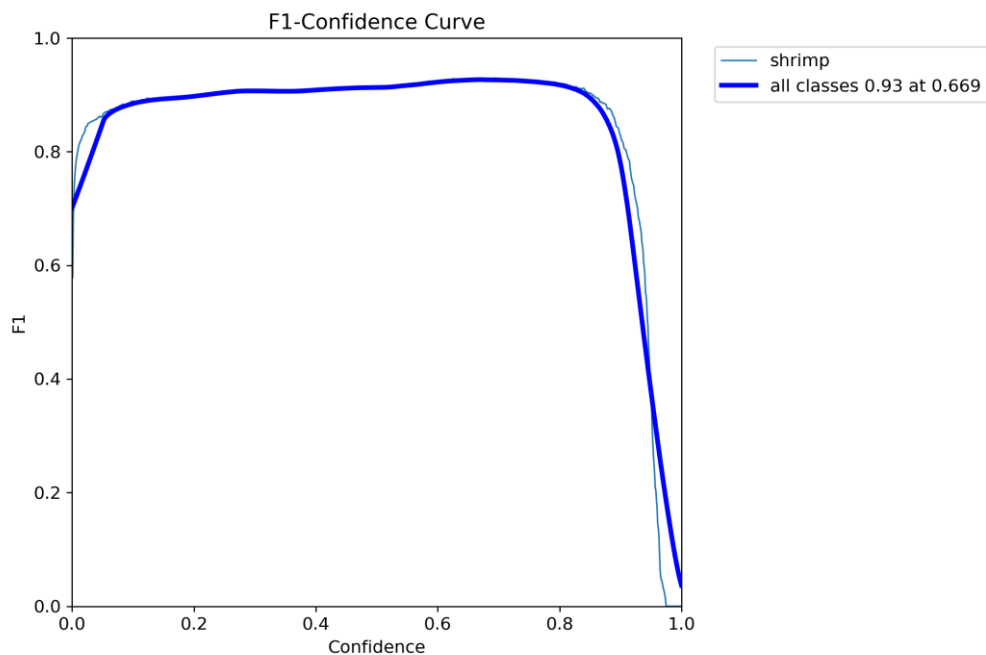
Biểu đồ kết quả huấn luyện ở Hình 5.7 có hình dáng đặc trưng của quá trình hội tụ. Các loss giảm dốc ở giai đoạn đầu vì mô hình nhanh chóng học được những dấu hiệu dễ nhận biết của tôm như vùng thân nổi trên nền sáng, hình dạng kéo dài và biên đối tượng tương đối rõ. Ở các epoch sau, đường loss giảm chậm hơn vì mô hình không còn chỉ học sự khác biệt tổng quát giữa tôm và nền, mà chuyển sang tinh chỉnh vị trí biên bounding box và các trường hợp khó hơn như tôm cong, biên mờ hoặc vùng gần thước đo.

Các đường Precision và Recall trong Hình 5.7 tăng nhanh sau vài epoch đầu vì bài toán chỉ có một lớp shrimp, nền ảnh tương đối ổn định và vùng đối tượng có hình dạng khác biệt so với nền. Sau giai đoạn tăng nhanh, các đường này dao động quanh vùng giá trị cao thay vì tăng đều tuyệt đối. Dạng dao động này xuất hiện vì từng epoch vẫn có thể thay đổi nhẹ ở các mẫu biên khó, nhưng đường làm mượt cho thấy xu hướng chung đã ổn định. mAP50 tăng nhanh và sớm đi vào vùng bão hòa vì ngưỡng IoU 0.50 cho phép bounding box chỉ cần khớp ở mức tổng quát. Ngược lại, mAP50-95 tăng chậm hơn vì dải ngưỡng này đòi hỏi biên bounding box khít hơn, nên nhạy với sai lệch nhỏ ở đầu, đuôi và vùng thân cong.



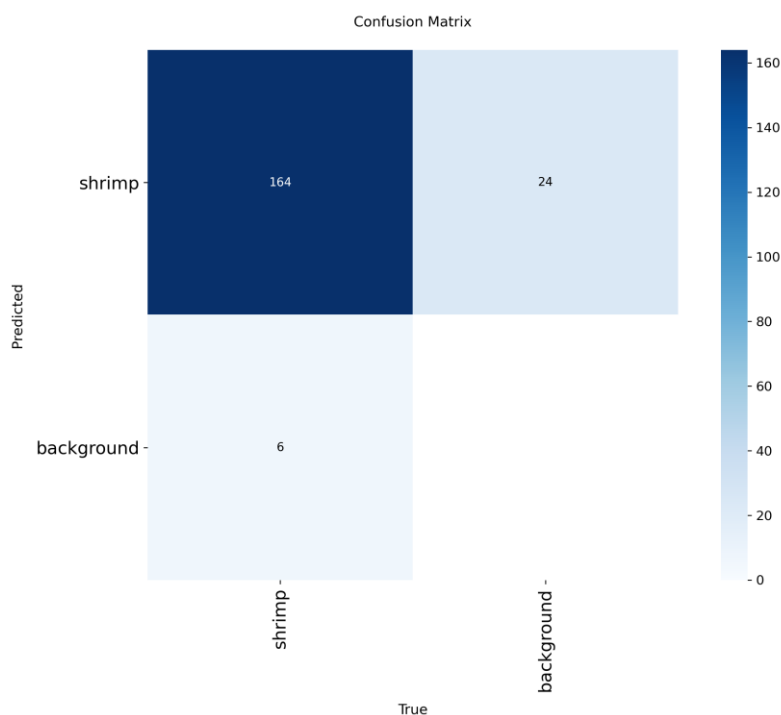
Hình 5.8. Đường cong Precision-Recall của YOLOv8n detection trong quá trình huấn luyện

Đường cong Precision-Recall ở Hình 5.8 nằm sát vùng trên của đồ thị và không rơi mạnh khi Recall tăng. Hình dáng này cho thấy mô hình vẫn giữ được nhiều dự đoán đúng ngay cả khi mở rộng dải phát hiện để thu đủ tôm. Kết quả này đạt được vì dữ liệu huấn luyện có nền kiểm soát, lớp cần phát hiện duy nhất là shrimp và bounding box tập trung vào vùng thân chính. Với vai trò giới hạn vùng quan tâm cho bước segmentation, đường cong như vậy là phù hợp vì detection cần giữ Recall cao để không bỏ sót tôm, đồng thời Precision vẫn phải đủ cao để tránh đưa vùng nền sai sang mô hình segmentation.



Hình 5.9. Đường cong F1-score của YOLOv8n detection

Đường cong F1-score ở Hình 5.9 có vùng cao tương đối rộng thay vì chỉ đạt đỉnh tại một điểm rất hẹp. Điều này cho thấy mô hình có khoảng confidence vận hành thuận lợi, nơi Precision và Recall được cân bằng. Khi confidence còn thấp, một số dự đoán chưa chắc chắn vẫn được giữ lại nên Precision có thể bị ảnh hưởng. Khi confidence tiến gần vùng quá cao, đường F1 giảm vì nhiều dự đoán của các trường hợp khó bị loại bỏ, làm Recall giảm. Vì vậy, hình dáng của đường F1 cho thấy ngưỡng confidence nên được chọn trong vùng ổn định, không nên đẩy quá sát 1.

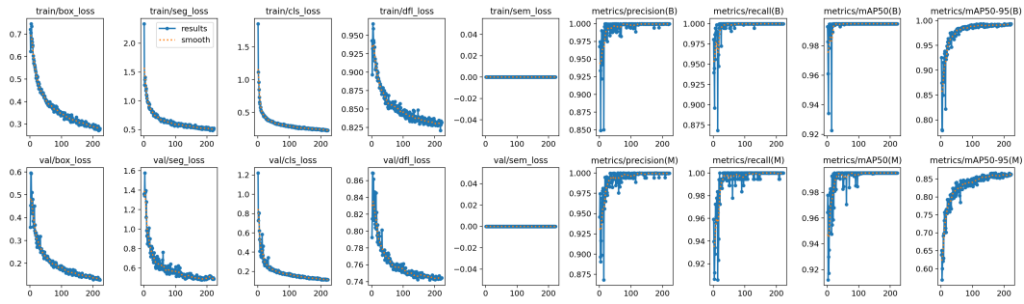


Hình 5.10. Confusion matrix của YOLOv8n detection

Confusion matrix ở Hình 5.10 có giá trị tập trung chủ yếu tại ô shrimp được dự đoán đúng. Hình dáng này phù hợp với bài toán một lớp, vì mô hình chủ yếu cần phân biệt vùng tôm với nền. Nếu còn giá trị ở ô nhầm giữa shrimp và background, phần đó thường xuất hiện ở những vùng có biên thân không rõ, chi tiết nhỏ gần râu hoặc vùng nền có hình dạng gây nhiễu. Do đó, confusion matrix cho thấy mô hình phân biệt lớp ổn, nhưng vẫn cần các bước sau để kiểm soát vùng quan tâm sai hoặc thiếu.

5.2.3 Kết quả huấn luyện mô hình phân đoạn

Kết quả huấn luyện YOLOv8n segmentation được đánh giá qua nhóm biểu đồ Box và nhóm biểu đồ Mask. Mô hình được thiết lập tối đa 500 epoch với patience là 50 để có đủ thời gian học đồng thời vị trí đối tượng và vùng segmentation mask.

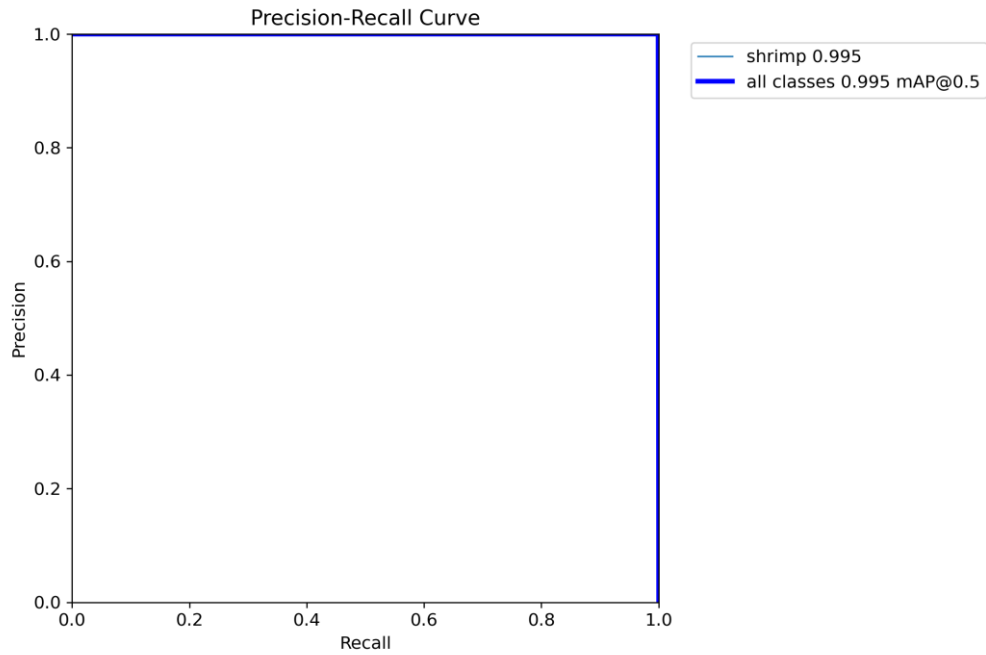


Hình 5.11. Biểu đồ kết quả huấn luyện YOLOv8n segmentation

Biểu đồ kết quả huấn luyện ở Hình 5.11 cho thấy train box loss, train seg loss, train cls loss và train dfl loss đều giảm theo epoch. Ở giai đoạn đầu, các đường loss giảm nhanh vì dataset segmentation đã được tạo từ các vùng crop theo bounding box, mỗi ảnh crop tập trung vào một con tôm trên nền xám và chỉ có một lớp đối tượng là shrimp. Cách tổ chức dữ liệu này làm cho mô hình nhanh chóng học được vùng thân tôm nổi bật so với nền. Sau đó, độ dốc giảm nhỏ dần vì phần còn lại của bài toán nằm ở các chi tiết khó hơn như biên thân trong mờ, vùng đầu và đuôi, phần chân hoặc râu dễ bị lẫn vào mask. Validation loss giảm cùng chiều với training loss cho thấy mô hình không chỉ ghi nhớ training set, mà còn giữ được xu hướng ổn định trên validation set.

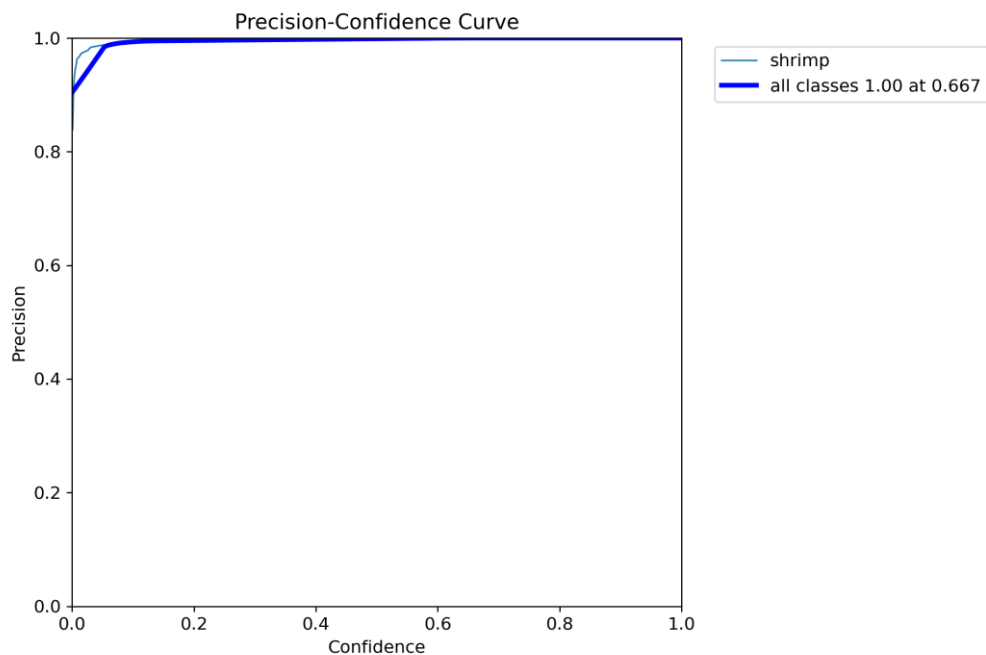
Nhóm chỉ số Box trong Hình 5.11 tăng rất nhanh và đạt vùng gần bão hòa vì mỗi ảnh segmentation đã tập trung vào một cá thể tôm, làm cho nhiệm vụ định vị bounding box trở nên rõ ràng hơn so với ảnh có nhiều đối tượng. mAP50 của bounding box ổn định ở mức cao từ sớm vì chỉ cần định vị đúng vùng tổng quát của tôm. mAP50-95 tăng chậm hơn vì chỉ số này yêu cầu bounding box khớp tốt hơn ở nhiều ngưỡng IoU chặt, nên mô hình cần thêm thời gian để tinh chỉnh ranh giới quanh đầu, đuôi và thân cong.

Nhóm chỉ số Mask cũng tăng nhanh nhưng mAP50-95 của Mask thấp hơn mAP50-95 của Box. Hình dáng này hợp lý vì bounding box chỉ cần bao quanh đối tượng, còn segmentation mask phải khớp theo từng vùng pixel của thân tôm. Với tôm, biên thân có thể trong mờ, phần chân và râu tạo nhiều chi tiết nhỏ, tư thế cong làm vùng mask phức tạp hơn. Vì vậy, mô hình đạt mAP50 cao ở mức khớp tổng quát, nhưng mAP50-95 thấp hơn khi yêu cầu biên mask khít hơn. Hai ô sem loss trong Hình 5.11 hiển thị đường bằng 0 vì dataset segmentation chỉ có một mask cho một con tôm trong từng ảnh crop, không đặt ra bài toán semantic segmentation nhiều lớp pixel riêng biệt. Do đó, chất lượng mask trong lần huấn luyện này được nhận xét chủ yếu qua seg loss và các chỉ số Mask, còn sem loss không được dùng làm căn cứ phân tích chính.



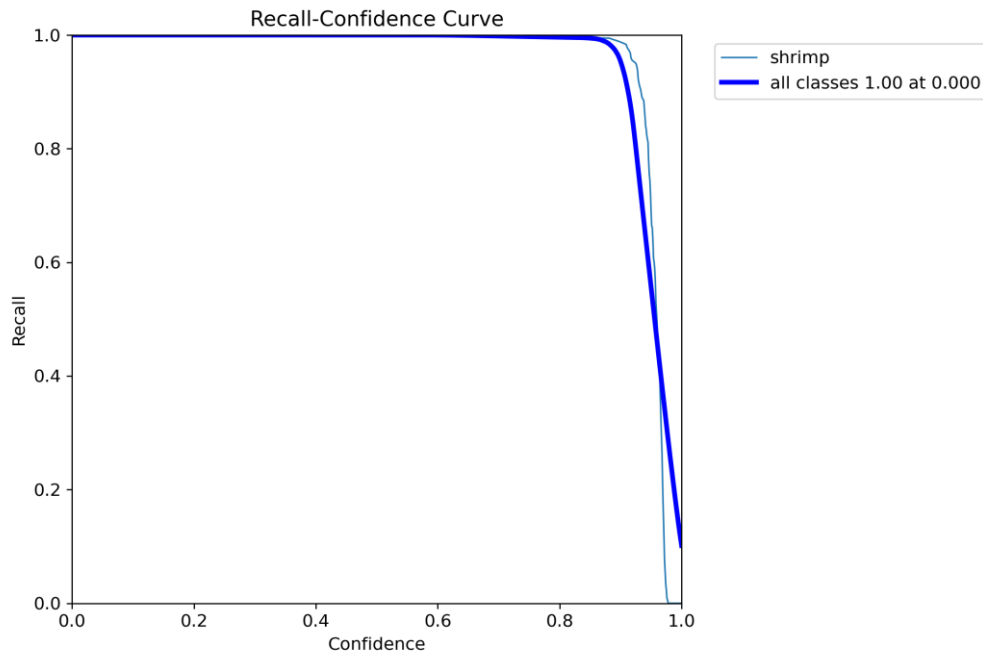
Hình 5.12. Đường cong Box Precision-Recall của YOLOv8n segmentation

Đường cong Box Precision-Recall ở Hình 5.12 nằm sát vùng Precision cao và Recall cao. Hình dáng này xuất hiện vì ảnh segmentation đã được crop theo bounding box, mỗi ảnh chủ yếu chứa một con tôm trên nền xám và chỉ có một lớp shrimp. Do đó, nhiệm vụ định vị bounding box trong tập segmentation đơn giản hơn so với ảnh nhiều đối tượng. Khi Recall tăng, Precision vẫn không giảm mạnh vì mô hình ít gặp trường hợp phải phân biệt nhiều cá thể hoặc nhiều lớp trong cùng một ảnh crop. Bounding box ổn định là điều kiện cần để mô hình tạo segmentation mask cho đúng cá thể đã được định vị.



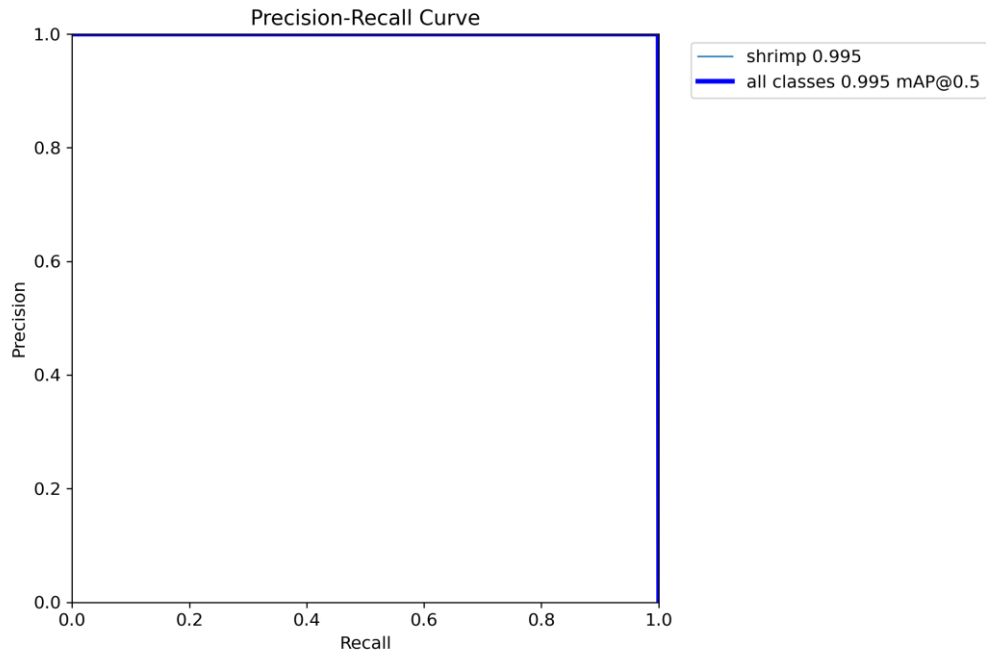
Hình 5.13. Đường cong Box Precision theo confidence của YOLOv8n segmentation

Đường cong Box Precision theo confidence ở Hình 5.13 có xu hướng tăng rồi giữ ở vùng cao khi confidence đủ lớn. Với ảnh crop chỉ tập trung vào một con tôm, các dự đoán có confidence cao thường khớp với vùng đối tượng chính trong ảnh. Khi tăng ngưỡng confidence, các dự đoán yếu ở vùng biên, nền xám hoặc chi tiết phụ bị loại bỏ trước, làm Precision tăng. Tuy nhiên, nếu confidence quá cao, một số ảnh crop có tôm cong, biên mờ hoặc vùng thân không rõ có thể bị loại dù vẫn còn hữu ích cho bước tạo mask.



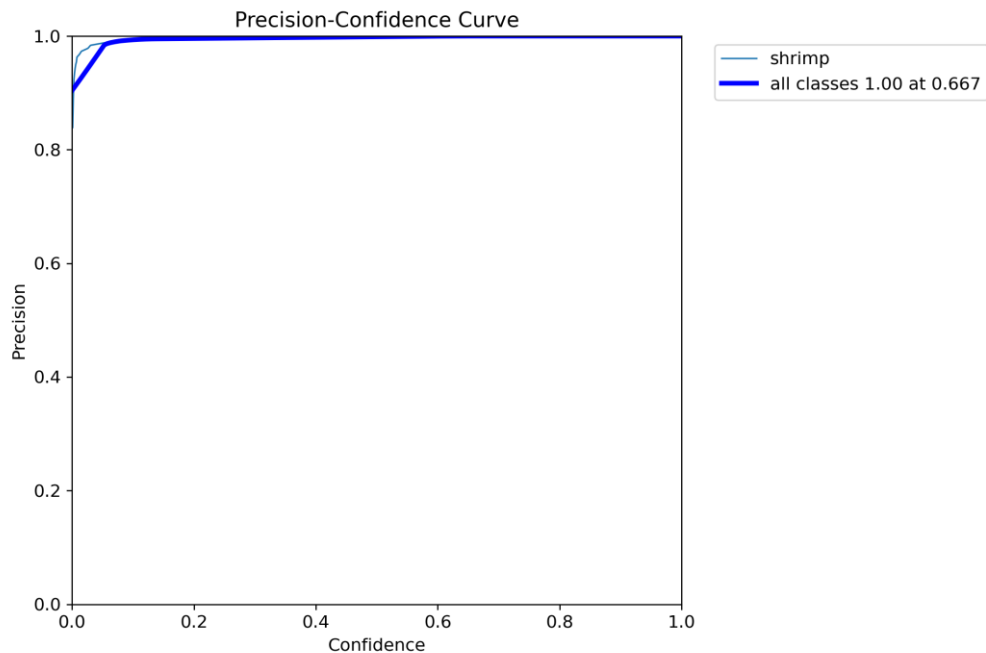
Hình 5.14. Đường cong Box Recall theo confidence của YOLOv8n segmentation

Đường cong Box Recall theo confidence ở Hình 5.14 giảm khi confidence tăng. Hình dáng này đối lập với Precision vì ngưỡng confidence càng cao thì càng nhiều dự đoán bị loại bỏ. Trong dataset segmentation, mỗi ảnh crop đã đại diện cho một con tôm, nên nếu dự đoán của ảnh đó bị loại ở bước bounding box thì toàn bộ mask tương ứng cũng không còn đầu vào để xử lý. Vì vậy, phần giảm Recall cho thấy không nên chọn ngưỡng quá chặt nếu mục tiêu là giữ đủ vùng tôm để đo chiều dài.



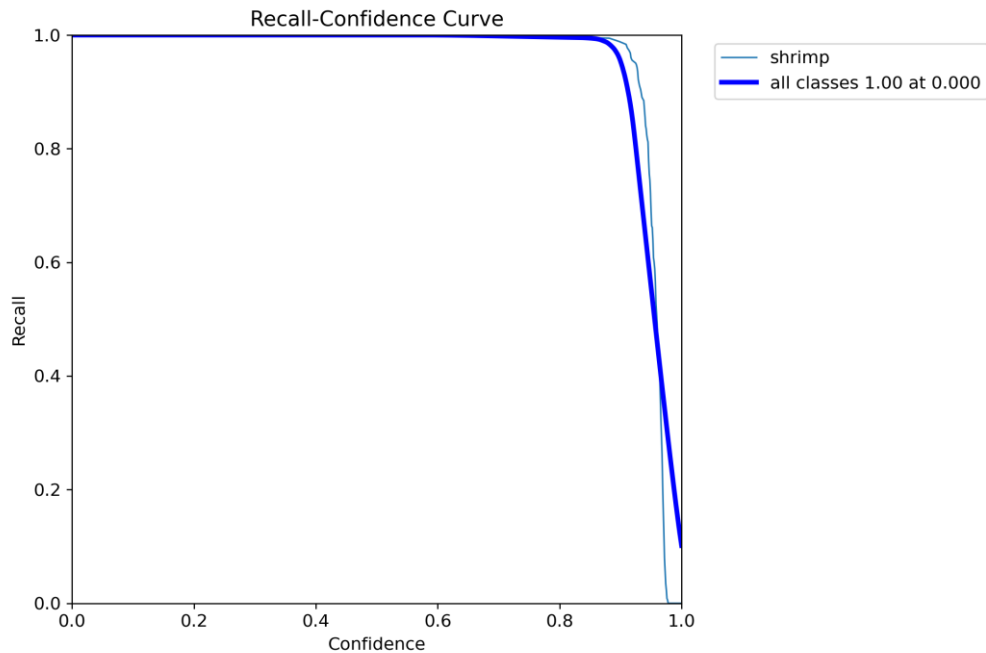
Hình 5.15. Đường cong Mask Precision-Recall của YOLOv8n segmentation

Đường cong Mask Precision-Recall ở Hình 5.15 nằm gần vùng trên của đồ thị, cho thấy mô hình tạo được mask ổn định ở mức instance. Kết quả này phù hợp với cách tạo dataset segmentation: mỗi ảnh crop chỉ cần học một mask của một con tôm, nền xám chiếm phần còn lại và không có nhiều lớp pixel khác nhau. Tuy nhiên, đường cong cao không có nghĩa là biên mask đã hoàn toàn khít với thân tôm. Với bài toán đo chiều dài, sai lệch nhỏ ở vùng đầu, đuôi, chân hoặc râu vẫn có thể làm skeleton bị lệch, nên cần đọc đường cong này cùng mAP50-95 của Mask trong Hình 5.11.



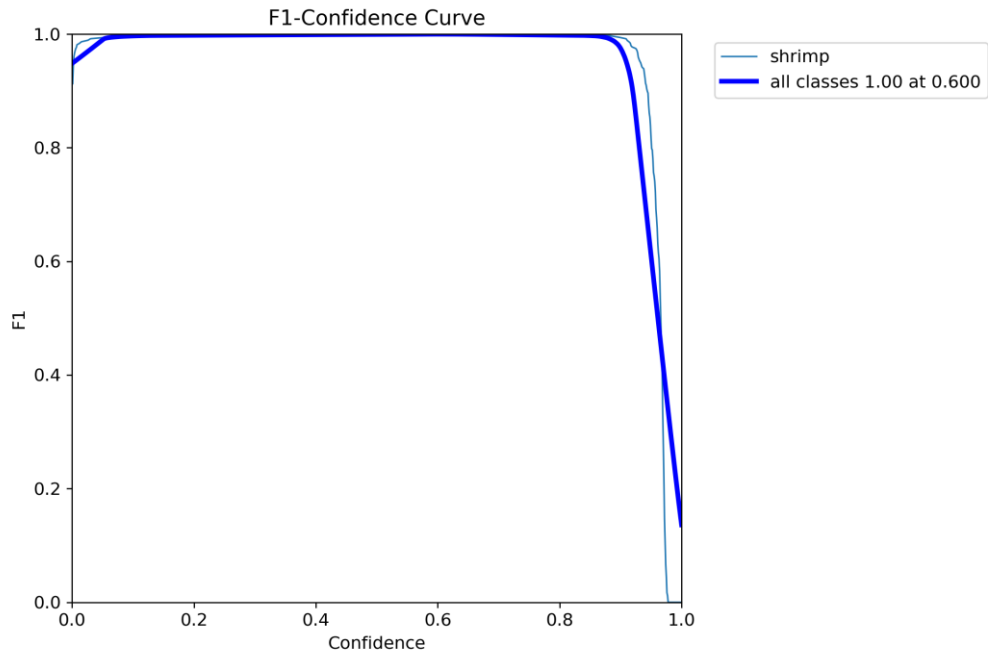
Hình 5.16. Đường cong Mask Precision theo confidence của YOLOv8n segmentation

Đường cong Mask Precision theo confidence ở Hình 5.16 tăng và giữ ở vùng cao vì các mask có confidence thấp thường liên quan đến vùng biên không chắc chắn, phần chân, râu hoặc chi tiết nền còn sót trong ảnh crop. Khi ngưỡng confidence tăng, các mask yếu bị loại trước nên phần mask được giữ lại có độ tin cậy cao hơn. Tuy vậy, ảnh crop chỉ có một mask chính cho con tôm không có nghĩa là mask luôn sạch hoàn toàn. Mask vẫn có thể lấy dư chi tiết mảnh ở biên, nên confidence cần được kết hợp với bước lọc connected component trước khi trích xuất skeleton.



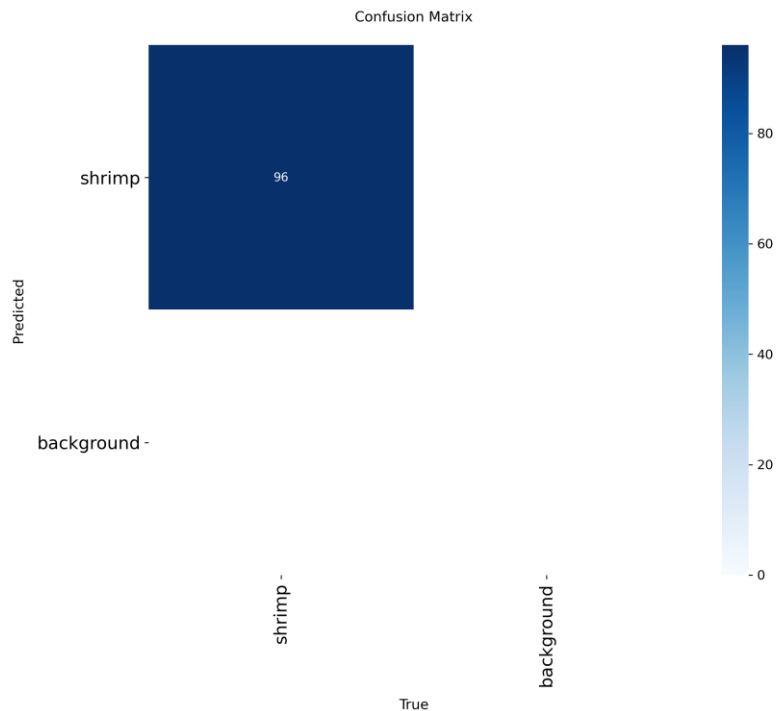
Hình 5.17. Đường cong Mask Recall theo confidence của YOLOv8n segmentation

Đường cong Mask Recall theo confidence ở Hình 5.17 giảm khi confidence tăng vì các mask ít chắc chắn bị loại bỏ dần. Với dataset crop một con tôm, mask bị loại đồng nghĩa với việc ảnh crop đó không còn vùng thân để đo. Một số mask có confidence thấp vẫn có thể là mask đúng nhưng thuộc các trường hợp khó như tôm cong, thân trong mờ, vùng đầu hoặc đuôi không rõ. Vì vậy, ngưỡng confidence quá cao có thể làm mất vùng thân cần đo và làm giảm khả năng xử lý đủ mẫu.



Hình 5.18. Đường cong Mask F1 của YOLOv8n segmentation

Đường cong Mask F1 ở Hình 5.18 có vùng cao trên một khoảng confidence rộng. Đây là vùng cân bằng giữa hai xu hướng ngược nhau: Precision tăng khi loại bỏ mask yếu, còn Recall giảm khi ngưỡng quá chặt làm mất một số mask đúng. Với dataset mỗi ảnh crop chỉ có một mask cho một con tôm, vùng F1 cao cho thấy mô hình có thể chọn được khoảng confidence đủ ổn định để vừa giữ lại vùng thân cần đo, vừa hạn chế các mask chứa dư nền hoặc chi tiết phụ. Điều này phù hợp với pipeline đo chiều dài vì bước skeleton cần mask đủ đầy nhưng không quá nhiều.



Hình 5.19. Confusion matrix của YOLOv8n segmentation

Confusion matrix ở Hình 5.19 tập trung chủ yếu tại ô shrimp được dự đoán đúng, cho thấy mô hình nhận đúng lớp đối tượng trong các ảnh crop. Điều này phù hợp với dataset segmentation chỉ có một lớp shrimp và mỗi ảnh crop tập trung vào một con tôm. Tuy nhiên, confusion matrix chỉ phản ánh đúng sai ở mức lớp đối tượng, không cho biết mask có bám sát vùng thân theo pixel hay không. Vì vậy, confusion matrix chỉ dùng để xác nhận mô hình không nhầm lớp nghiêm trọng, còn chất lượng mask phục vụ skeleton cần được đánh giá qua seg loss, các đường cong Mask và mAP50-95 của Mask.

5.3 ĐÁNH GIÁ MÔ HÌNH

5.3.1 Đánh giá mô hình phát hiện trên test set

Sau huấn luyện, trọng số tốt nhất best.pt của YOLOv8n detection được đánh giá trên test set gồm 47 con tôm khác nhau theo cấu hình validation mặc định của Ultralytics. Kết quả đạt được như sau:

Bảng 5.2. Kết quả đánh giá YOLOv8n detection trên test set

Chỉ số	Giá trị
Precision	0.9882
Recall	1.0000
F1-score	0.9940
mAP50	0.9941
mAP75	0.9931
mAP50-95	0.9293

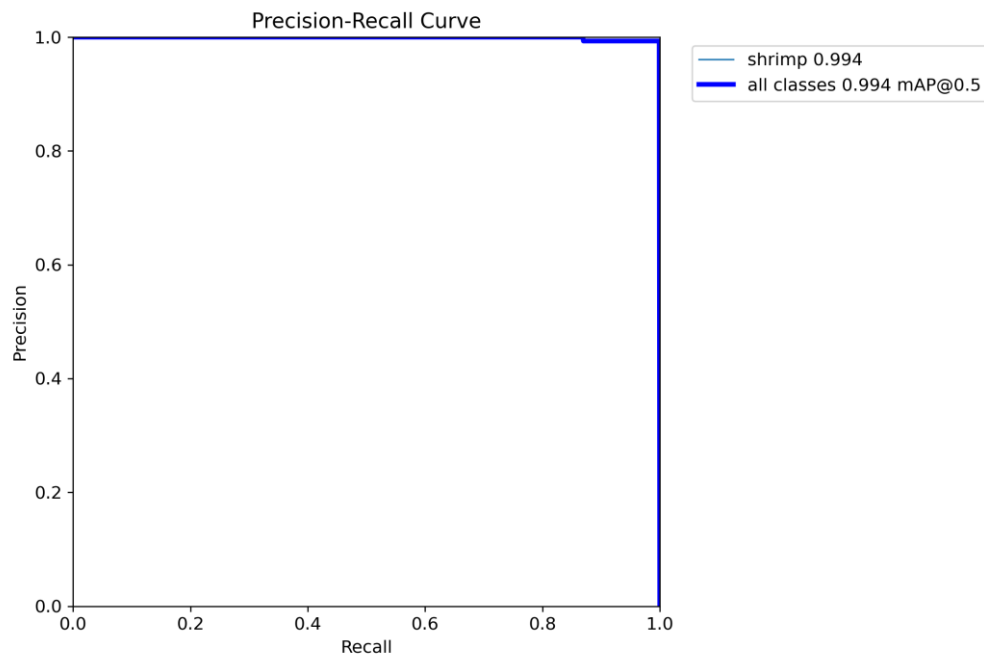
Precision đạt 0.9882, cho thấy phần lớn dự đoán trên test set là đúng. Chỉ số này phản ánh khả năng hạn chế dự đoán nhầm nền, thước đo hoặc vùng không phải tôm thành tôm. Precision chưa đạt 1.0000 nên không thể kết luận mô hình hoàn toàn không có dự đoán sai, nhưng giá trị gần 0.99 cho thấy số dự đoán sai trong test set là thấp.

Recall đạt 1.0000 trong điều kiện đánh giá trên test set gồm 47 con tôm khác nhau và theo cấu hình validation mặc định của Ultralytics. Kết quả này cho thấy mô hình phát hiện được các đối tượng tôm đã được gán nhãn trong test set đã xây dựng. Đây là chỉ số quan trọng đối với pipeline xử lý, vì nếu detection bỏ sót tôm thì bước segmentation, trích xuất skeleton và đo chiều dài sẽ không có vùng đầu vào để tiếp tục xử lý.

F1-score đạt 0.9940, phản ánh sự cân bằng giữa Precision và Recall. Do Precision gần 0.99 và Recall đạt 1.0000, F1-score đạt giá trị cao trên test set. Chỉ số này phù hợp để đánh giá tổng hợp mô hình detection trong bối cảnh vừa cần phát hiện đủ tôm, vừa cần hạn chế vùng quan tâm sai.

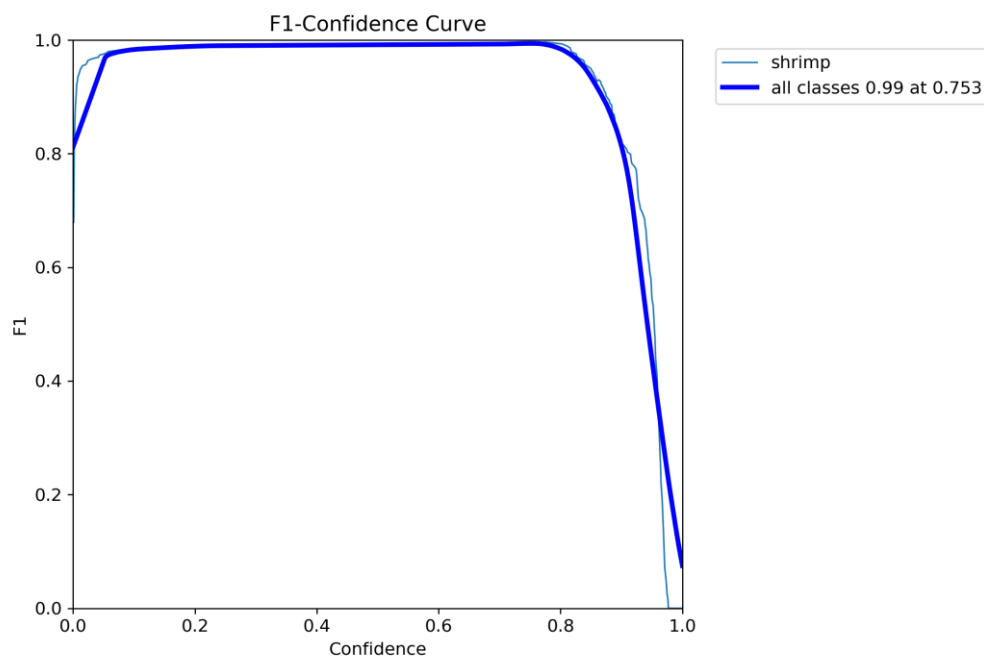
mAP50 đạt 0.9941 và mAP75 đạt 0.9931, cho thấy bounding box dự đoán vẫn khớp tốt với nhãn khi tăng ngưỡng IoU từ 0.50 lên 0.75. Tuy nhiên, mAP50-95 đạt 0.9293, thấp hơn hai chỉ số trên vì chỉ số này lấy trung bình trên nhiều ngưỡng IoU chặt

hơn. Điều này cho thấy mô hình detection định vị tốt ở mức tổng quát, nhưng vẫn có sai lệch nhỏ ở biên bounding box khi yêu cầu khớp rất chặt.



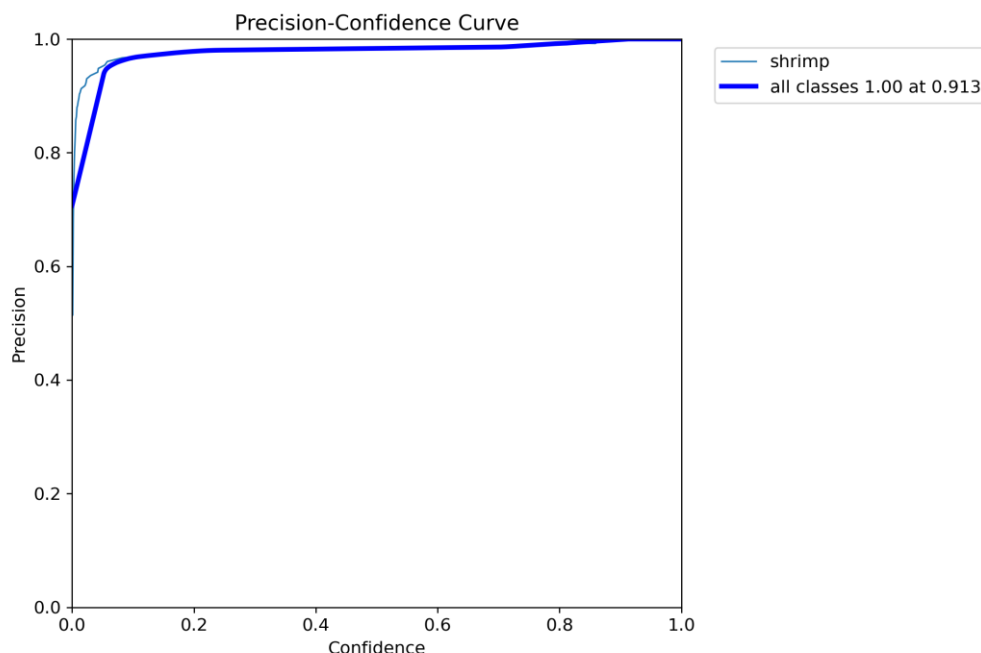
Hình 5.20. Đường cong Precision-Recall của YOLOv8n detection trên test set

Đường cong Precision-Recall trên test set ở Hình 5.20 nằm gần sát cạnh trên và cạnh phải của đồ thị, với mAP50 hiển thị 0.994. Hình dáng này cho thấy mô hình giữ Precision rất cao trên phần lớn dải Recall, nghĩa là vẫn phát hiện đủ tôm mà không tạo nhiều dự đoán sai. Phần gãy nhẹ ở vùng Recall rất cao xuất hiện khi mô hình cố giữ lại gần như toàn bộ đối tượng, bao gồm cả các mẫu khó như tôm cong, nhiều con nằm gần nhau hoặc vùng có thước đo. Điều này phù hợp với Precision 0.9882 trong Bảng 5.2.



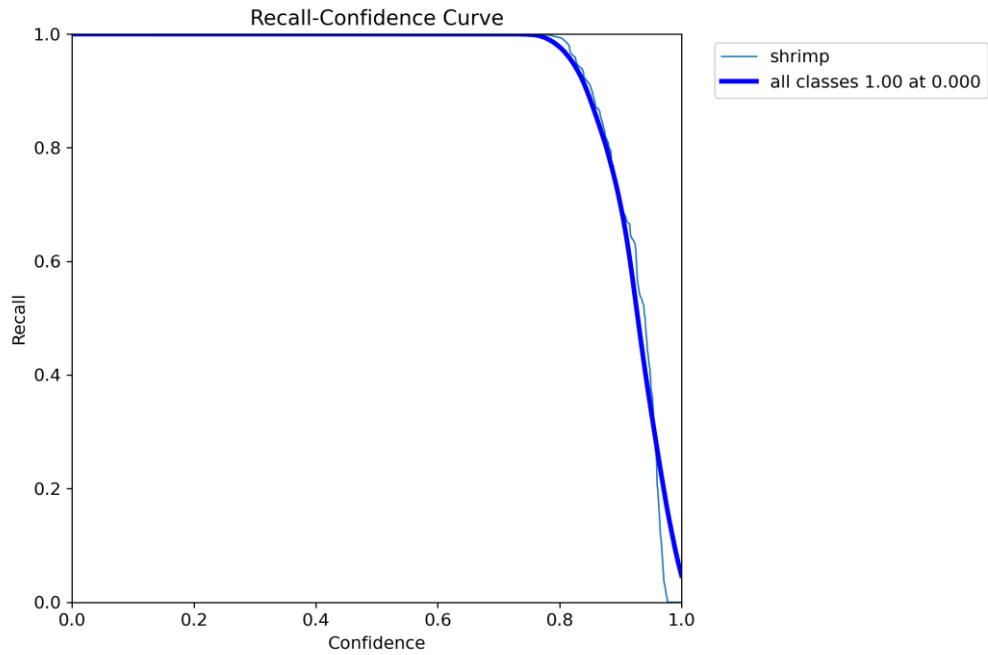
Hình 5.21. Đường cong F1-score của YOLOv8n detection trên test set

Đường cong F1-score ở Hình 5.21 tăng nhanh ở vùng confidence thấp, giữ gần mức 1 trong một khoảng confidence rộng và đạt khoảng 0.99 tại confidence 0.753 theo chú thích của biểu đồ. Vùng cao rộng này cho thấy mô hình có khoảng ngưỡng cân bằng tốt giữa phát hiện đủ tôm và hạn chế dự đoán sai. Khi confidence tiến gần 1, F1-score giảm mạnh vì nhiều dự đoán bị loại bỏ, làm Recall giảm. Biểu hiện này cho thấy ngưỡng confidence không nên đặt quá cao nếu hệ thống ưu tiên đo đầy đủ các cá thể tôm.



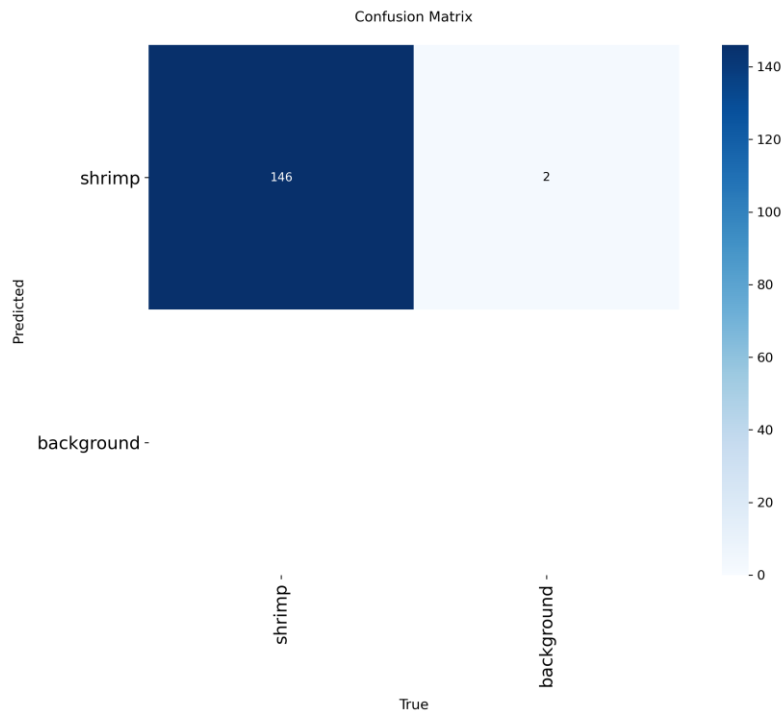
Hình 5.22. Đường cong Precision theo confidence của YOLOv8n detection trên test set

Đường cong Precision theo confidence ở Hình 5.22 tăng và ổn định khi confidence tăng vì các dự đoán yếu bị loại bỏ dần. Phần đường đi lên cho thấy mô hình gán confidence thấp hơn cho nhiều vùng chưa chắc chắn, còn các dự đoán được giữ lại ở ngưỡng cao thường đáng tin cậy hơn. Tuy nhiên, biểu đồ này không được đọc riêng lẻ, vì nếu tăng confidence quá mức thì Recall có thể giảm, làm mất đối tượng cần đo.



Hình 5.23. Đường cong Recall theo confidence của YOLOv8n detection trên test set

Đường cong Recall theo confidence ở Hình 5.23 thể hiện xu hướng ngược với Precision. Khi confidence tăng, Recall giảm vì mô hình loại bỏ thêm các dự đoán có confidence thấp, trong đó có thể có tôm thật ở tư thế khó hoặc vùng biên không rõ. Với bài toán đo chiều dài tôm, hình dáng này cho thấy cần chọn ngưỡng confidence cân bằng, tránh tình trạng loại bỏ tôm khó phát hiện trước khi chuyển sang bước segmentation.



Hình 5.24. Confusion matrix của YOLOv8n detection trên test set

Confusion matrix ở Hình 5.24 hiển thị phần lớn kết quả nằm ở ô shrimp được dự đoán là shrimp. Hình dáng tập trung này cho thấy mô hình nhận diện lớp chính tốt trên test set. Ô shrimp-background có giá trị nhỏ, thể hiện một lượng nhỏ vùng nền bị dự đoán thành shrimp. Không quan sát thấy giá trị ở ô background-shrimp trong biểu đồ này, phù hợp với Recall 1.0000 trong điều kiện đánh giá đã nêu. Vì vẫn còn ô nhầm nền, mô hình detection cần được dùng kèm bước segmentation và các bước hậu xử lý để giảm tác động của vùng quan tâm sai.

5.3.2 Đánh giá mô hình phân đoạn trên test set

Trọng số tốt nhất best.pt của YOLOv8n segmentation được đánh giá trên test set gồm 47 con tôm khác nhau theo cấu hình validation mặc định của Ultralytics. Kết quả được chia thành hai nhóm: chỉ số cho bounding box và chỉ số cho segmentation mask.

Bảng 5.3. Kết quả bounding box của YOLOv8n segmentation trên test set

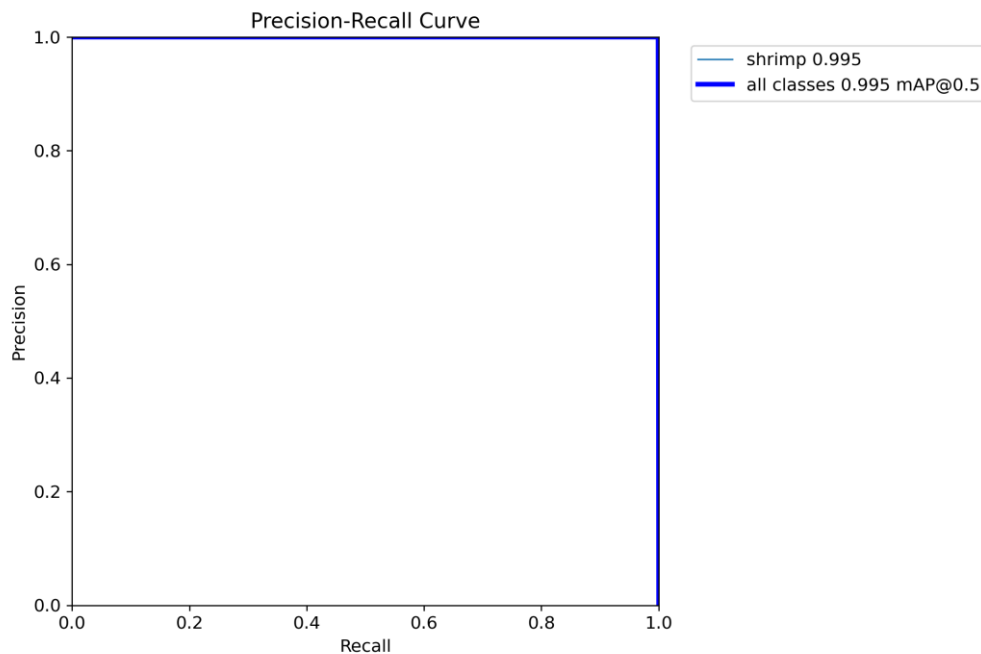
Chỉ số bounding box	Giá trị
Precision	0.9996
Recall	1.0000
F1-score	0.9998
mAP50	0.9950
mAP75	0.9950
mAP50-95	0.9758

Bảng 5.4. Kết quả segmentation mask của YOLOv8n segmentation trên test set

Chỉ số segmentation mask	Giá trị
Precision	0.9996
Recall	1.0000
F1-score	0.9998
mAP50	0.9950
mAP75	0.9950
mAP50-95	0.8960

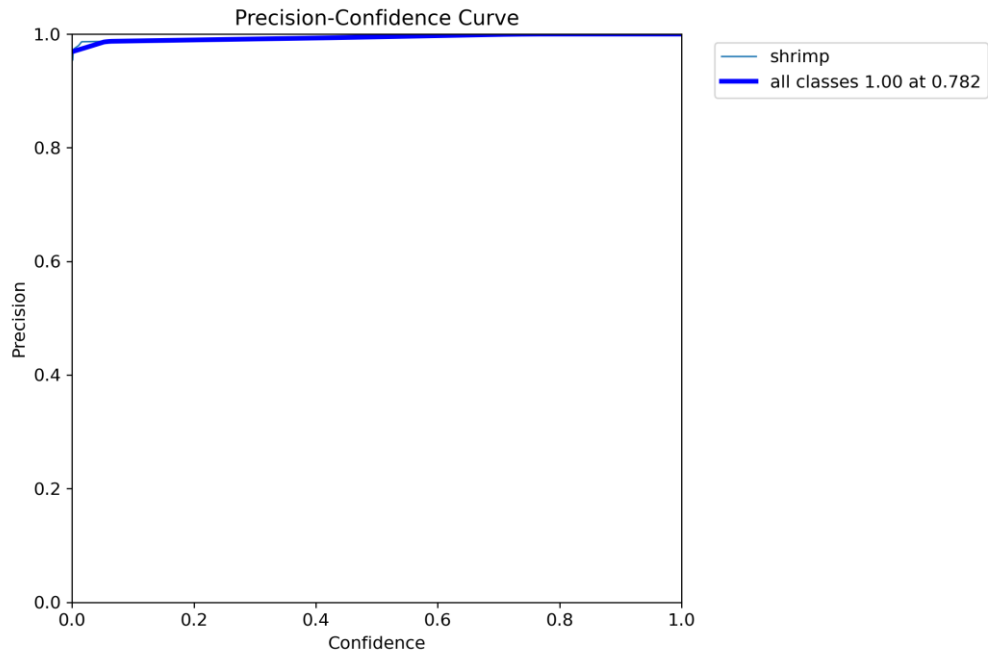
Ở nhóm bounding box, Precision đạt 0.9996 và Recall đạt 1.0000 trong điều kiện đánh giá trên test set gồm 47 con tôm khác nhau và theo cấu hình validation mặc định của Ultralytics. Kết quả này cho thấy mô hình định vị được các tôm có nhãn trong test set đã xây dựng và số dự đoán sai ở mức bounding box là thấp. mAP50 và mAP75 đều đạt 0.9950, còn mAP50-95 đạt 0.9758, cho thấy bounding box của mô hình segmentation khớp tốt với nhãn ngay cả khi xét nhiều ngưỡng IoU.

Trường hợp mAP50 và mAP75 cùng đạt 0.9950 là điểm cần chú ý. Dataset huấn luyện segmentation được tạo bằng cách crop tập trung từng cá thể tôm trên nền xám, nên mô hình có điều kiện học bounding box khá khít quanh vùng tôm. Vì vậy, khi ngưỡng IoU tăng từ 0.50 lên 0.75, mAP chưa giảm rõ rệt. Khi xét đến mAP50-95, yêu cầu khớp biên chặt hơn làm điểm giảm nhẹ, phản ánh độ nhạy của bounding box ở vùng đầu, đuôi và thân cong.



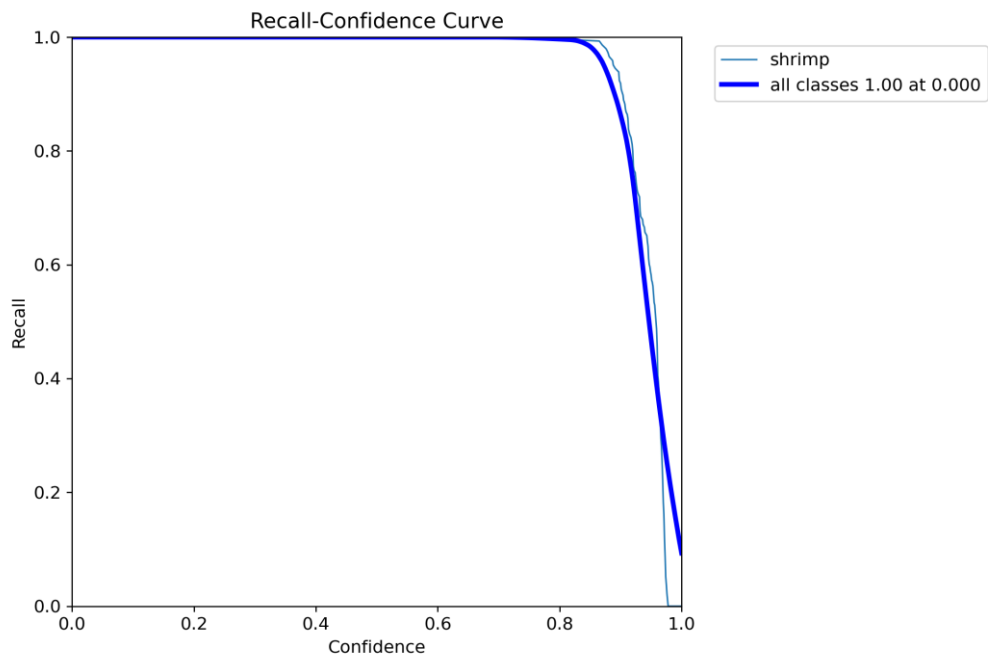
Hình 5.25. Đường cong Box Precision-Recall của YOLOv8n segmentation trên test set

Đường cong Box Precision-Recall ở Hình 5.25 gần như áp sát góc trên phải của đồ thị, với mAP50 hiển thị 0.995. Hình dáng này cho thấy bounding box của mô hình segmentation vừa giữ được Precision cao, vừa giữ được Recall cao trên test set. Kết quả này đạt được vì ảnh segmentation đã tập trung vào từng cá thể tôm, nền xám ít phức tạp và lớp đối tượng duy nhất là shrimp. Đây là nền tảng tốt để sinh segmentation mask vì mỗi segmentation mask được gắn với một instance đã được định vị.



Hình 5.26. Đường cong Box Precision theo confidence của YOLOv8n segmentation trên test set

Đường cong Box Precision theo confidence ở Hình 5.26 nằm ở vùng rất cao khi confidence đủ lớn. Dạng đường này cho thấy các bounding box được giữ lại ở ngưỡng confidence cao ít bị nhầm với nền, vì các dự đoán yếu đã bị loại bỏ trước đó. Tuy nhiên, tương tự mô hình detection, ngưỡng confidence quá cao có thể làm giảm Recall.



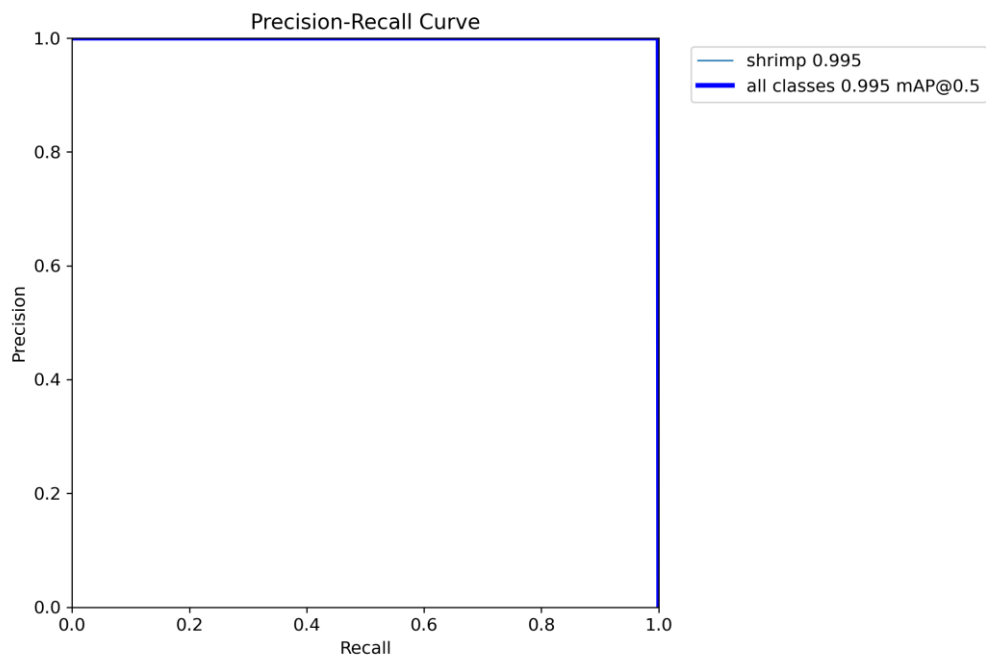
Hình 5.27. Đường cong Box Recall theo confidence của YOLOv8n segmentation trên test set

Đường cong Box Recall theo confidence ở Hình 5.27 giữ cao trên phần lớn vùng confidence và giảm khi confidence tiến gần vùng rất cao. Hình dáng này cho thấy mô

hình định vị ổn định trong vùng confidence vận hành rộng, nhưng vẫn có thể mất một số dự đoán khi ngưỡng bị đẩy quá chặt. Khi triển khai, biểu hiện giảm ở vùng confidence quá cao là lý do cần tránh chọn ngưỡng quá chặt.

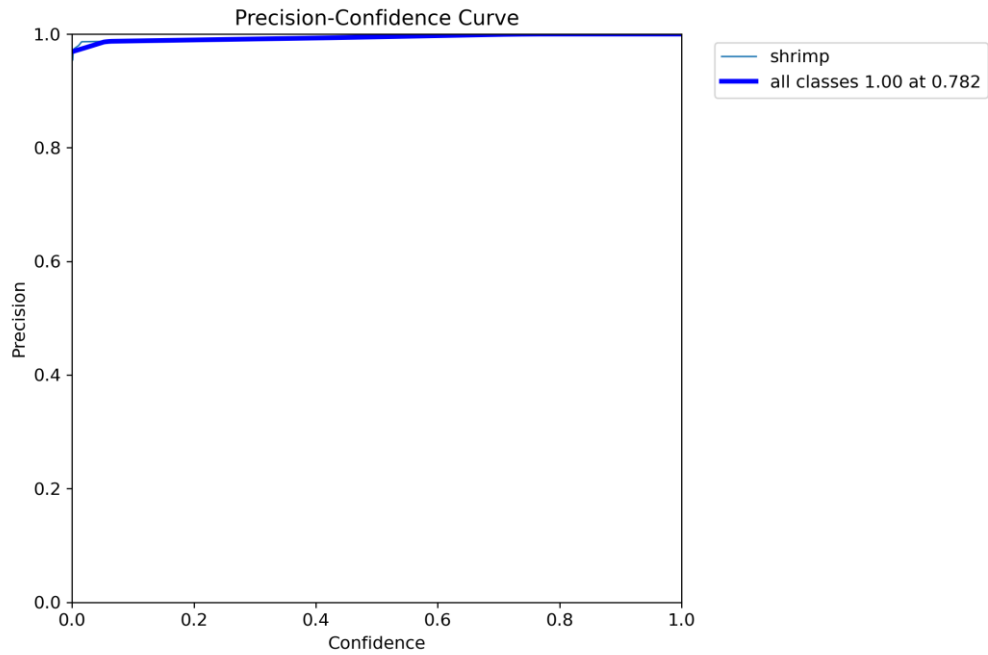
Ở nhóm segmentation mask, Precision đạt 0.9996, Recall đạt 1.0000 và F1-score đạt 0.9998. Ba chỉ số này giống với nhóm bounding box vì trong đánh giá của Ultralytics, bounding box và segmentation mask được tính trên cùng một tập dự đoán instance. Nói cách khác, Precision, Recall và F1-score cùng phản ánh việc mô hình phát hiện đúng cá thể tôm trong test set, còn khác biệt chính nằm ở hình dạng vùng được so khớp khi tính mAP: bounding box dùng vùng hộp chữ nhật, trong khi segmentation mask dùng vùng pixel chi tiết.

mAP50 và mAP75 của segmentation mask đều đạt 0.9950, cho thấy ở hai ngưỡng IoU này vùng segmentation mask vẫn khớp tốt với nhãn ở mức tổng quát. Tuy nhiên, mAP50-95 của segmentation mask đạt 0.8960, thấp hơn mAP50-95 của bounding box. Sự khác biệt này xuất hiện vì segmentation mask phải khớp chi tiết theo từng pixel, trong khi bounding box chỉ cần bao quanh vùng tổng quát của tôm. Ở các ngưỡng IoU cao hơn, sai lệch nhỏ ở biên thân, vùng chân, râu hoặc tư thế cong của tôm đều có thể làm điểm segmentation mask giảm rõ hơn.



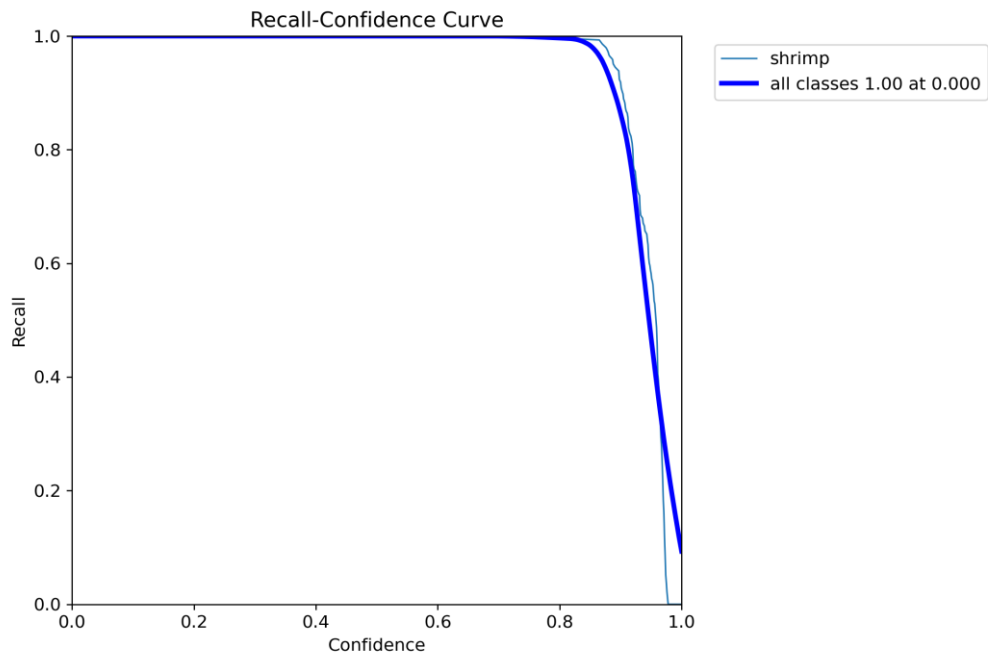
Hình 5.28. Đường cong Mask Precision-Recall của YOLOv8n segmentation trên test set

Đường cong Mask Precision-Recall ở Hình 5.28 cũng áp sát vùng trên phải của đồ thị, với mAP50 hiển thị 0.995. Hình dáng này cho thấy mô hình tạo segmentation mask đủ tốt ở ngưỡng IoU 0.50, tức là vùng mask nhìn chung phủ đúng thân tôm. Tuy nhiên, kết quả này cần được đọc cùng mAP50-95 bằng 0.8960 trong Bảng 5.4, vì đường PR ở mAP50 chưa phản ánh đầy đủ độ khớp biên ở các ngưỡng IoU chặt hơn.



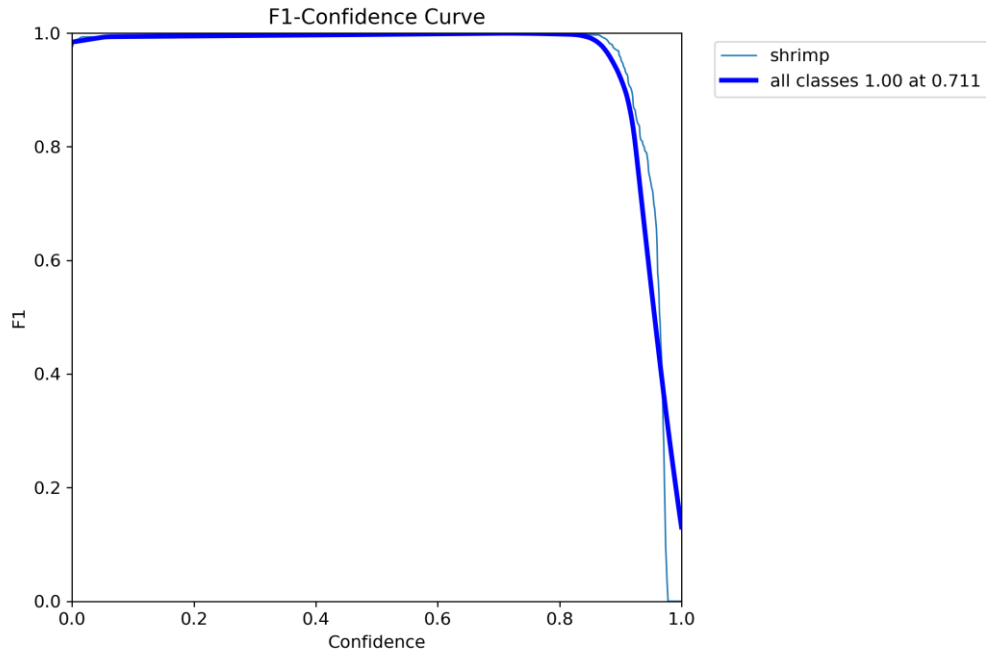
Hình 5.29. Đường cong Mask Precision theo confidence của YOLOv8n segmentation trên test set

Đường cong Mask Precision theo confidence ở Hình 5.29 tăng khi confidence cao hơn vì các mask yếu, dễ lấy dư nền hoặc chi tiết không thuộc thân tôm, bị loại bỏ dần. Nếu Mask Precision thấp, segmentation mask có thể chứa thêm vùng chân, râu hoặc nền, làm tăng nguy cơ sinh nhánh phụ khi trích xuất skeleton. Do đó, Precision cao là điều kiện cần để giảm lỗi nhánh phụ, nhưng vẫn cần hậu xử lý để loại bỏ các vùng segmentation mask rời.



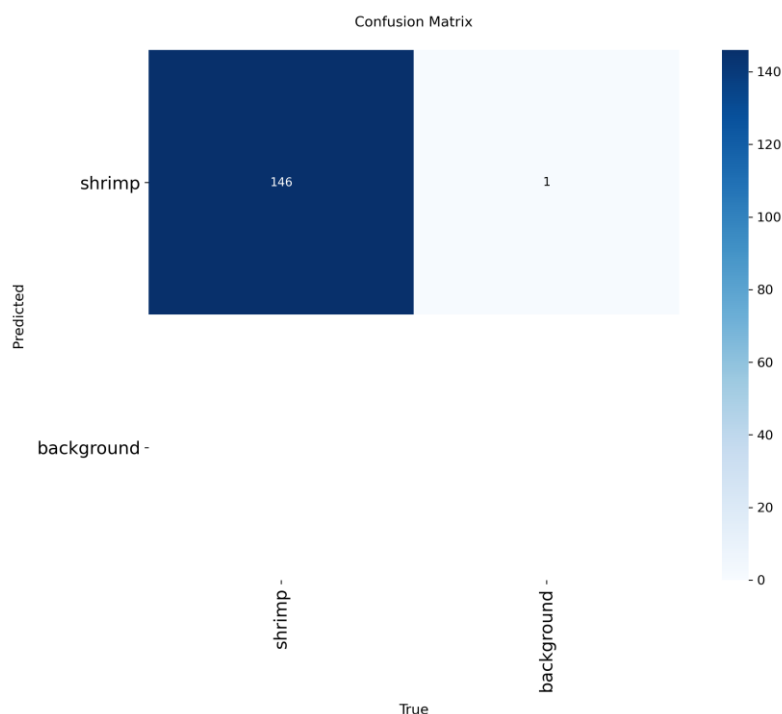
Hình 5.30. Đường cong Mask Recall theo confidence của YOLOv8n segmentation trên test set

Đường cong Mask Recall theo confidence ở Hình 5.30 giảm khi confidence quá cao vì một số mask đúng nhưng chưa đủ chắc chắn bị loại bỏ. Nếu Mask Recall giảm, segmentation mask có thể thiếu vùng đầu, đuôi hoặc một phần thân, làm đường skeleton ngắn hơn so với hình dạng thực tế của tôm. Đây là lý do bước chọn ngưỡng confidence cần ưu tiên giữ đủ vùng thân trước khi đo chiều dài.



Hình 5.31. Đường cong Mask F1 của YOLOv8n segmentation trên test set

Đường cong Mask F1 ở Hình 5.31 giữ gần mức 1 trên một khoảng confidence rộng và đạt 1.00 tại confidence 0.711 theo chú thích của biểu đồ. Hình dáng này thể hiện vùng cân bằng tốt giữa việc loại bỏ mask yếu và giữ lại đủ vùng thân tôm. Khi confidence tiến gần 1, đường cong giảm mạnh vì Recall bị giảm. Biểu hiện này cho thấy mô hình có vùng confidence vận hành thuận lợi, nhưng nếu chọn ngưỡng quá cao thì segmentation mask có thể bị loại bỏ dù vẫn có ích cho bước đo.



Hình 5.32. Confusion matrix của YOLOv8n segmentation trên test set

Confusion matrix ở Hình 5.32 hiển thị phần lớn kết quả ở ô shrimp được dự đoán là shrimp và chỉ có một giá trị nhỏ ở ô shrimp-background. Hình dáng tập trung này phù hợp với Precision 0.9996 và Recall 1.0000 trong Bảng 5.3 và Bảng 5.4. Tuy nhiên, confusion matrix chỉ phản ánh đúng sai ở mức instance, không đánh giá được biên segmentation mask theo pixel. Vì vậy, mAP50-95 của segmentation mask vẫn là chỉ số cần chú ý khi dùng kết quả segmentation cho skeleton và đo chiều dài.

5.3.3 So sánh và lựa chọn mô hình sử dụng

YOLOv8n detection và YOLOv8n segmentation có vai trò khác nhau trong hệ thống. YOLOv8n detection có nhiệm vụ xác định nhanh vị trí tôm bằng bounding box. Kết quả detection phù hợp để xử lý ảnh hoặc video có nhiều đối tượng, vì mô hình nhẹ và đầu ra đơn giản. Khi đã có bounding box, hệ thống có thể giới hạn vùng quan tâm và giảm lượng thông tin nền đưa vào bước sau.

YOLOv8n segmentation có nhiệm vụ tạo segmentation mask cho từng cá thể tôm. Segmentation mask giữ thông tin hình dạng chi tiết hơn bounding box, nên phù hợp để tạo skeleton và đo chiều dài theo đường thân. Đây là thông tin mà mô hình detection không cung cấp được.

Nếu chỉ dùng YOLOv8n segmentation cho toàn bộ ảnh hoặc video, hệ thống có thể lấy được segmentation mask trực tiếp nhưng chi phí xử lý cao hơn, đặc biệt khi video có nhiều frame. Nếu chỉ dùng YOLOv8n detection, hệ thống phát hiện nhanh nhưng không có vùng thân chi tiết để đo chiều dài theo hình dạng cong. Vì vậy, phương án sử dụng kết hợp hai mô hình là phù hợp hơn: YOLOv8n detection định vị nhanh và giới hạn vùng quan tâm, YOLOv8n segmentation phân đoạn vùng đã được giới hạn để lấy segmentation mask phục vụ skeleton và đo chiều dài.

Với các kết quả đánh giá đã trình bày, YOLOv8n detection được chọn cho bước phát hiện và định vị tumor, còn YOLOv8n segmentation được chọn cho bước phân đoạn thân tumor. Cách lựa chọn này tạo sự cân bằng giữa tốc độ xử lý và chất lượng dữ liệu hình dạng, phù hợp với pipeline xử lý gồm phát hiện, giới hạn vùng quan tâm, tạo segmentation mask, trích xuất skeleton và đo chiều dài.

CHƯƠNG 6. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

6.1 TỔNG QUAN HỆ THỐNG ĐO VÀ PHÂN LOẠI KÍCH CỠ TÔM

6.1.1 Mục tiêu của hệ thống

Hệ thống được thiết kế để hỗ trợ đo chiều dài tôm tự động từ ảnh hoặc video. Từ dữ liệu đầu vào, hệ thống xác định vị trí tôm, tách vùng thân, đo chiều dài theo đường trục, quy đổi kết quả sang milimet và phân loại tôm theo bảng kích cỡ do người dùng cấu hình.

Kết quả đầu ra không chỉ là nhãn kích cỡ cuối cùng, mà còn gồm chiều dài theo pixel, chiều dài thực theo milimet, Track ID nếu dữ liệu là video, ảnh minh họa đường đo, log xử lý và dữ liệu kết quả. Các thông tin này giúp người dùng vừa xem kết quả tổng hợp, vừa kiểm tra lại từng bước xử lý khi cần.

6.1.2 Dữ liệu đầu vào và đầu ra

Dữ liệu đầu vào gồm ảnh tĩnh hoặc video chứa tôm cần xử lý. Ảnh tĩnh phù hợp với trường hợp kiểm tra từng mẫu hoặc nhiều con tôm trong cùng một ảnh. Video phù hợp với trường hợp tôm di chuyển qua vùng quan sát, trong đó cùng một cá thể có thể xuất hiện ở nhiều frame liên tiếp.

Ngoài dữ liệu hình ảnh, hệ thống sử dụng các cấu hình như ngưỡng detection, ngưỡng segmentation, khoảng mở rộng bounding box, tham số chọn frame, scale factor và bảng phân loại kích cỡ. Các tham số vận hành cho phép người dùng điều chỉnh cách đọc video, hướng di chuyển, mức lưu ảnh minh họa và cách dọn dữ liệu sau khi xử lý.

Đầu ra của hệ thống được tổ chức theo từng nguồn dữ liệu. Mỗi nguồn có một file JSON kết quả, trong đó lưu source file, source stem, thời điểm xử lý, scale factor và danh sách tôm đã đo. Mỗi cá thể trong danh sách có Track ID, frame được chọn, chiều dài theo pixel, chiều dài thực theo milimet, nhãn kích cỡ và đường dẫn ảnh minh họa nếu chế độ lưu ảnh được bật.

6.1.3 Bảng tham số cấu hình

Các tham số cấu hình được lưu trong settings.json và được đọc lại mỗi lần hệ thống chạy pipeline. Bảng 6.1 trình bày các tham số chính ảnh hưởng trực tiếp đến quá trình đo tôm.

Bảng 6.1. Các tham số cấu hình chính của hệ thống

Nhóm	Tham số	Ý nghĩa
Đường dẫn	INPUT_DIR	Thư mục chứa ảnh hoặc video đầu vào cần xử lý.
Đường dẫn	OUTPUT_DIR	Thư mục lưu run kết quả, JSON, log và ảnh debug.
Nhận diện	CONF_DET	Ngưỡng tin cậy của mô hình phát hiện tôm. Giá trị cao giúp giảm nhận nhầm, nhưng có thể bỏ sót tôm khó nhận diện.

Nhóm	Tham số	Ý nghĩa
Nhận diện	CONF_SEG	Ngưỡng tin cậy của mô hình segmentation. Giá trị cao làm mask chặt hơn, giá trị thấp giúp lấy đủ thân hơn nhưng dễ thêm nhiễu.
Nhận diện	BBOX_PAD	Số pixel mở rộng quanh bounding box trước khi tạo vùng đo. Tham số này giúp tránh cắt sát đầu hoặc đuôi tôm.
Quy đổi	SCALE	Hệ số quy đổi từ pixel sang milimet, đơn vị mm/px. Chiều dài thực được tính bằng $\text{pixel_length} * \text{SCALE}$.
Video	TARGET_FPS	Số frame mỗi giây được lấy từ video. Giá trị 0 nghĩa là xử lý toàn bộ frame.
Video	LINE_GAP_RATIO	Tỉ lệ khoảng cách giữa các vạch tham chiếu quanh tâm khung hình.
Video	TOUCH_THRESHOLD	Dung sai khi kiểm tra tâm bounding box có chạm vạch tham chiếu hay không.
Video	REQUIRE_TOUCH	Số vạch cần chạm trước khi một Track ID được đưa sang bước segmentation.
Video	CONVEYOR_VERTICAL	Xác định hướng di chuyển của tôm. Bật khi tôm di chuyển theo chiều dọc ảnh, tắt khi di chuyển theo chiều ngang.
Video	CHUNK_MODE	Cho phép giữ trạng thái tracking giữa nhiều video liên tiếp nếu các video là các đoạn của cùng một lần đo.
Lưu trữ	SAVE	Bật hoặc tắt việc lưu ảnh debug ở các flow F3, F4, F5 và F6.
Lưu trữ	CLEAR_INPUT	Nếu bật, file đầu vào được xóa sau khi kết quả JSON của nguồn đó đã được ghi thành công.
Lưu trữ	CLEAR_OUTPUT	Nếu bật, thư mục output cũ được xóa trước khi tạo run mới.
Định dạng	IMG_EXTS	Danh sách đuôi file ảnh được hệ thống chấp nhận.
Định dạng	VID_EXTS	Danh sách đuôi file video được hệ thống chấp nhận.

Ngoài các tham số pipeline, hệ thống còn có nhóm tham số kích cỡ để phân loại kết quả đo.

Bảng 6.2. Các tham số cấu hình phân loại kích cỡ tôm

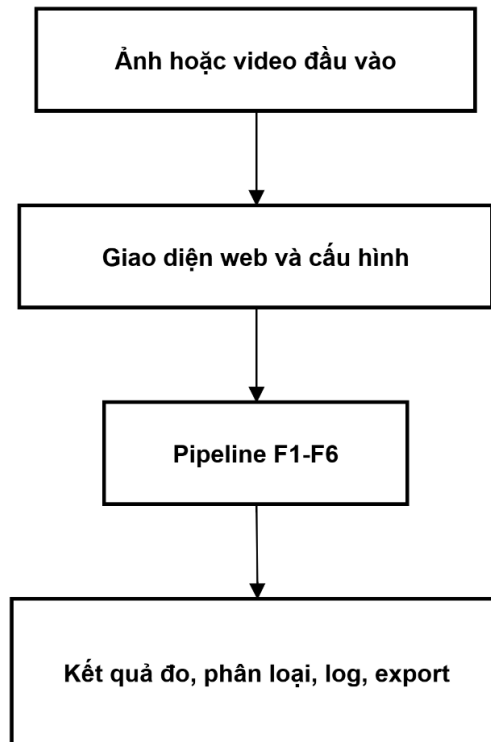
Tham số	Ý nghĩa
SIZE_RANGES	Định nghĩa các khoảng chiều dài tôm theo milimet.
UNDERSIZE_LABEL	Nhãn cho trường hợp tôm nhỏ hơn khoảng kích cỡ đầu tiên.
OVERSIZE_LABEL	Nhãn cho trường hợp tôm lớn hơn khoảng kích cỡ cuối cùng.
FALLBACK_LABEL	Nhãn dự phòng khi kết quả đo không khớp khoảng nào.

6.1.4 Các thành phần chính của hệ thống

Hệ thống gồm ba nhóm thành phần chính. Nhóm thứ nhất là pipeline xử lý ảnh và video, nơi các flow F1 đến F6 thực hiện toàn bộ quá trình đo và phân loại. Nhóm thứ hai là giao diện web, cho phép người dùng nạp dữ liệu, chỉnh cấu hình, chạy pipeline, theo dõi log và xem kết quả. Nhóm thứ ba là phần lưu trữ và export dữ liệu, phục vụ ghi nhận kết quả xử lý sau mỗi lần chạy.

Cách tổ chức này giúp tách rõ phần xử lý lõi và phần vận hành. Pipeline tập trung vào suy luận mô hình, xử lý ảnh, tracking, segmentation mask, skeleton và quy đổi chiều dài. Giao diện web đóng vai trò điều khiển và hiển thị. Phần lưu trữ bảo đảm kết quả được ghi lại để kiểm tra, hiệu chuẩn và export.

Sơ đồ tổng quan của hệ thống được mô tả như sau:



Hình 6.1. Lưu đồ tổng quan hệ thống đo và phân loại kích cỡ tôm

Lưu đồ thể hiện ý tưởng tách hệ thống thành ba lớp rõ ràng. Giao diện web là lớp vận hành, giúp người dùng đưa dữ liệu vào, điều chỉnh cấu hình và theo dõi tiến trình. Pipeline F1-F6 là lõi xử lý, nơi dữ liệu được phát hiện, tách mask, đo skeleton và quy đổi chiều dài. Kết quả đo, log và dữ liệu export là lớp kiểm tra và báo cáo, giúp người dùng đối chiếu lại kết quả sau mỗi lần chạy.

Giao diện web được chia thành các tab Dữ liệu, Kết quả, Cấu hình, Kích cỡ và Log. Tab Dữ liệu hỗ trợ nạp file hoặc folder bằng thao tác chọn file và kéo thả, hiển thị danh sách file đầu vào và cho phép xóa file không cần xử lý. Tab Kết quả cho phép chọn run, lọc theo kích cỡ, xem ảnh debug, export CSV hoặc Excel và bật chế độ tính scale. Tab Cấu hình và tab Kích cỡ cho phép lưu tham số vận hành và bảng phân loại vào settings.json, còn tab Log hiển thị tiến trình xử lý theo thời gian thực.

Trước khi pipeline chạy, hệ thống kiểm tra settings.json. Nếu file cấu hình bị thiếu hoặc hỏng, hệ thống phục hồi file về trạng thái mặc định và yêu cầu người dùng kiểm tra lại trước khi đo. Khi tải mô hình, hệ thống dùng OpenVINO cache trong thư mục dự án để giảm thời gian khởi tạo ở các lần chạy sau.

Ở mức thiết kế, giao diện web không thực hiện trực tiếp các thuật toán đo mà đóng vai trò điều khiển quy trình. Người dùng nạp dữ liệu, điều chỉnh tham số, khởi động pipeline và theo dõi kết quả thông qua giao diện. Phần backend tiếp nhận yêu cầu, kiểm tra cấu hình, gọi chương trình xử lý chính và đọc dữ liệu đầu ra để hiển thị lại cho người dùng. Cách phân chia này giúp phần xử lý lõi có thể chạy độc lập, đồng thời vẫn có giao diện thuận tiện cho vận hành và kiểm thử.

6.2 KIẾN TRÚC XỬ LÝ TỔNG THỂ CỦA HỆ THỐNG

6.2.1 Luồng xử lý từ dữ liệu đầu vào đến kết quả đầu ra

Luồng xử lý bắt đầu từ dữ liệu ảnh hoặc video được người dùng đưa vào hệ thống. Với ảnh, từng ảnh được xem như một mẫu quan sát độc lập. Với video, hệ thống đọc từng frame và theo dõi đối tượng qua thời gian để tránh đo lặp lại cùng một con tôm.

Sau khi có dữ liệu đầu vào, hệ thống phát hiện vùng chứa tôm bằng YOLOv8 Detect. Với ảnh tĩnh, mỗi bounding box hợp lệ được chuyển tiếp sang bước segmentation. Với video, bounding box còn được dùng cho ByteTrack để duy trì Track ID trước khi chọn frame đại diện. Khi có vùng cần đo, YOLOv8 Segment tạo segmentation mask, sau đó hệ thống tạo skeleton và tính chiều dài theo pixel. Kết quả cuối cùng được quy đổi sang milimet bằng scale factor rồi phân loại theo bảng kích cỡ.

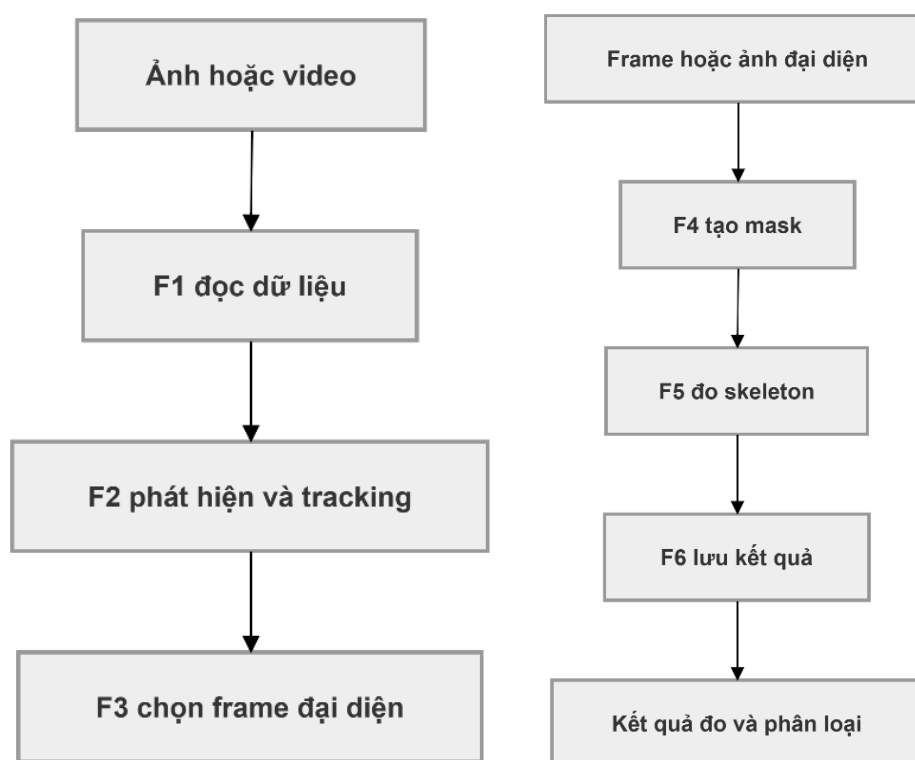
6.2.2 Kiến trúc pipeline đa luồng

Pipeline được chia thành sáu flow gồm F1 đọc dữ liệu, F2 phát hiện và tracking, F3 chọn frame đại diện, F4 segmentation, F5 đo chiều dài và F6 lưu kết quả. Mỗi flow được tổ chức như một bước độc lập trong chuỗi xử lý. Dữ liệu giữa hai flow liên kế được truyền qua hàng đợi trung gian, nhờ đó flow phía trước có thể gửi kết quả đã hoàn thành, còn flow phía sau nhận dữ liệu theo thứ tự để tiếp tục xử lý.

Lý do chọn kiến trúc đa luồng xuất phát từ yêu cầu xử lý video. Video tạo ra lượng frame lớn. Nếu xử lý tuần tự, một frame phải đi hết toàn bộ các bước trước khi frame tiếp theo được xử lý, làm nhiều bước phía sau phải chờ. Khi chia thành nhiều flow và chạy gói đầu nhau, F1 có thể đọc frame mới trong khi F2 phát hiện frame khác, F4 segmentation dữ liệu khác và F6 lưu kết quả đã hoàn thành.

Để bảo đảm quá trình xử lý ổn định, các flow không thao tác trực tiếp lên cùng một dữ liệu trung gian mà trao đổi dữ liệu thông qua hàng đợi. Cách tổ chức này giúp giảm nguy cơ xung đột khi nhiều flow chạy đồng thời. Khi một flow hoàn tất hoặc không còn dữ liệu đầu vào, hệ thống gửi sentinel sang hàng đợi kế tiếp để các flow phía sau dừng theo đúng thứ tự. Nếu một flow gặp lỗi, stop event được bật và sentinel được đưa vào các hàng đợi còn lại để tránh trường hợp flow khác tiếp tục chờ dữ liệu không còn được tạo ra.

6.2.3 Lưu đồ pipeline F1 đến F6



Hình 6.2. Lưu đồ pipeline từ F1 đến F6

Hai lưu đồ mô tả pipeline theo hai phần. Phần F1-F3 là phần chuẩn bị dữ liệu đo, gồm đọc dữ liệu, định vị vùng chứa tâm và chọn ảnh hoặc frame đại diện. Phần F4-F6 là phần đo và lưu kết quả, gồm tạo mask sạch, đo chiều dài theo skeleton, quy đổi sang milimet và phân loại kích cỡ. Cách chia này giúp phần xử lý nặng chỉ chạy trên dữ liệu đã được chọn lọc thay vì xử lý dư trên toàn bộ frame video.

Dữ liệu trung gian thay đổi theo từng flow. Sau F2, hệ thống có bounding box và thông tin tracking. Sau F3, dữ liệu video có frame đại diện. Sau F4, hệ thống có crop mask nhị phân của vùng cần đo. Sau F5, hệ thống có chiều dài theo pixel. Sau F6, dữ liệu được chuyển thành kết quả đo, nhóm kích cỡ và dữ liệu lưu trữ. Việc tách detection, chọn frame đại diện và segmentation thành các flow riêng giúp video không phải chạy segmentation trên mọi frame, đồng thời vẫn giữ được frame đủ rõ để đo.

6.2.4 Cơ chế ghi log và kiểm soát lỗi

Hệ thống ghi log trong suốt quá trình chạy pipeline để người dùng biết đang xử lý đến bước nào. Log cũng giúp nhận biết các trường hợp bị bỏ qua như ảnh lỗi, video không đọc được, không có segmentation mask hoặc skeleton không hợp lệ.

Khi một flow gặp lỗi, hệ thống dừng quy trình theo cách có kiểm soát để tránh các flow khác bị treo khi đang chờ dữ liệu. Cơ chế dừng gửi tín hiệu kết thúc đến các hàng đợi phía sau, nhờ đó toàn bộ quy trình có thể thoát ổn định thay vì dừng đột ngột giữa chừng.

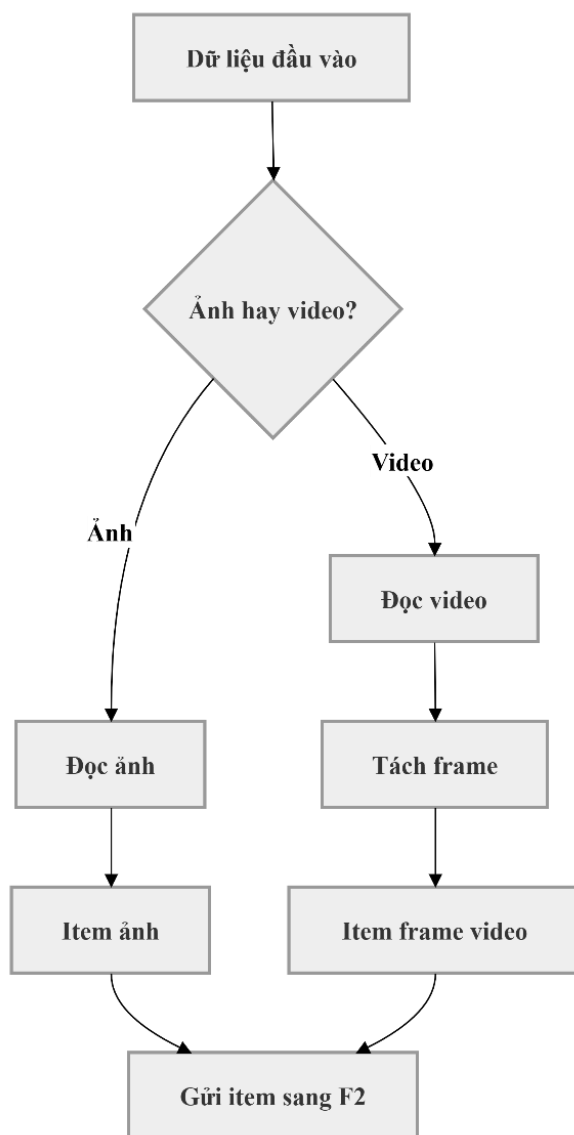
Ngoài log sự kiện, hệ thống còn ghi nhận thời gian xử lý của từng flow. Thông tin này giúp xác định bước nào chiếm nhiều thời gian trong pipeline, đặc biệt khi đánh

giá dữ liệu video. Khi có lỗi trong một thread, tên flow, loại lỗi và traceback được ghi vào log để hỗ trợ kiểm tra nguyên nhân thay vì chỉ thông báo pipeline thất bại chung chung.

6.3 THIẾT KẾ CÁC FLOW XỬ LÝ

6.3.1 F1 - Đọc dữ liệu đầu vào

F1 thực hiện bước đọc dữ liệu đầu vào. Với ảnh tĩnh, hệ thống đọc ảnh và tạo dữ liệu ảnh để chuyển sang F2. Với video, hệ thống mở video, đọc thông tin kích thước, tốc độ frame, tính các vạch tham chiếu và lần lượt đưa các frame vào pipeline.



Hình 6.3. Lưu đồ đọc dữ liệu đầu vào trong F1

Lưu đồ cho thấy F1 tách hai luồng dữ liệu chính. Với ảnh, hệ thống đọc ảnh và tạo item ảnh. Với video, hệ thống đọc video, tách frame và tạo item frame video trước khi chuyển sang F2.

Ý nghĩa của F1 là chuẩn hóa nhiều dạng đầu vào về cùng một cấu trúc item để các flow sau xử lý thống nhất. Nhờ đó, từ F2 trở đi pipeline không cần quan tâm dữ liệu

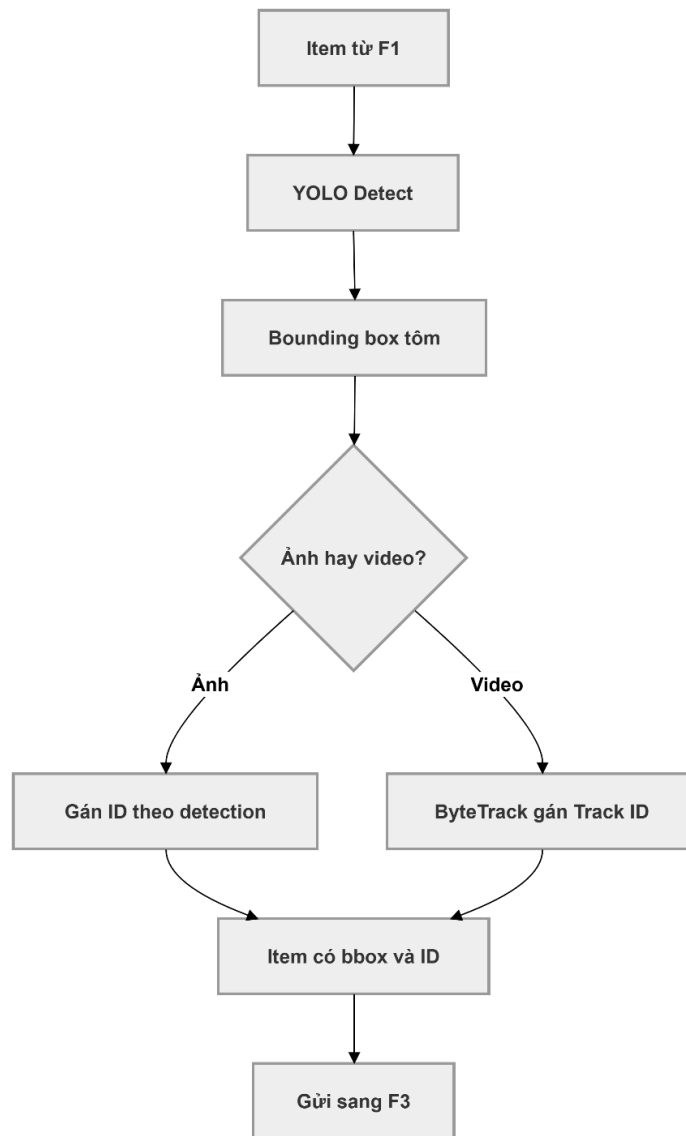
ban đầu được nạp từ ảnh tĩnh hay từ video, mà chỉ cần đọc các thông tin đã được đóng gói trong item như ảnh/frame, nguồn dữ liệu và metadata đi kèm.

6.3.2 F2 - Phát hiện và tracking

F2 thực hiện phát hiện tôm và tracking trong video. Ở giai đoạn đầu, hệ thống thử xử lý trực tiếp bằng YOLOv8n-seg. Cách này cho mask khá tốt khi ảnh chỉ có một con tôm, vì đối tượng cần tách rõ ràng và ít bị ảnh hưởng bởi cá thể khác.

Khi mở rộng sang trường hợp nhiều con tôm nằm cạnh nhau, mask từ YOLOv8n-seg dễ bị đè lẫn giữa các cá thể, làm vùng thân cần đo không còn tách bạch. Để cải thiện chất lượng mask, hệ thống thử nâng lên YOLOv8xl-seg. Mô hình lớn hơn cho kết quả segmentation tốt hơn trong một số ảnh nhiều con, nhưng khi chuyển sang video thì phát sinh vấn đề về tốc độ. Video cần tracking qua nhiều frame, nếu dùng mô hình segmentation lớn cho từng frame thì quá trình xử lý trở nên rất chậm.

Vì vậy, phương án cuối cùng là tách bài toán thành hai mô hình. F2 dùng mô hình detection để lấy bounding box nhanh và kết hợp ByteTrack để duy trì Track ID trong video. Sau đó, các vùng cần đo mới được chuyển sang bước segmentation ở F4. Cách tổ chức này giúp cân bằng giữa tốc độ xử lý video và chất lượng mask ở bước đo chiều dài.



Hình 6.4. Lưu đồ phát hiện và tracking trong F2

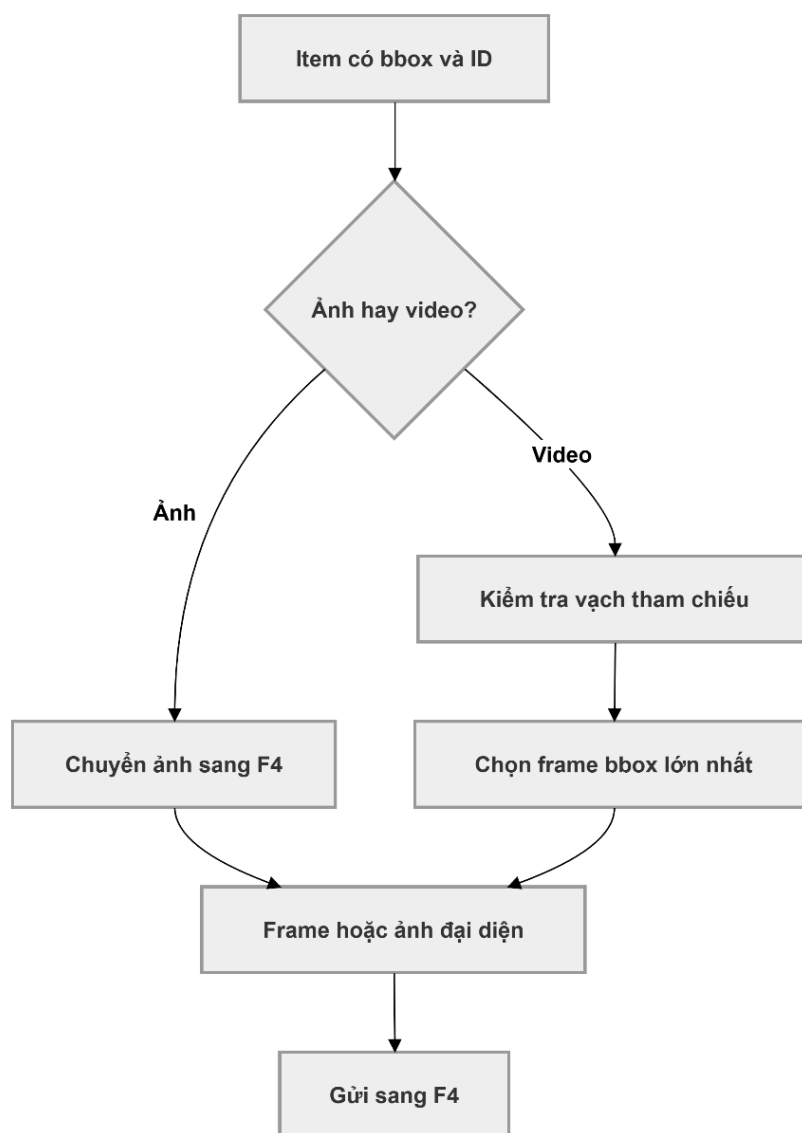
Lưu đồ cho thấy F2 nhận item từ F1, dùng YOLO Detect để tạo bounding box tôm, sau đó rẽ nhánh theo loại dữ liệu. Với ảnh, ID được gán theo detection. Với video, ByteTrack gán Track ID trước khi gửi dữ liệu sang F3.

Ý nghĩa của F2 là định vị nhanh vùng chứa tôm, chưa thực hiện đo chiều dài ở bước này. Bounding box là đầu ra quan trọng vì nó giúp F3 theo dõi cá thể trong video và giúp F4 giới hạn vùng segmentation, tránh xử lý toàn bộ ảnh. Với video, ByteTrack giữ định danh cá thể qua nhiều frame để hệ thống biết các detection nào thuộc cùng một con tôm.

Nếu mô hình không phát hiện tôm, F2 không ghi log riêng cho từng frame mà quay lại nhận item tiếp theo. Cách xử lý này phù hợp với video vì nhiều frame có thể không chứa đối tượng cần đo, nếu ghi log cho từng frame trống thì log vận hành sẽ bị nhiễu.

6.3.3 F3 - Chọn frame đại diện

F3 xử lý khác nhau giữa ảnh và video. Nếu dữ liệu là ảnh, dữ liệu được chuyển thẳng sang F4 vì ảnh không có chuỗi frame để lựa chọn. Nếu dữ liệu là video, hệ thống theo dõi từng Track ID, kiểm tra tóm đã chạm đủ các vạch tham chiếu và chọn frame đại diện trước khi đưa sang segmentation.



Hình 6.5. Lưu đồ chọn frame đại diện trong F3

Lưu đồ thể hiện điểm khác nhau giữa ảnh và video. Với ảnh, dữ liệu được chuyển thẳng sang F4. Với video, F3 kiểm tra vạch tham chiếu và chọn frame có bounding box lớn nhất làm frame đại diện trước khi gửi sang F4.

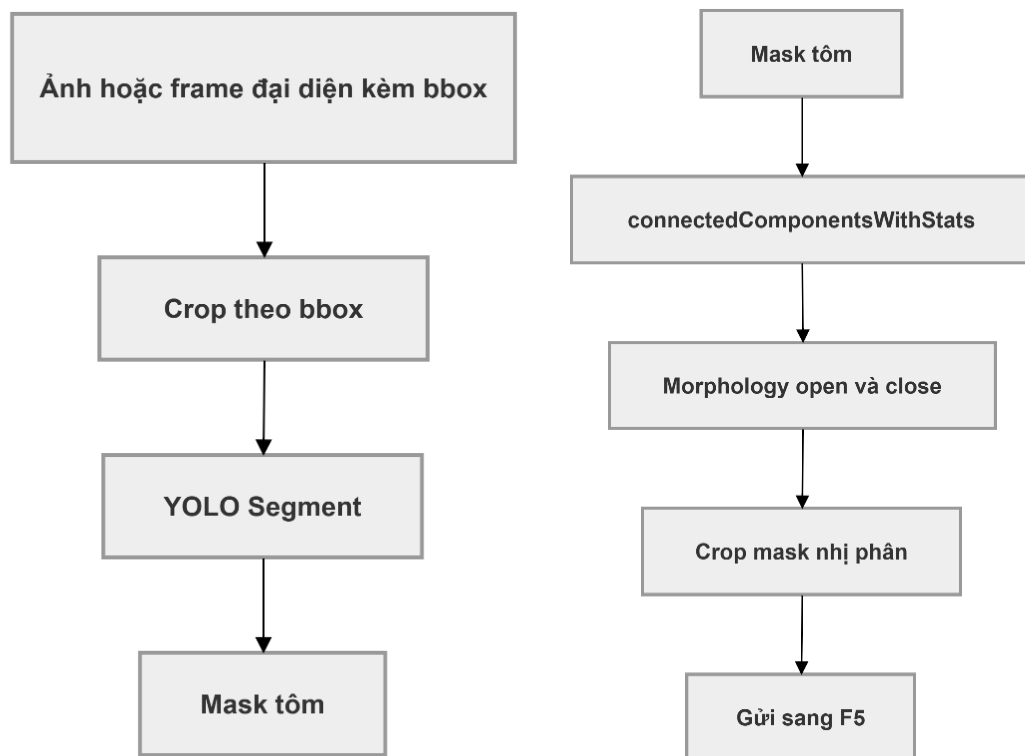
Ý nghĩa của F3 là tránh đo lặp cùng một cá thể trong video. Một con tôm có thể xuất hiện ở nhiều frame liên tiếp, nhưng hệ thống chỉ cần chọn một frame đại diện đủ tốt để đo. Frame đại diện giúp giảm số lần segmentation, giảm thời gian xử lý và ưu tiên thời điểm tôm xuất hiện rõ hơn trong vùng quan sát.

F3 có hai thời điểm cần flush track. Khi 'CHUNK_MODE' tắt, các track của video cũ được flush trước khi hệ thống chuyển sang video mới. Khi 'CHUNK_MODE'

bật, hệ thống giữ trạng thái tracking qua các đoạn video liên tiếp và chỉ flush các track còn lại khi toàn bộ input kết thúc. Ở thời điểm kết thúc, track có thể được chuyển sang F4 nếu đã có frame đại diện hợp lệ và đã chạm ít nhất một vạch tham chiếu.

6.3.4 F4 - Segmentation vùng tôm

F4 dùng YOLOv8 Segment để lấy segmentation mask của vùng tôm đã được giới hạn bởi kết quả detection. Hệ thống không cắt ảnh nhỏ rồi mới chạy segmentation. Thay vào đó, F2 hoặc F3 tạo một ảnh có cùng kích thước với frame gốc, trong đó vùng ngoài bounding box được che bằng nền xám. YOLOv8 Segment chạy trên ảnh đã được che này để tập trung vào cá thể cần đo, sau đó F4 mới crop mask theo bounding box segmentation.



Hình 6.6. Lưu đồ tạo mask trong F4

Hai lưu đồ mô tả F4 theo hai phần. Phần đầu nhận ảnh hoặc frame đại diện kèm bounding box, tạo ảnh đã che vùng ngoài bounding box và chạy YOLOv8 Segment để tạo mask tôm. Phần sau giữ thành phần liên thông chính bằng 'connectedComponentsWithStats', áp dụng morphology open và close, sau đó tạo crop mask nhị phân để gửi sang F5.

Ý nghĩa của F4 là biến kết quả segmentation thành crop mask nhị phân đủ sạch để F5 có thể tạo skeleton. 'connectedComponentsWithStats' giúp giữ vùng tôm chính khi mask có nhiều thành phần rời rạc, còn morphology open và close giúp giảm nhiễu nhỏ và lấp các lỗ nhỏ trong vùng thân tôm. Vì F5 đo trên mask, chất lượng của F4 ảnh hưởng trực tiếp đến độ ổn định của chiều dài skeleton.

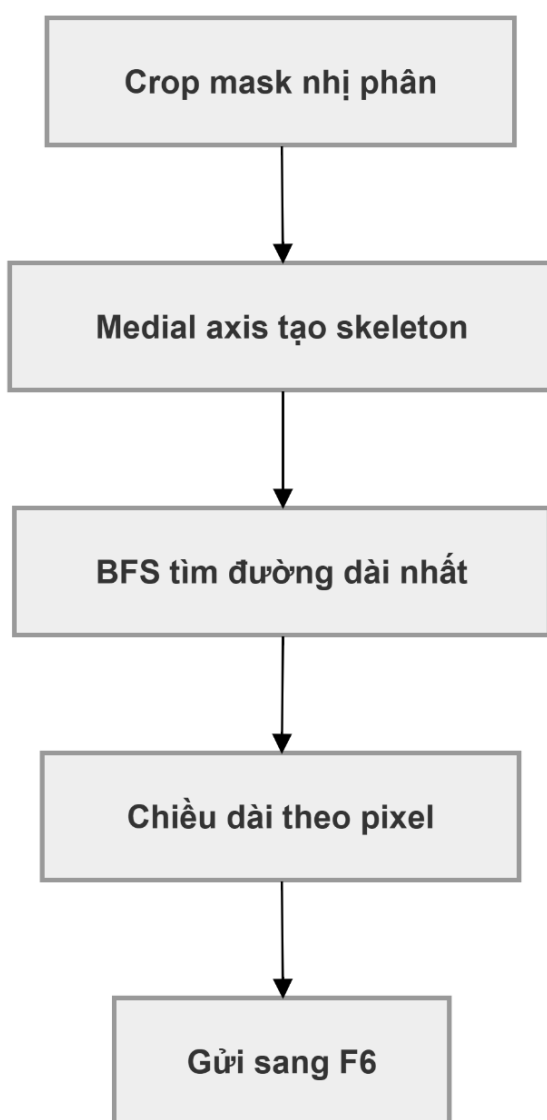
F4 vận hành theo vòng nhận dữ liệu liên tục. Khi nhận tín hiệu kết thúc, flow chuyển tiếp tín hiệu này sang F5 và kết thúc. Khi không có mask, item hiện tại được bỏ

qua sau khi ghi warning. Khi có mask, hệ thống tạo crop_mask và crop_box rồi chuyển dữ liệu sang F5.

6.3.5 F5 - Đo chiều dài bằng skeleton

F5 nhận segmentation mask đã cắt từ F4 và tạo skeleton để đo chiều dài. Cách đo bằng bounding box không phù hợp khi tôm nằm cong, vì bounding box chỉ biểu diễn khu vực bao quanh theo hình chữ nhật và không bám theo đường thân tôm.

Ở phiên bản ban đầu, mask được truyền sang F5 theo đúng kích thước gốc của frame. Cách này vẫn cho phép tạo skeleton, nhưng hàm medial axis phải xử lý toàn bộ vùng ảnh lớn, trong khi phần thực sự cần đo chỉ là vùng thân tôm. Sau đó, hệ thống chuyển sang sử dụng crop mask quanh vùng segmentation của tôm. Nhờ giảm kích thước vùng xử lý, số pixel đưa vào bước tạo skeleton ít hơn, từ đó thời gian xử lý của medial axis giảm rõ rệt hơn.



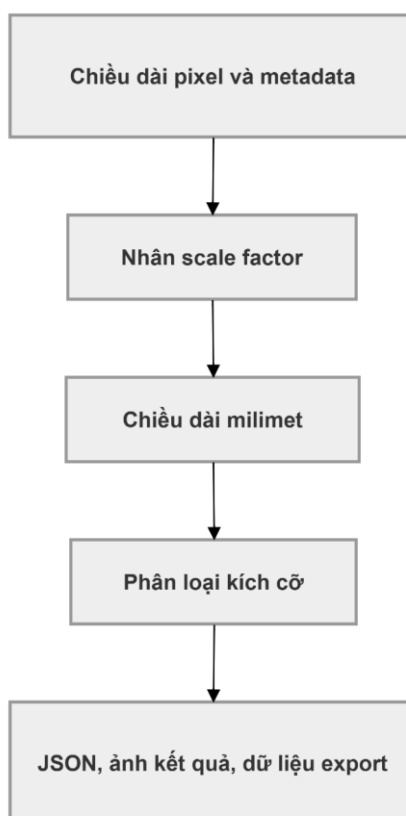
Hình 6.7. Lưu đồ đo chiều dài bằng skeleton trong F5

Lưu đồ mô tả F5 nhận crop mask nhị phân, tạo skeleton bằng medial axis, dùng BFS để tìm đường dài nhất và trả ra chiều dài theo pixel cho F6.

Ý nghĩa của F5 là chuyển mask vùng thân tôm thành đường trục để đo theo hình dạng thực tế thay vì đo theo khung bao. Skeleton thu được không phải lúc nào cũng là một đường thẳng duy nhất, mà có thể xuất hiện các nhánh phụ do râu, chân, nhiều mask hoặc vùng thân bị đứt nhẹ. Vì vậy, BFS được dùng để tìm đường dài nhất trên skeleton, xem đường này là đại diện cho trục thân chính của tôm.

6.3.6 F6 - Quy đổi, phân loại và lưu kết quả

F6 nhận chiều dài theo pixel từ F5, quy đổi sang chiều dài thực theo milimet bằng scale factor, phân loại kích cỡ và lưu kết quả. Kết quả lưu gồm thông tin nguồn dữ liệu, Track ID nếu có, chiều dài pixel, chiều dài thực, nhóm kích cỡ và ảnh minh họa.



Hình 6.8. Lưu đồ quy đổi, phân loại và lưu kết quả trong F6

Lưu đồ cho thấy F6 nhận chiều dài pixel và metadata, quy đổi sang milimet, phân loại kích cỡ và tạo dữ liệu kết quả. Kết quả này gồm JSON, ảnh kết quả nếu cần và dữ liệu phục vụ export.

Ý nghĩa của F6 là biến kết quả đo theo pixel thành dữ liệu có thể sử dụng trong vận hành: chiều dài milimet, nhãn kích cỡ, tệp kết quả và dữ liệu export. Đây là điểm kết thúc của pipeline, nơi kết quả từ các flow trước được gom lại thành bản ghi hoàn chỉnh. F6 lưu kết quả theo từng nguồn dữ liệu riêng, nhờ đó kết quả của các ảnh hoặc video khác nhau không bị trộn lẫn khi hệ thống xử lý nhiều nguồn liên tiếp.

6.4 CÁC LỰA CHỌN THIẾT KẾ CHÍNH

6.4.1 Kết hợp YOLOv8 Detect và YOLOv8 Segment

Hệ thống không dùng duy nhất một mô hình cho toàn bộ pipeline. YOLOv8 Detect được dùng để phát hiện nhanh vị trí tôm, đặc biệt trong video có nhiều frame. YOLOv8 Segment được dùng sau khi vùng quan tâm đã được giới hạn, giúp segmentation mask tập trung hơn vào thân tôm cần đo.

Lựa chọn này giải quyết hai yêu cầu cùng lúc. Bước detection giữ tốc độ xử lý ở mức phù hợp hơn cho video, trong khi bước segmentation cung cấp hình dạng chi tiết để tạo skeleton. Nếu chỉ dùng detection, hệ thống không có mask đủ chi tiết để đo tôm cong. Nếu chạy segmentation trực tiếp trên toàn bộ video, chi phí xử lý sẽ tăng mạnh.

6.4.2 Chọn frame đại diện trước khi segmentation

Với video, cùng một con tôm có thể xuất hiện ở nhiều frame liên tiếp. Nếu đưa toàn bộ frame sang segmentation, hệ thống sẽ đo lặp lại cùng một cá thể và làm tăng thời gian xử lý. Vì vậy, F3 chọn frame đại diện dựa trên Track ID, vạch tham chiếu và diện tích bounding box.

Frame tốt nhất được chọn khi cá thể đã chạm đủ số vạch cần thiết và có bounding box lớn nhất trong các frame ứng viên. Tiêu chí này giúp ưu tiên thời điểm tôm xuất hiện rõ trong vùng quan sát, đồng thời giảm số lần phải chạy segmentation.

6.4.3 Đo chiều dài trên skeleton thay vì bounding box

Bounding box phù hợp để định vị nhanh nhưng không phản ánh đường thân khi tôm cong hoặc nằm chéo. Vì vậy, hệ thống chuyển segmentation mask thành skeleton rồi tìm đường dài nhất trên skeleton. Chiều dài đường này được xem là chiều dài theo pixel của thân tôm.

Cách đo bằng skeleton phù hợp với mục tiêu đo chiều dài theo hình dạng thân. Dù vậy, chất lượng kết quả vẫn phụ thuộc vào segmentation mask. Nếu mask bị thiếu vùng đầu, đuôi hoặc chứa nhiều từ chân, râu và cá thể bên cạnh, skeleton có thể bị rút ngắn hoặc sinh nhánh phụ.

6.4.4 Hiệu chuẩn scale factor bằng nhiều mẫu

Scale factor quyết định trực tiếp việc quy đổi chiều dài từ pixel sang milimet. Ở cách làm ban đầu, hệ số này có thể được ước lượng bằng một mẫu hoặc trung bình từ một số mẫu đo thử. Tuy nhiên, nếu một vài mẫu có đường đo chưa chính xác do mask hoặc skeleton bị lệch, kết quả hiệu chuẩn cũng bị kéo lệch theo.

Vì vậy, hệ thống dùng nhiều mẫu tôm đã biết chiều dài thật để ước lượng scale factor. Người dùng có thể nhập trực tiếp chiều dài thật trong bảng kết quả hoặc nạp file CSV/XLSX có một cột mm để tự động điền nhanh. Thay vì phụ thuộc vào một mẫu đơn lẻ, hệ thống ước lượng hệ số quy đổi từ nhiều cặp giá trị giữa chiều dài pixel và chiều dài thật, sau đó trả về RMSE để người dùng đánh giá sai số hiệu chuẩn.

6.4.5 Tổ chức kết quả theo run và hỗ trợ export

Mỗi lần chạy pipeline tạo một thư mục run theo thời gian trong OUTPUT_DIR. Bên trong run, kết quả được tách theo từng nguồn dữ liệu để người dùng dễ đối chiếu giữa file đầu vào, ảnh debug và JSON kết quả.

Giao diện đọc danh sách run mới nhất, cho phép chọn run cần xem, lọc theo nhãn kích cỡ và export dữ liệu ra CSV hoặc Excel. Cách tổ chức này giúp kết quả đo không bị ghi đè giữa các lần chạy, đồng thời thuận tiện cho việc tổng hợp số liệu sau thí nghiệm.

Mỗi bản ghi kết quả cần chứa đủ thông tin để truy vết lại quá trình đo, gồm tên nguồn, Track ID nếu có, chiều dài theo pixel, chiều dài theo milimet, nhãn kích cỡ và các đường dẫn ảnh minh họa. Khi cần kiểm tra sai số, người dùng có thể mở lại run tương ứng, xem JSON kết quả, đối chiếu ảnh debug và xuất bảng dữ liệu để xử lý tiếp bằng công cụ bên ngoài.

CHƯƠNG 7. XÂY DỰNG ỨNG DỤNG VÀ KIỂM THỬ VẬN HÀNH

7.1 XÂY DỰNG PIPELINE XỬ LÝ

7.1.1 Tổ chức chương trình chính

Ứng dụng được xây dựng theo hướng tách chương trình điều phối, pipeline xử lý, cấu hình, giao diện web và các hàm hỗ trợ vẽ ảnh minh họa. Chương trình chính chịu trách nhiệm đọc cấu hình, tạo thư mục kết quả, khởi tạo logger, tải hai mô hình YOLOv8, tạo các queue trung gian và chạy sáu flow xử lý trong các thread riêng.

Hai mô hình được dùng trong giai đoạn vận hành là YOLOv8 Detect và YOLOv8 Segment. Mô hình Detect được dùng ở F2 để phát hiện nhanh bounding box và hỗ trợ tracking. Mô hình Segment được dùng ở F4 để tạo segmentation mask cho vùng tumor đã được giới hạn. Khi mô hình đã được export sang định dạng OpenVINO, pipeline có thể dùng định dạng này cho bước suy luận trong môi trường cục bộ.

7.1.2 Cài đặt F1 đọc dữ liệu đầu vào

F1 quét thư mục đầu vào và xử lý file theo phần mở rộng. File ảnh hợp lệ được đọc thành item ảnh, còn file video hợp lệ được tách thành các frame theo step cấu hình. Nếu `TARGET\_FPS` lớn hơn 0, step được tính từ FPS gốc của video để giảm số frame cần xử lý. Nếu `TARGET\_FPS = 0`, hệ thống xử lý toàn bộ frame với step bằng 1. Các file không thuộc nhóm ảnh hoặc video được bỏ qua âm thầm để tránh làm nhiễu vận hành. Trường hợp ảnh hoặc video không đọc được vẫn được ghi log vì đây là lỗi dữ liệu đầu vào cần người dùng biết.

```

def flow1_read_input(
    q_f1_f2,
    flow_times: dict[str, float],
    run_dir: str,
    cfg: dict,
    stop_event,
) -> None:
    """
    Đọc ảnh hoặc video từ INPUT_DIR và đẩy frame sang F2.

    Ảnh tạo một item có type `image`, frame_idx 0 và không có line tham chiếu.
    Video tạo nhiều item type `video`, mỗi item có frame, frame_idx, lines và
    run_dir. Khi quét xong hoặc INPUT_DIR không tồn tại, hàm gửi sentinel None
    vào q_f1_f2 và ghi thời gian vào flow_times["F1"].
    """

    log = get_logger()
    start_time = time.perf_counter()
    input_dir = cfg["INPUT_DIR"]
    img_exts = set(cfg["IMG_EXTS"])
    vid_exts = set(cfg["VID_EXTS"])
    input_path = Path(input_dir)
    stop_state = {"logged": False}

    if not input_path.exists():
        log.warning(f"[F1] Thư mục '{input_dir}' không tồn tại!")
        _put_queue(q_f1_f2, None, stop_event)
        flow_times["F1"] = time.perf_counter() - start_time
        return

    log.info(f"[F1] Quét thư mục '{input_dir}'")
    for fpath in sorted(input_path.iterdir()):
        if _flow1_should_stop(stop_event, log, stop_state) or not _flow1_process_file(
            fpath,
            q_f1_f2,
            run_dir,
            cfg,
            img_exts,
            vid_exts,
            stop_event,
            log,
            stop_state,
        ):
            break

    _put_queue(q_f1_f2, None, stop_event)
    flow_times["F1"] = time.perf_counter() - start_time
    if _stop_requested(stop_event):
        log.info(f"[F1] Dừng sớm | {flow_times['F1']:.2f}s")
    else:
        log.info(f"[F1] Hoàn tất | {flow_times['F1']:.2f}s")

```

Hình 7.1. Mã nguồn đọc dữ liệu đầu vào trong F1

7.1.3 Cài đặt F2 phát hiện và tracking

F2 nhận dữ liệu từ F1 và chạy YOLOv8 Detect trên từng frame hoặc ảnh. Với ảnh tĩnh, mỗi detection hợp lệ tạo một item riêng và được gán ID theo thứ tự detection. Với video, detection được đưa qua ByteTrack để duy trì Track ID qua nhiều frame. Nếu

một frame không có detection, F2 quay lại nhận dữ liệu tiếp theo mà không ghi log riêng.

```
def flow2_detect_track(model_det, q_f1_f2, q_f2_f3, flow_times: dict[str, float], cfg: dict, stop_event) -> None:
    """
    Chạy YOLO detect và gán track_id trước khi chuyển sang F3.

    Với ảnh tĩnh, mỗi detection hợp lệ tạo một item riêng có masked_img,
    orig_img, track_id và debug_images rỗng. Với video, hàm dùng ByteTrack để
    cập nhật tracker_id và chỉ chuyển các frame có detection. Khi nhận sentinel
    None từ F1, hàm gửi None sang q_f2_f3.
    """
    log = get_logger()
    start_time = time.perf_counter()
    bbox_pad = cfg["BBOX_PAD"]
    chunk_mode = cfg["CHUNK_MODE"]
    conf_det = cfg["CONF_DET"]
    tracker: sv.ByteTrack | None = None
    current_video = None

    while True:
        item = _get_queue(q_f1_f2, stop_event)
        if item is None:
            _put_queue(q_f2_f3, None, stop_event)
            break

        results = model_det.predict(
            source=item["frame"],
            verbose=False,
            conf=conf_det,
            device=DEVICE,
        )[0]
        detections = sv.Detections.from_ultralytics(results)

        if item["type"] == "image":
            if len(detections) > 0:
                log.info(f"[F2] {item['source_file']}: phát hiện {len(detections)} con tôm")
                for i, box_xyxy in enumerate(detections.xyxy):
                    masked_img = get_masked_image(item["frame"], box_xyxy, pad=bbox_pad)
                    if masked_img is not None:
                        pushed = _put_queue(q_f2_f3, {
                            "type": "image",
                            "source_file": item["source_file"],
                            "source_stem": item["source_stem"],
                            "track_id": i + 1,
                            "frame_idx": 0,
                            "masked_img": masked_img,
                            "orig_img": item["frame"],
                            "lines": {},
                            "debug_images": {},
                            "run_dir": item["run_dir"],
                        }, stop_event)
                    if not pushed:
                        break

            elif item["type"] == "video":
                if item["path"] != current_video:
                    if tracker is None or not chunk_mode:
                        tracker = sv.ByteTrack()
                    log.info(f"[F2] Tracker mới cho: {item['path'].name}")
                    current_video = item["path"]

                detections = tracker.update_with_detections(detections)
                if len(detections) > 0:
                    item["detections"] = detections
                    if not _put_queue(q_f2_f3, item, stop_event):
                        break

    flow_times["F2"] = time.perf_counter() - start_time
    log.info(f"[F2] Hoàn tất | {flow_times['F2']:.2f}s")
```

Hình 7.2. Mã nguồn phát hiện và tracking trong F2

7.1.4 Cài đặt F3 chọn frame đại diện

F3 chuyển thẳng dữ liệu ảnh sang F4, nhưng xử lý video theo Track ID. Với mỗi Track ID, hệ thống ghi nhận các vạch tham chiếu đã chạm, lưu frame có bounding box lớn nhất và flush track khi đủ điều kiện. Khi `CHUNK\_MODE` tắt, track của video cũ được flush khi hệ thống chuyển sang video mới. Khi `CHUNK\_MODE` bật, trạng thái tracking được giữ qua các đoạn video và chỉ flush khi toàn bộ input kết thúc.

```
def flow3_touch_logic(q_f2_f3, q_f3_f4, flow_times: dict[str, float], cfg: dict, stop_event) -> None:
    """
    Chọn frame tốt nhất trước khi segmentation.

    Ảnh tĩnh được chuyển thẳng sang F4. Với video, hàm theo dõi từng track,
    ghi nhận các vạch đã chạm, lưu frame có bbox lớn nhất và chỉ gửi sang F4
    khi track chạm đủ 3 vạch. Khi video đổi hoặc nhận sentinel None, các track
    còn hoạt động được flush sang F4 nếu có frame tốt nhất.
    """
    log = get_logger()
    start_time = time.perf_counter()
    bbox_pad = cfg["BBOX_PAD"]
    chunk_mode = cfg["CHUNK_MODE"]
    conveyor_vertical = cfg["CONVEYOR_VERTICAL"]
    save_debug = cfg["SAVE"]
    touch_threshold = cfg["TOUCH_THRESHOLD"]
    required_touches = cfg["REQUIRE_TOUCH"]
    active_tracks = {}
    completed_tracks = set()
    current_video = None
    current_lines = {}
    current_run_dir = ""

    while True:
        item = _get_queue(q_f2_f3, stop_event)

        if item is None:
            if not _stop_requested(stop_event):
                _flow3_flush_all_active_tracks(
                    current_video,
                    active_tracks,
                    current_lines,
                    current_run_dir,
                    bbox_pad,
                    save_debug,
                    required_touches,
                    q_f3_f4,
                    stop_event,
                )
                _put_queue(q_f3_f4, None, stop_event)
                break

            if item["type"] == "image":
                if not _put_queue(q_f3_f4, item, stop_event):
                    break
                continue

            video_path = item["path"]
            frame_idx = item["frame_idx"]
            lines = item["lines"]
            detections = item["detections"]
```



```
if video_path != current_video:
    if not chunk_mode:
        _flow3_flush_all_active_tracks(
            current_video,
            active_tracks,
            current_lines,
            current_run_dir,
            bbox_pad,
            save_debug,
            required_touches,
            q_f3_f4,
            stop_event,
        )
        active_tracks.clear()
        completed_tracks.clear()
    current_video = video_path
    current_lines = lines
    current_run_dir = item["run_dir"]

centers = detections.get_anchors_coordinates(sv.Position.CENTER)

for box_xyxy, track_id, area, (cx, cy) in zip(
    detections.xyxy,
    detections.tracker_id,
    detections.area,
    centers,
):

    track_id = int(track_id)
    area = float(area)
    coord = float(cy) if conveyor_vertical else float(cx)

    if track_id in completed_tracks:
        continue

    if track_id not in active_tracks:
        active_tracks[track_id] = {
            "lines_touched": set(),
            "touch_records": [] if save_debug else None,
            "best_box_xyxy": None,
            "best_frame": None,
            "best_frame_idx": None,
            "best_area": 0.0,
        }

    track_data = active_tracks[track_id]
```



```
for line_id, line_pos in lines.items():
    if line_id in track_data["lines_touched"]:
        continue
    if not box_touches_line(coord, line_pos, touch_threshold):
        continue

    track_data["lines_touched"].add(line_id)

    if save_debug and track_data["touch_records"] is not None:
        track_data["touch_records"].append({
            "frame": item["frame"],
            "box_xyxy": box_xyxy.copy(),
            "area": area,
            "frame_idx": frame_idx,
            "line_id": line_id,
        })

    if area > track_data["best_area"]:
        track_data["best_box_xyxy"] = box_xyxy.copy()
        track_data["best_frame"] = item["frame"]
        track_data["best_frame_idx"] = frame_idx
        track_data["best_area"] = area

if _flow3_has_required_touches(track_data, lines, required_touches):
    _flow3_flush_track_to_f4(
        track_id,
        track_data,
        video_path.stem,
        video_path.name,
        lines,
        item["run_dir"],
        bbox_pad,
        save_debug,
        required_touches,
        q_f3_f4,
        stop_event,
    )
    completed_tracks.add(track_id)
    del active_tracks[track_id]

flow_times["F3"] = time.perf_counter() - start_time
log.info(f"[F3] Hoàn tất | {flow_times['F3']:.2f}s")
```

Hình 7.3. Mã nguồn kiểm tra vạch tham chiếu trong F3

Các ảnh minh họa F3 cho thấy hệ thống lưu nhiều frame ứng viên khi tôm chạm vạch tham chiếu, sau đó chọn frame có diện tích bounding box lớn nhất làm frame đại diện.



Hình 7.4. Frame ứng viên khi tôm chạm vạch tham chiếu trong F3



Hình 7.5. Frame đại diện được chọn trong F3

7.1.5 Cài đặt F4 segmentation vùng tôm

F4 chạy YOLOv8 Segment trên ảnh đã được che vùng ngoài bounding box, không chạy trên ảnh crop nhỏ. Sau khi mô hình trả kết quả, hệ thống chọn detection có diện tích lớn nhất, giữ connected component lớn nhất nếu mask bị rời rạc, crop mask theo bbox segmentation, áp dụng morphology open và close rồi chuyển dữ liệu sang F5. Nếu không có mask, item hiện tại được bỏ qua sau khi ghi warning.

```

def flow4_segment(model_seg, q_f3_f4, q_f4_f5, flow_times: dict[str, float], cfg: dict, stop_event) -> None:
    """
    Chạy YOLO segment trên masked_img và tạo crop mask cho F5.

    Hàm chọn detection có bbox lớn nhất, giữ component mask lớn nhất nếu mask
    có nhiều vùng rời rạc, crop mask theo bbox đã pad và cập nhật item bằng
    'crop_mask', 'crop_box', 'cx_label', 'cy_label'. Nếu không tìm thấy mask,
    item bị bỏ qua và lỗi được ghi log.
    """
    log = get_logger()
    start_time = time.perf_counter()
    bbox_pad = cfg["BBOX_PAD"]
    conf_seg = cfg["CONF_SEG"]
    save_debug = cfg["SAVE"]
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))

    while True:
        item = _get_queue(q_f3_f4, stop_event)
        if item is None:
            _put_queue(q_f4_f5, None, stop_event)
            break

        results = model_seg.predict(
            source=item["masked_img"],
            verbose=False,
            conf=conf_seg,
            retina_masks=True,
            device=DEVICE,
        )[0]
        detections = sv.Detections.from_ultralytics(results)

        if len(detections) == 0 or detections.mask is None:
            log.warning(f"[F4] {item['source_stem']} ID {item['track_id']}: không tìm thấy mask")
            continue

        xyxy = detections.xyxy
        box_areas = (
            (xyxy[:, 2] - xyxy[:, 0]) *
            (xyxy[:, 3] - xyxy[:, 1])
        )
        best_idx = int(box_areas.argmax())
        best_det = detections[[best_idx]]
        seg_xyxy = best_det.xyxy[0]
        mask_full = best_det.mask[0].astype(np.uint8)

        num_labels, labels, stats, _ = cv2.connectedComponentsWithStats(mask_full)
        if num_labels > 2:
            largest_label = 1 + int(np.argmax(stats[1:, cv2.CC_STAT_AREA]))
            mask_full = (labels == largest_label).astype(np.uint8)

        h, w = mask_full.shape
        x1 = max(0, int(seg_xyxy[0]) - bbox_pad)
        y1 = max(0, int(seg_xyxy[1]) - bbox_pad)
        x2 = min(w, int(seg_xyxy[2]) + bbox_pad)
        y2 = min(h, int(seg_xyxy[3]) + bbox_pad)
        crop_mask = mask_full[y1:y2, x1:x2]

        crop_mask = cv2.morphologyEx(crop_mask, cv2.MORPH_OPEN, kernel)
        crop_mask = cv2.morphologyEx(crop_mask, cv2.MORPH_CLOSE, kernel)

        if save_debug:
            paths = save_f4_debug(item, mask_full, seg_xyxy, crop_mask)
            item["debug_images"].update(paths)

        annot_cx, annot_cy = best_det.get_anchors_coordinates(sv.Position.CENTER)[0]

        item.pop("masked_img")
        item.update({
            "crop_mask": crop_mask,
            "crop_box": (x1, y1, x2, y2),
            "cx_label": annot_cx,
            "cy_label": annot_cy,
        })
        if not _put_queue(q_f4_f5, item, stop_event):
            break
        log.info(f"[F4] {item['source_stem']} ID {item['track_id']} phân đoạn xong")

    flow_times["F4"] = time.perf_counter() - start_time
    log.info(f"[F4] Hoàn tất | {flow_times['F4']:.2f}s")

```

Hình 7.6. Mã nguồn segmentation vùng tôm trong F4



Hình 7.7. Kết quả segmentation vùng tôm trong F4



Hình 7.8. Mask thu được sau segmentation trong F4

7.1.6 Cài đặt F5 đo chiều dài bằng skeleton

F5 nhận crop mask từ F4, tạo skeleton bằng medial axis và tìm đường dài nhất bằng BFS hai lần. Kết quả chiều dài theo pixel được tính theo đường dài nhất trên skeleton, trong đó bước ngang hoặc dọc được tính là `1`, còn bước chéo được tính là ` $\sqrt{2}$ `.

```

def flow5_longest_path(q_f4_f5, q_f5_f6, flow_times: dict[str, float], cfg: dict, stop_event) -> None:
    """
    Tính skeleton và chiều dài pixel của tôm.

    Hàm dùng medial_axis trên crop_mask, gọi find_longest_path() để lấy
    path_mask và pixel_length, lưu ảnh debug F5 nếu SAVE bật, rồi chuyển item
    đã cập nhật sang F6. Sentinel None được chuyển tiếp sang q_f5_f6.
    """
    log = get_logger()
    start_time = time.perf_counter()
    save_debug = cfg["SAVE"]

    while True:
        item = _get_queue(q_f4_f5, stop_event)
        if item is None:
            _put_queue(q_f5_f6, None, stop_event)
            break

        skeleton = medial_axis(item["crop_mask"], rng=42)
        path_mask, pixel_length = find_longest_path(skeleton)

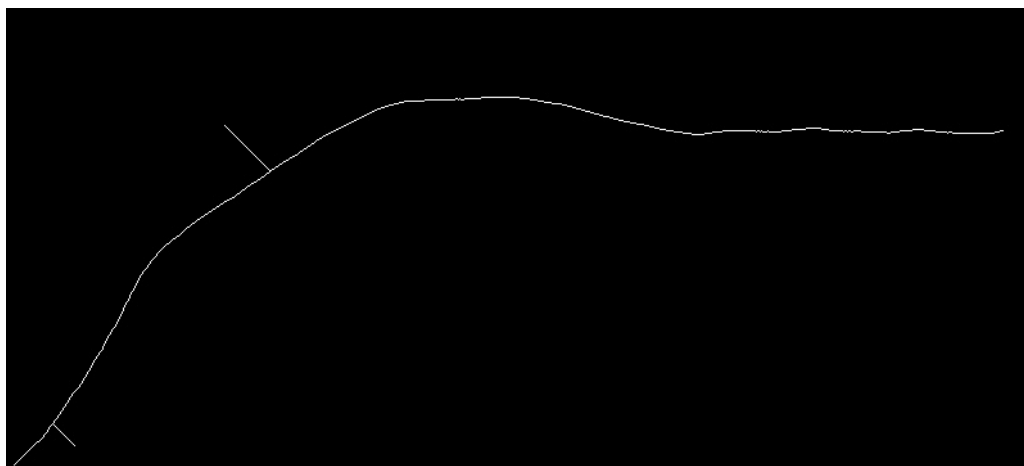
        if save_debug:
            paths = save_f5_debug(item, skeleton, path_mask)
            item["debug_images"].update(paths)

        item.update({
            "path_mask": path_mask,
            "pixel_length": pixel_length,
        })
        if not _put_queue(q_f5_f6, item, stop_event):
            break

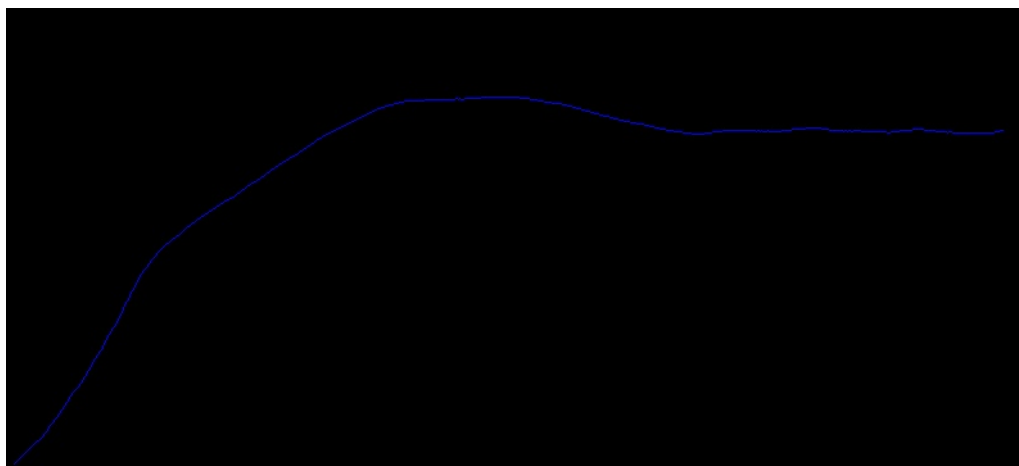
    flow_times["F5"] = time.perf_counter() - start_time
    log.info(f"[F5] Hoàn tất | {flow_times['F5']:.2f}s")

```

Hình 7.9. Mã nguồn đo chiều dài bằng skeleton trong F5



Hình 7.10. Skeleton được tạo từ mask trong F5

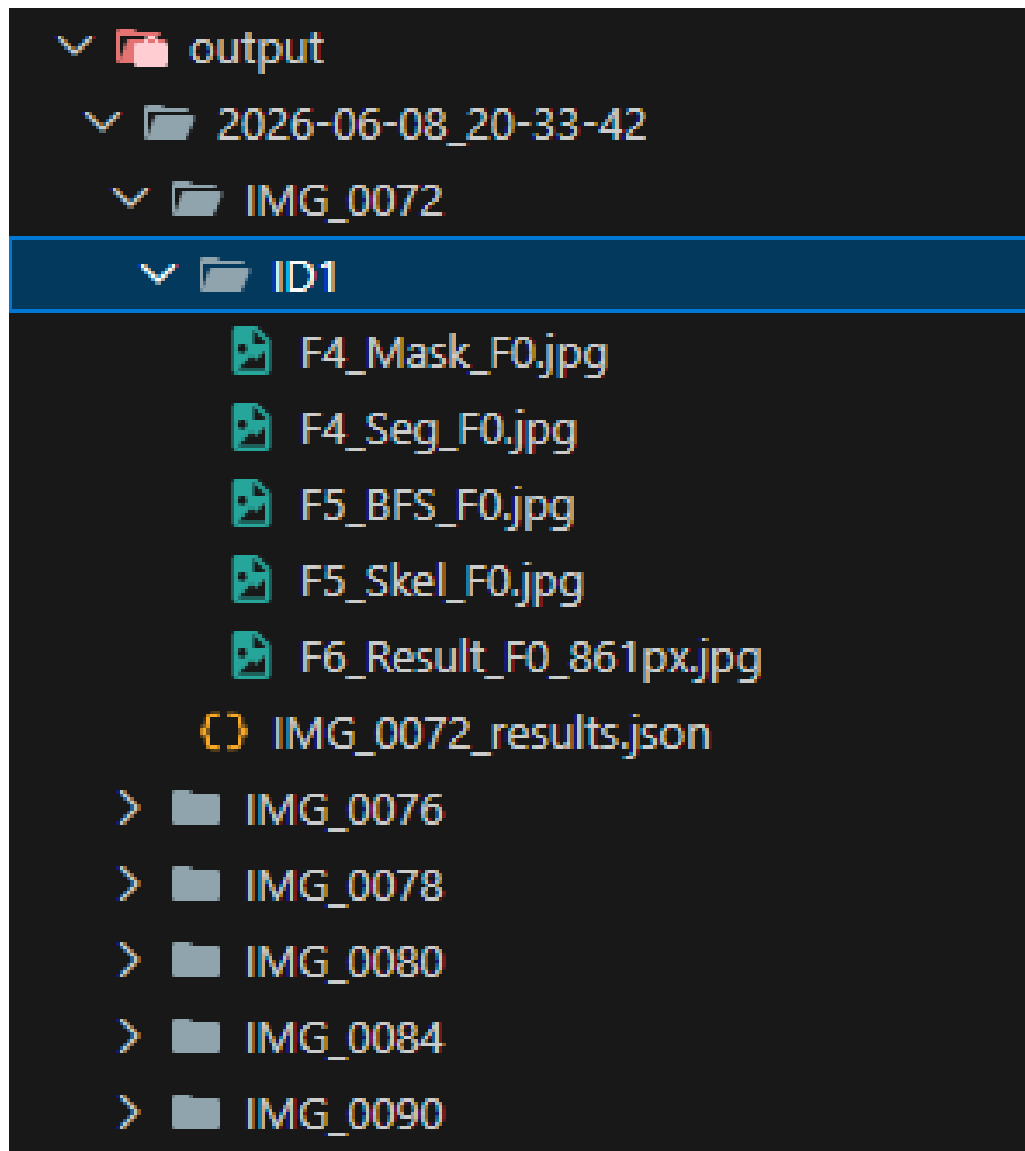


Hình 7.11. Đường dài nhất trên skeleton trong F5

7.1.7 Cài đặt F6 quy đổi và lưu kết quả

F6 nhận chiều dài theo pixel từ F5, nhân với scale factor để có chiều dài thực theo milimet, phân loại theo bảng kích cỡ và ghi kết quả vào JSON. Kết quả được gom theo source_stem. Khi source_stem thay đổi, JSON của source trước được flush ra file trước khi xử lý source mới. Khi nhận sentinel cuối pipeline, source cuối cùng cũng được flush để bảo đảm không mất kết quả.

Sau khi F6 ghi kết quả, dữ liệu đầu ra được tổ chức theo từng run. Mỗi run gồm các thư mục theo nguồn dữ liệu, bên trong tiếp tục tách theo từng ID cá thể. Cách tổ chức này giúp người dùng đối chiếu file JSON kết quả với các ảnh minh họa trung gian của cùng một cá thể.

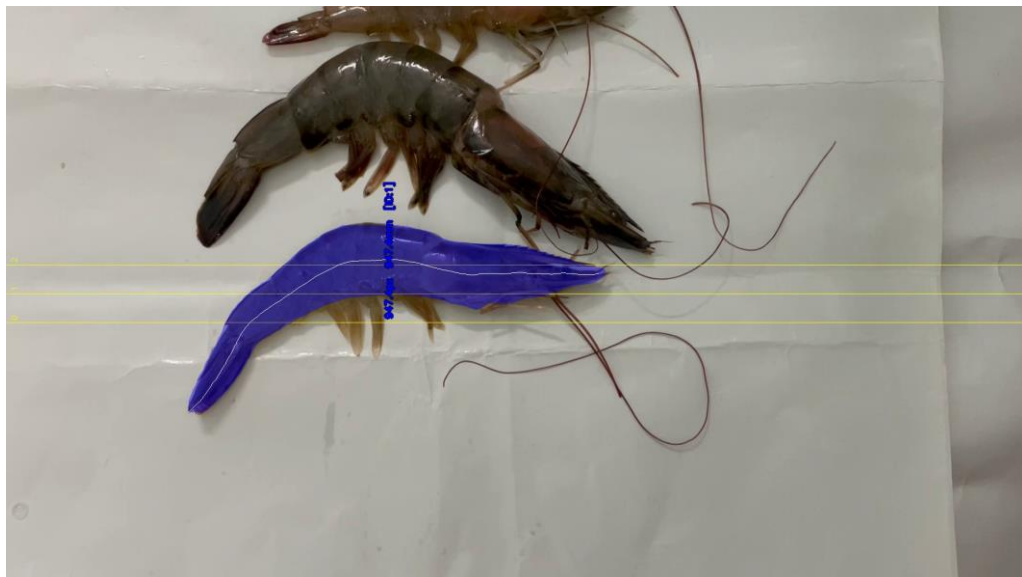


Hình 7.12. Cấu trúc thư mục output sau khi pipeline hoàn thành

File JSON kết quả lưu thông tin nguồn dữ liệu, thời điểm xử lý, scale factor và danh sách cá thể đã đo. Với mỗi cá thể, file lưu Track ID, frame được chọn, chiều dài theo pixel, chiều dài theo milimet, nhân kích cỡ và đường dẫn tới các ảnh minh họa.

```
{
  "source_file": "IMG_0072.jpg",
  "source_stem": "IMG_0072",
  "processed_at": "2026-06-08T20:33:47",
  "scale_mm_per_px": 0.188475,
  "shrimps": [
    {
      "track_id": 1,
      "frame_idx": 0,
      "pixel_length": 861.0,
      "real_length_mm": 162.28,
      "size": "M",
      "images": {
        "f4_seg": "output\\2026-06-08_20-33-42\\IMG_0072\\ID1\\F4_Seg_F0.jpg",
        "f4_mask": "output\\2026-06-08_20-33-42\\IMG_0072\\ID1\\F4_Mask_F0.jpg",
        "f5_skel": "output\\2026-06-08_20-33-42\\IMG_0072\\ID1\\F5_Skel_F0.jpg",
        "f5_bfs": "output\\2026-06-08_20-33-42\\IMG_0072\\ID1\\F5_BFS_F0.jpg",
        "f6_result": "output\\2026-06-08_20-33-42\\IMG_0072\\ID1\\F6_Result_F0_861px.jpg"
      }
    }
  ]
}
```

Hình 7.13. Cấu trúc file JSON kết quả sau khi xử lý



Hình 7.14. Kết quả đo chiều dài và phân loại trong F6

7.1.8 Cấu hình suy luận bằng OpenVINO

Trong giai đoạn vận hành, đường dẫn mô hình detection và segmentation được khai báo trong config.py ở dạng thư mục OpenVINO. Khi main.py khởi động, chương trình cấu hình cache OpenVINO trong thư mục dự án trước khi Ultralytics tải mô hình. Cách làm này giúp pipeline dùng mô hình đã export cho suy luận cục bộ, đồng thời tránh phụ thuộc vào file trọng số huấn luyện trong quá trình chạy ứng dụng.

Việc tách mô hình huấn luyện và mô hình suy luận cũng làm rõ vai trò của từng giai đoạn. Chương trình huấn luyện tạo ra trọng số tốt nhất, sau đó mô hình được export sang định dạng OpenVINO để dùng trong pipeline vận hành. Trong pipeline, F2 dùng mô hình detection OpenVINO để phát hiện và tracking, còn F4 dùng mô hình segmentation OpenVINO để tạo segmentation mask cho vùng đã được giới hạn.

7.2 XÂY DỰNG GIAO DIỆN WEB

7.2.1 Flask và API vận hành

Giao diện web được xây dựng bằng Flask để người dùng vận hành pipeline mà không cần chạy lệnh thủ công. Backend cung cấp các API phục vụ upload file, chạy pipeline, đọc trạng thái, đọc log theo offset, lưu cấu hình, lưu bảng kích cỡ, đọc danh sách run, đọc kết quả, export CSV/XLSX và hiệu chuẩn scale factor.

Trước khi chạy pipeline, backend kiểm tra cấu hình hiện tại. Nếu settings.json bị thiếu hoặc sai định dạng, hệ thống phục hồi giá trị mặc định và yêu cầu người dùng kiểm tra lại cấu hình trước khi chạy. Cách xử lý này giúp hạn chế trường hợp pipeline chạy với cấu hình không hợp lệ.

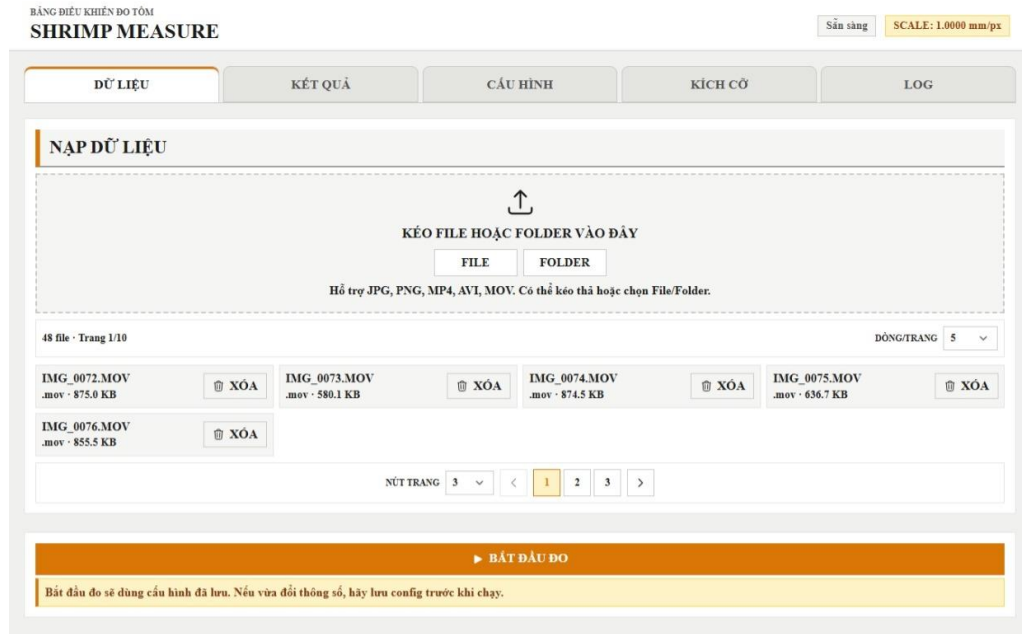
Backend cũng kiểm tra giá trị cấu hình trước khi lưu, gồm ngưỡng confidence, scale factor, khoảng cách vạch, số vạch cần chạm và các tham số dạng số. Với file đầu vào, hệ thống chỉ nhận phần mở rộng nằm trong danh sách cho phép, chuẩn hóa tên file khi upload và kiểm tra đường dẫn khi xóa file. Khi phục vụ ảnh debug từ thư mục output, backend kiểm tra đường dẫn để tránh truy cập ra ngoài thư mục kết quả.

Khi người dùng yêu cầu chạy pipeline, backend khởi động chương trình xử lý chính như một subprocess riêng. Trạng thái chạy được lưu ở backend để tránh trường hợp người dùng bấm chạy nhiều lần trong khi pipeline cũ chưa kết thúc. Trong quá trình xử lý, giao diện gọi API trạng thái để biết pipeline còn đang chạy hay đã hoàn tất, đồng thời gọi API log theo offset để chỉ lấy phần log mới thay vì tải lại toàn bộ file log.

7.2.2 Giao diện nạp dữ liệu và chạy pipeline

Tab dữ liệu là nơi người dùng chọn ảnh hoặc video đầu vào và bắt đầu chạy pipeline. Cách tổ chức này giúp thao tác vận hành tập trung ở một màn hình, phù hợp hơn với người dùng không trực tiếp làm việc với mã nguồn.

Người dùng có thể nạp nhiều file ảnh hoặc video trong một lần thao tác. Backend kiểm tra phần mở rộng theo danh sách định dạng được cấu hình, lưu các file hợp lệ vào thư mục input và đưa các file không hỗ trợ vào danh sách bị từ chối. Nếu tên file bị trùng, hệ thống tự tạo tên mới để tránh ghi đè dữ liệu đã có. Tab dữ liệu cũng cho phép xem danh sách file đầu vào hiện tại và xóa các file không cần xử lý trước khi chạy pipeline.



Hình 7.15. Giao diện tab dữ liệu dùng để nạp đầu vào và chạy pipeline

7.2.3 Giao diện cấu hình tham số xử lý

Tab cấu hình cho phép người dùng điều chỉnh các tham số ảnh hưởng trực tiếp đến pipeline như ngưỡng detection, ngưỡng segmentation, khoảng mở rộng bounding box, scale factor, tùy chọn lưu ảnh minh họa và các tham số liên quan đến video.

Nhóm tham số video hỗ trợ điều chỉnh cách hệ thống lấy frame và kiểm tra tôm đi qua vùng quan sát. Người dùng có thể cấu hình số vạch cần chạm, khoảng cách giữa các vạch tham chiếu, hướng di chuyển của băng chuyền và tốc độ lấy frame xử lý. Các tùy chọn lưu ảnh minh họa, xóa dữ liệu đầu vào sau khi chạy, xóa kết quả cũ hoặc giữ lại kết quả cũ giúp người dùng linh hoạt hơn khi thử nghiệm nhiều dataset.

BẢNG ĐIỀU KHIỂN ĐO Tôm

SHRIMP MEASURE

hình ảnh

SCALE: 1.0000 mm/px

DỮ LIỆU

KẾT QUẢ

CẤU HÌNH

KÍCH CỠ

LOG

CẤU HÌNH XỬ LÝ

QUY ĐỔI

SCALE quyết định một pixel tương ứng bao nhiêu milimet.

SCALE

1

Giá trị mm/pixel. SCALE càng lớn thì chiều dài tính ra theo mm càng lớn.

NHẬN DIỆN

Các ngưỡng này ảnh hưởng trực tiếp đến việc tìm tôm, tách thân và cắt vùng đo.

CONF_DET

50%

Độ chắc chắn tối thiểu để nhận một vật là tôm. Tăng để bớt nhận nhầm, giảm để ít bỏ sót.

CONF_SEG

50%

Độ chắc chắn khi tách thân tôm khỏi nền. Tăng để mạnh mẽ hơn, giảm để lấy đủ thân hơn.

BBOX_PAD

5

Số pixel nơi thêm quanh khung tôm trước khi đo. Dùng khi ảnh bị cắt quá sát thân tôm.

TOUCH_THRESHOLD

10

Khoảng dung sai khi xét tôm chạm vạch biên. Giá trị lớn hơn sẽ dễ tính là chạm vạch hơn.

VIDEO

Chọn cách đọc video, hướng di chuyển và điều kiện chạm vạch tham chiếu.

LINE_GAP_RATIO

10%

Tỷ lệ khoảng cách từ vạch giữa ra hai vạch biên, tính trên nửa chiều di chuyển.

REQUIRE_TOUCH

3 vạch

Số vạch tôm bắt buộc phải chạm trước khi được chuyển sang bước phân đoạn.

TARGET_FPS

0

Số frame lấy mỗi giây từ video. Nhập 0 để xử lý toàn bộ frame.

CHUNK_MODE

Bật khi nhiều video là các đoạn nối tiếp của cùng một lần đo.

CONVEYOR_VERTICAL

Bật nếu tôm di chuyển theo chiều dọc ảnh; tắt nếu tôm di chuyển ngang.

THƯ MỤC & LƯU/XÓA

Chọn nơi lấy dữ liệu đầu vào, nơi ghi kết quả và các tùy chọn dọn file.

INPUT_DIR

input

Thư mục chứa ảnh/video đầu vào cần đo.

OUTPUT_DIR

output

Thư mục lưu ảnh kết quả và file JSON đầu ra.

SAVE

Bật để lưu ảnh debug và ghi đường dẫn ảnh vào JSON.

CLEAR_INPUT

Bật để xóa ảnh/video đầu vào sau khi đo xong. Cần nhắc nếu cần giữ file gốc.

CLEAR_OUTPUT

Bật để xóa kết quả cũ trong OUTPUT_DIR trước khi bắt đầu lần đo mới.

LƯU CONFIG

Hình 7.16. Giao diện tab cấu hình tham số xử lý

7.2.4 Giao diện kết quả và hiệu chuẩn scale factor

Tab kết quả hỗ trợ người dùng xem danh sách tôm đã đo, ảnh minh họa, chiều dài theo pixel, chiều dài thực theo milimet và nhóm kích cỡ. Từ dữ liệu kết quả này, người dùng có thể nhập chiều dài thật để hiệu chuẩn lại scale factor.

Hệ thống dùng mô hình hồi quy tuyến tính qua gốc tọa độ với công thức chiều dài thực bằng scale factor nhân với chiều dài theo pixel. Từ các cặp dữ liệu chiều dài theo pixel và chiều dài thật, scale factor được tính sao cho sai số bình phương giữa chiều dài quy đổi và chiều dài thật là nhỏ nhất.

Kết quả được tổ chức theo từng run để người dùng có thể xem lại các lần chạy trước. Khi mở tab kết quả, giao diện có thể đọc run mới nhất hoặc run được chọn, sau đó hiển thị danh sách nguồn dữ liệu và các cá thể tôm đã đo. Dữ liệu trong run cũng là cơ sở để export CSV/XLSX, giúp người dùng đưa kết quả sang phần mềm bảng tính hoặc dùng cho báo cáo kiểm thử.

Ở chế độ hiệu chuẩn, người dùng có thể nhập chiều dài thật trực tiếp hoặc nạp file CSV/XLSX có một cột giá trị milimet. Các giá trị này được ghép tuần tự với danh sách tôm đang hiển thị trong run. Backend lấy chiều dài theo pixel từ JSON kết quả, tính scale factor mới, trả về RMSE của quá trình hiệu chuẩn và lưu lại hệ số SCALE vào cấu hình nếu dữ liệu hợp lệ.

BẢNG ĐIỀU KIỆN ĐO TÓM

SHRIMP MEASURE

Sản xuất

SCALE: 1.0000 mm/px

DỮ LIỆU

KẾT QUẢ

CẤU HÌNH

KÍCH CỠ

LOG

DANH SÁCH ĐO

2026-05-26_05-40-19 - 27 tôm

Tất cả size

TÍNH SCALE

CSV

EXCEL

TÁI LẠI

1-10 / 27 - Trang 1/3

ĐÓNG TRANG 10

27 người

27 tôm

Run: 2026-05-26_05-40-19

IMG_0072.JPG

1/1 tìm trên trang · tổng 1 · scale 0.16953

ID	FRAME	PIXEL	MM	SIZE	ẢNH
1	0	929.6	157.6	M	ẢNH

IMG_0074.JPG

1/1 tìm trên trang · tổng 1 · scale 0.16953

ID	FRAME	PIXEL	MM	SIZE	ẢNH
1	0	1139.5	193.19	L	ẢNH

IMG_0076.JPG

1/1 tìm trên trang · tổng 1 · scale 0.16953

ID	FRAME	PIXEL	MM	SIZE	ẢNH
1	0	707.1	119.87	S	ẢNH

IMG_0078.JPG

1/1 tìm trên trang · tổng 1 · scale 0.16953

ID	FRAME	PIXEL	MM	SIZE	ẢNH
1	0	991.5	168.09	L	ẢNH

IMG_0080.JPG

1/1 tìm trên trang · tổng 1 · scale 0.16953

ID	FRAME	PIXEL	MM	SIZE	ẢNH
1	0	1229	208.36	Ngoại cỡ lớn	ẢNH

IMG_0082.JPG

1/1 tìm trên trang · tổng 1 · scale 0.16953

ID	FRAME	PIXEL	MM	SIZE	ẢNH
1	0	918.6	155.73	M	ẢNH

IMG_0084.JPG

1/1 tìm trên trang · tổng 1 · scale 0.16953

ID	FRAME	PIXEL	MM	SIZE	ẢNH
1	0	802.4	136.03	M	ẢNH

IMG_0086.JPG

1/1 tìm trên trang · tổng 1 · scale 0.16953

ID	FRAME	PIXEL	MM	SIZE	ẢNH
1	0	1138.1	192.94	L	ẢNH

IMG_0088.JPG

1/1 tìm trên trang · tổng 1 · scale 0.16953

ID	FRAME	PIXEL	MM	SIZE	ẢNH
1	0	992.6	168.28	L	ẢNH

IMG_0090.JPG

1/1 tìm trên trang · tổng 1 · scale 0.16953

ID	FRAME	PIXEL	MM	SIZE	ẢNH
1	0	936.5	158.76	M	ẢNH

NÚT TRANG 3

1

2

3

Hình 7.17. Giao diện tab kết quả hỗ trợ xem kết quả và hiệu chuẩn scale factor

BẢNG ĐIỀU KHIỂN ĐO TÔM

SHRIMP MEASURE

Sản xuất

SCALE: 1.0000 mm/px

DỮ LIỆU

KẾT QUẢ

CẤU HÌNH

KÍCH CỠ

LOG

DANH SÁCH ĐO

2026-05-26_05-40-19 - 27 tôm

Tất cả size

TÍNH SCALE

CSV

EXCEL

TÁI LẠI

1-10 / 27 - Trang 1/3

ĐÓNG TRANG 10

27 người

27 tôm

Run: 2026-05-26_05-40-19

IMG_0072.JPG						1/1 tìm trên trang · tổng 1 · scale 0.16953
ID	FRAME	PIXEL	MM	SIZE	ẢNH	
1	0	929.6	157.6	M		ẢNH
IMG_0074.JPG						1/1 tìm trên trang · tổng 1 · scale 0.16953
ID	FRAME	PIXEL	MM	SIZE	ẢNH	
1	0	1139.5	193.19	L		ẢNH
IMG_0076.JPG						1/1 tìm trên trang · tổng 1 · scale 0.16953
ID	FRAME	PIXEL	MM	SIZE	ẢNH	
1	0	707.1	119.87	S		ẢNH
IMG_0078.JPG						1/1 tìm trên trang · tổng 1 · scale 0.16953
ID	FRAME	PIXEL	MM	SIZE	ẢNH	
1	0	991.5	168.09	L		ẢNH
IMG_0080.JPG						1/1 tìm trên trang · tổng 1 · scale 0.16953
ID	FRAME	PIXEL	MM	SIZE	ẢNH	
1	0	1229	208.36	Ngoài cỡ lớn		ẢNH
IMG_0082.JPG						1/1 tìm trên trang · tổng 1 · scale 0.16953
ID	FRAME	PIXEL	MM	SIZE	ẢNH	
1	0	918.6	155.73	M		ẢNH
IMG_0084.JPG						1/1 tìm trên trang · tổng 1 · scale 0.16953
ID	FRAME	PIXEL	MM	SIZE	ẢNH	
1	0	802.4	136.03	M		ẢNH
IMG_0086.JPG						1/1 tìm trên trang · tổng 1 · scale 0.16953
ID	FRAME	PIXEL	MM	SIZE	ẢNH	
1	0	1138.1	192.94	L		ẢNH
IMG_0088.JPG						1/1 tìm trên trang · tổng 1 · scale 0.16953
ID	FRAME	PIXEL	MM	SIZE	ẢNH	
1	0	992.6	168.28	L		ẢNH
IMG_0090.JPG						1/1 tìm trên trang · tổng 1 · scale 0.16953
ID	FRAME	PIXEL	MM	SIZE	ẢNH	
1	0	936.5	158.76	M		ẢNH

NÚT TRANG 3

< 1 2 3 >

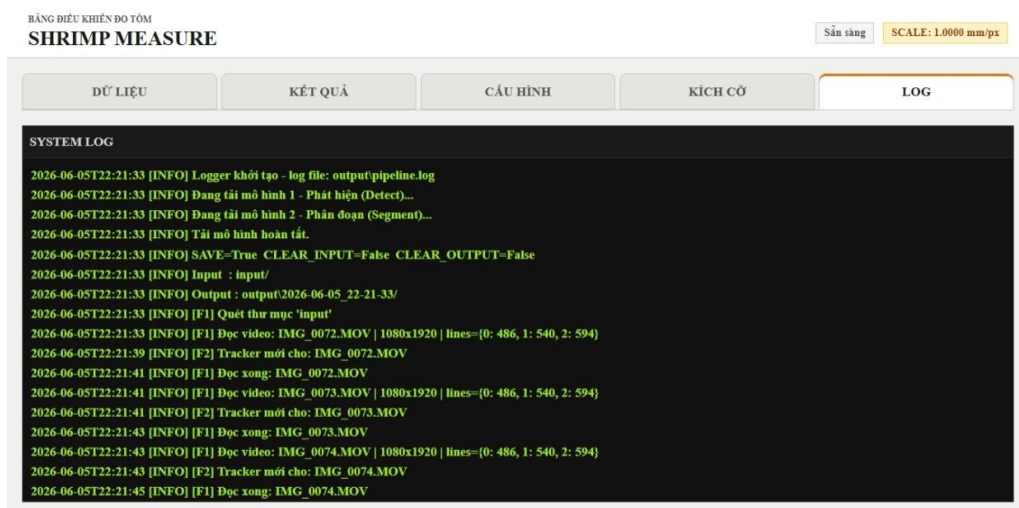
Hình 7.18. Giao diện tính scale factor từ dữ liệu đo thực tế

7.2.5 Giao diện log, ảnh minh họa và bảng kích cỡ

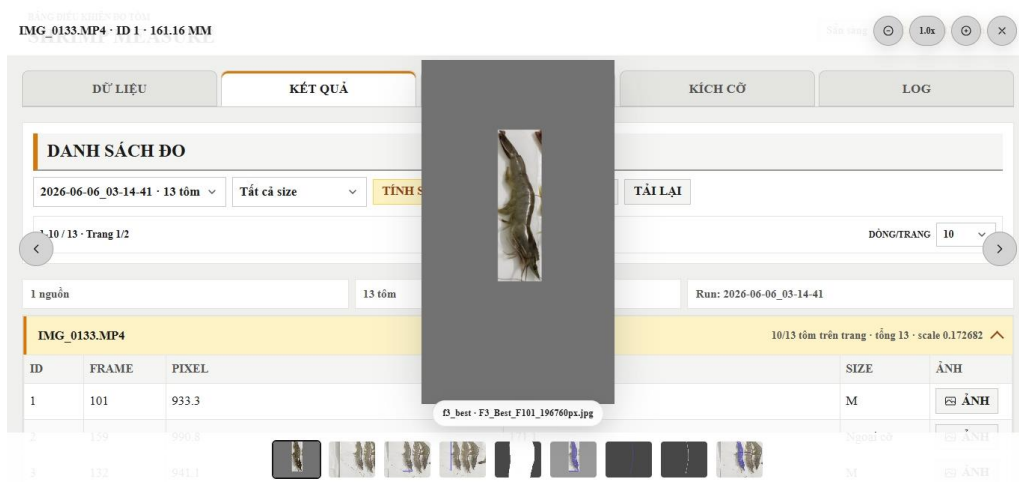
Tab log giúp người dùng theo dõi quá trình chạy pipeline theo thời gian. Khi một ảnh hoặc một cá thể không tạo được kết quả, log giúp xác định nguyên nhân như không phát hiện được tôm, không có mask hoặc skeleton không hợp lệ.

Ảnh minh họa kết quả giúp kiểm tra lại các bước trung gian gồm bounding box, frame đại diện, segmentation mask, skeleton và kết quả đo. Việc lưu ảnh kiểm tra làm cho kết quả đo minh bạch hơn, đặc biệt khi cần phân tích các trường hợp sai số lớn.

Các ảnh minh họa được lưu trong thư mục output của từng run và được phục vụ lại qua backend để giao diện có thể mở trực tiếp. Nhờ đó, người dùng không chỉ nhìn thấy giá trị chiều dài cuối cùng mà còn kiểm tra được các bước trung gian dẫn đến kết quả đó. Với các mẫu có sai số lớn, ảnh debug giúp xác định lỗi đến từ detection, frame đại diện, mask, skeleton hay bước quy đổi.



Hình 7.19. Giao diện tab log dùng để theo dõi trạng thái xử lý



Hình 7.20. Giao diện xem ảnh minh họa kết quả xử lý

Tab kích cỡ cho phép người dùng cấu hình các nhóm phân loại. Sau khi pipeline hoàn thành, kết quả có thể được lưu và export theo dạng dữ liệu phục vụ kiểm tra hoặc báo cáo.

Các khoảng kích cỡ được lưu trong settings.json cùng với nhãn cho trường hợp nhỏ hơn khoảng đầu tiên, lớn hơn khoảng cuối cùng hoặc không khớp khoảng nào. Khi F6 tạo kết quả, chiều dài milimet được so với bảng kích cỡ hiện tại để gán nhãn phân loại. Việc cho phép chỉnh bảng kích cỡ từ giao diện giúp hệ thống thích nghi với nhiều quy tắc phân cỡ khác nhau mà không cần sửa mã nguồn.

DỮ LIỆU

KẾT QUẢ

CẤU HÌNH

KÍCH CỠ

LOG

PHÂN LOẠI KÍCH CỠ

TÊN CỠ	TỪ MM	ĐẾN MM	THAO TÁC
L	180	220	XÓA
M	140	170	XÓA
S	120	140	XÓA

+ THÊM CỠ

TÊN DƯỚI CỠ NHỎ NHẤT

Ngoại cỡ nhỏ

Tên dùng cho tôm nhỏ hơn khoảng đầu tiên trong bảng.

TÊN TRÊN CỠ LỚN NHẤT

Ngoại cỡ lớn

Tên dùng cho tôm lớn hơn khoảng cuối cùng trong bảng.

TÊN NGOÀI KHOẢNG PHÂN LOẠI

Ngoại cỡ

Tên dự phòng khi kết quả không nằm trong khoảng nào đã khai báo.

✓ LƯU PHÂN LOẠI

Hình 7.21. Giao diện tab kích cỡ dùng để cấu hình nhóm phân loại

7.3 KIỂM THỬ KẾT QUẢ ĐO CHIỀU DÀI

7.3.1 Tập hiệu chuẩn scale factor

Tập hiệu chuẩn được dùng để xác định hệ số quy đổi từ pixel sang milimet trước khi đánh giá kết quả đo. Tập này gồm 30 ảnh, được chọn thành 15 cặp mẫu tương ứng giữa tôm thẳng và tôm cong. Cách chọn này giúp hệ số quy đổi bớt lệch về một tư thế thân tôm, đồng thời phản ánh cả trường hợp thân duỗi tương đối thẳng và trường hợp thân cong thường gặp trong dữ liệu thực tế.

Mỗi ảnh trong tập hiệu chuẩn có chiều dài thật được lưu trong bảng dữ liệu hiệu chuẩn. Khi hiệu chuẩn, hệ thống ghép chiều dài thật với chiều dài theo pixel do pipeline sinh ra, sau đó tính scale factor bằng hồi quy tuyến tính qua gốc tọa độ. Tập này không được xem là tập đánh giá sai số độc lập trong phần này vì dữ liệu hiện có chỉ gồm chiều dài thật phục vụ tính hệ số quy đổi, chưa có bảng kết quả đo độc lập để đánh giá riêng sai số của bước hiệu chuẩn như độ lệch và các chỉ số MAE, RMSE, MAPE.



Hình 7.22. Ảnh đại diện dataset hiệu chuẩn scale factor

Nhìn chung, tập hiệu chuẩn có vai trò quyết định độ đúng của toàn bộ bước quy đổi đơn vị. Nếu scale factor bị lệch, mọi tập kiểm thử phía sau đều sẽ bị lệch theo cùng chiều khi chuyển từ pixel sang milimet. Việc đưa cả mẫu thẳng và cong vào tập hiệu chuẩn giúp hệ số quy đổi ổn định hơn so với chỉ dùng một kiểu tư thế, nhưng tập này vẫn còn hạn chế vì chưa bao phủ nhiều camera, nhiều khoảng cách chụp, nhiều góc nhìn và nhiều điều kiện chiếu sáng khác nhau.

7.3.2 Các tập đánh giá

Sau khi hiệu chuẩn scale factor, hệ thống được đánh giá trên các nhóm dữ liệu gồm tôm thẳng một con, tôm cong một con, năm ảnh nhiều con và hai video. Hai nhóm một con kiểm tra khả năng đo cá thể trong hai tư thế chính. Nhóm nhiều con kiểm tra trường hợp nhiều cá thể xuất hiện trong cùng ảnh. Hai nhóm video kiểm tra thêm tracking, chọn frame đại diện và khả năng vận hành trên chuỗi frame.



Hình 7.23. Ảnh đại diện nhóm tôm thẳng một con



Hình 7.24. Ảnh đại diện nhóm tôm cong một con



Hình 7.25. Ảnh đại diện nhóm nhiều con nằm cạnh nhau

Bảng 7.1 trình bày chỉ số tổng hợp của toàn bộ các tập kiểm thử. Với các tập ảnh, cột FPS xử lý không áp dụng nên được ký hiệu bằng dấu gạch ngang. Với hai tập video, FPS xử lý là FPS tương đương theo frame nguồn, được tính từ số frame lý thuyết của video chia cho tổng thời gian xử lý.

Bảng 7.1. Chỉ số tổng hợp theo từng tập kiểm thử

Tên tập kiểm thử	Số mẫu	MAE (mm)	RMSE (mm)	MAPE (%)	FPS xử lý	Mẫu lệch lớn nhất
Tập hiệu chuẩn	30	-	-	-	-	-
Tôm thẳng một con	32	5.008	6.127	3.05	-	IMG_0224, ID 1, 12.73 mm
Tôm cong một con	32	4.380	5.585	2.73	-	IMG_0216, ID 1, - 15.69 mm
Nhiều con 1	8	5.070	6.094	3.22	-	IMG_0126, ID 7, 11.55 mm
Nhiều con 2	7	7.293	8.195	4.38	-	IMG_0127, ID 3, 12.38 mm
Nhiều con 3	4	8.920	8.936	5.71	-	IMG_0129, ID 4, 9.49 mm
Nhiều con 4	9	2.247	2.618	1.37	-	IMG_0238, ID 7, 5.09 mm
Nhiều con 5	8	4.077	5.190	2.32	-	IMG_0239, ID 6, - 9.98 mm
Video 1	13	9.748	10.831	5.64	22.68	IMG_0130, ID 22, 18.39 mm
Video 2	14	6.004	6.866	3.83	23.17	IMG_0133, ID 2, 11.45 mm

7.3.3 Kết quả chi tiết theo từng tập kiểm thử

Thay vì gộp tất cả mẫu vào một bảng lớn, phần này trình bày kết quả chi tiết thành từng bảng riêng theo từng tập kiểm thử. Cách tách này giúp đối chiếu sai số theo đúng bối cảnh dữ liệu, đồng thời làm rõ nhóm nào ổn định, nhóm nào có xu hướng đo dư hoặc đo thiếu.

Bảng 7.2. Kết quả đo chi tiết tập tằm thẳng một con

STT	Nguồn	ID	Đo được (mm)	Thực tế (mm)	Độ lệch (mm)	Phần trăm lệch (%)
1	IMG_0074	1	198.44	195.00	3.44	1.76
2	IMG_0082	1	165.33	155.00	10.33	6.66
3	IMG_0086	1	198.69	195.00	3.69	1.89
4	IMG_0088	1	174.75	175.00	-0.25	-0.14
5	IMG_0092	1	160.18	160.00	0.18	0.11
6	IMG_0094	1	196.48	190.00	6.48	3.41
7	IMG_0096	1	212.49	205.00	7.49	3.65
8	IMG_0098	1	167.88	160.00	7.88	4.92
9	IMG_0100	1	157.53	155.00	2.53	1.63
10	IMG_0102	1	162.43	155.00	7.43	4.79
11	IMG_0104	1	158.87	155.00	3.87	2.50
12	IMG_0108	1	170.50	170.00	0.50	0.29
13	IMG_0110	1	161.89	160.00	1.89	1.18
14	IMG_0112	1	170.32	160.00	10.32	6.45
15	IMG_0114	1	171.39	160.00	11.39	7.12
16	IMG_0116	1	159.27	155.00	4.27	2.75
17	IMG_0118	1	154.01	155.00	-0.99	-0.64
18	IMG_0120	1	160.42	155.00	5.42	3.50
19	IMG_0122	1	161.20	155.00	6.20	4.00
20	IMG_0124	1	165.23	160.00	5.23	3.27
21	IMG_0201	1	179.23	180.00	-0.77	-0.43
22	IMG_0207	1	161.20	165.00	-3.80	-2.30
23	IMG_0209	1	159.46	155.00	4.46	2.88
24	IMG_0215	1	133.74	145.00	-11.26	-7.77
25	IMG_0217	1	123.86	127.00	-3.14	-2.47

STT	Nguồn	ID	Đo được (mm)	Thực tế (mm)	Độ lệch (mm)	Phần trăm lệch (%)
26	IMG_0224	1	215.73	203.00	12.73	6.27
27	IMG_0226	1	169.05	165.00	4.05	2.45
28	IMG_0228	1	155.46	165.00	-9.54	-5.78
29	IMG_0230	1	161.19	165.00	-3.81	-2.31
30	IMG_0232	1	157.58	160.00	-2.42	-1.51
31	IMG_0234	1	155.96	160.00	-4.04	-2.52
32	IMG_0236	1	154.53	155.00	-0.47	-0.30

Ở Bảng 7.2, tập tôm thẳng một con có đa số mẫu sai lệch nhỏ, nhiều mẫu nằm dưới 5 mm hoặc dưới 3%. Các mẫu IMG_0088, IMG_0092, IMG_0108, IMG_0118, IMG_0201 và IMG_0236 gần với chiều dài thật, cho thấy pipeline ổn định khi chỉ có một cá thể và thân tôm tương đối duỗi thẳng. Sai số lớn nhất là IMG_0224 với 12.73 mm, trong khi IMG_0215 và IMG_0228 bị đo thiếu lần lượt -11.26 mm và -9.54 mm. Các trường hợp này cho thấy dù điều kiện ảnh thuận lợi, kết quả vẫn phụ thuộc vào mức độ mask giữ đủ vùng đầu và đuôi trước khi tạo skeleton.

Bảng 7.3. Kết quả đo chi tiết tập tôm cong một con

STT	Nguồn	ID	Đo được (mm)	Thực tế (mm)	Độ lệch (mm)	Phần trăm lệch (%)
1	IMG_0075	1	193.40	195.00	-1.60	-0.82
2	IMG_0083	1	156.77	155.00	1.77	1.14
3	IMG_0087	1	185.73	195.00	-9.27	-4.75
4	IMG_0089	1	169.96	175.00	-5.04	-2.88
5	IMG_0093	1	154.69	160.00	-5.31	-3.32
6	IMG_0095	1	188.15	190.00	-1.85	-0.97
7	IMG_0097	1	208.27	205.00	3.27	1.60
8	IMG_0099	1	161.00	160.00	1.00	0.62
9	IMG_0101	1	148.09	155.00	-6.91	-4.46
10	IMG_0103	1	158.14	155.00	3.14	2.03
11	IMG_0105	1	151.62	155.00	-3.38	-2.18
12	IMG_0109	1	164.57	170.00	-5.43	-3.19

STT	Nguồn	ID	Đo được (mm)	Thực tế (mm)	Độ lệch (mm)	Phần trăm lệch (%)
13	IMG_0111	1	162.39	160.00	2.39	1.49
14	IMG_0113	1	159.30	160.00	-0.70	-0.44
15	IMG_0115	1	159.79	160.00	-0.21	-0.13
16	IMG_0117	1	159.86	155.00	4.86	3.14
17	IMG_0119	1	156.46	155.00	1.46	0.94
18	IMG_0121	1	159.02	155.00	4.02	2.59
19	IMG_0123	1	160.88	155.00	5.88	3.79
20	IMG_0125	1	161.43	160.00	1.43	0.89
21	IMG_0202	1	174.98	180.00	-5.02	-2.79
22	IMG_0208	1	163.16	165.00	-1.84	-1.12
23	IMG_0210	1	150.13	155.00	-4.87	-3.14
24	IMG_0216	1	129.31	145.00	-15.69	-10.82
25	IMG_0218	1	119.71	127.00	-7.29	-5.74
26	IMG_0225	1	205.62	203.00	2.62	1.29
27	IMG_0227	1	164.60	165.00	-0.40	-0.24
28	IMG_0229	1	151.10	165.00	-13.90	-8.42
29	IMG_0231	1	159.40	165.00	-5.60	-3.39
30	IMG_0233	1	162.72	160.00	2.72	1.70
31	IMG_0235	1	154.37	160.00	-5.63	-3.52
32	IMG_0237	1	149.34	155.00	-5.66	-3.65

Ở Bảng 7.3, tập tôm cong một con có nhiều mẫu sai số nhỏ như IMG_0113, IMG_0115, IMG_0227 và IMG_0099. Tuy nhiên, các mẫu cong mạnh có xu hướng đo thiếu rõ hơn, đặc biệt IMG_0216 lệch -15.69 mm và IMG_0229 lệch -13.90 mm. Điều này cho thấy khi thân tôm cong, đường skeleton có thể không đi đủ theo biên dạng thật nếu mask bị khuyết ở đoạn đầu, đuôi hoặc vùng cong. Vì vậy, chỉ số tổng hợp của tập này tốt nhưng vẫn cần xem xét kỹ từng mẫu có sai số âm lớn.

Bảng 7.4. Kết quả đo chi tiết tập nhiều con thứ nhất

STT	Nguồn	ID	Đo được (mm)	Thực tế (mm)	Độ lệch (mm)	Phần trăm lệch (%)
1	IMG_0126	1	191.23	195.00	-3.77	-1.93
2	IMG_0126	2	170.91	170.00	0.91	0.54
3	IMG_0126	3	131.60	125.00	6.60	5.28
4	IMG_0126	4	210.75	210.00	0.75	0.36
5	IMG_0126	5	144.37	140.00	4.37	3.12
6	IMG_0126	6	159.58	155.00	4.58	2.95
7	IMG_0126	7	166.55	155.00	11.55	7.45
8	IMG_0126	8	203.03	195.00	8.03	4.12

Ở Bảng 7.4, các cá thể trong cùng một ảnh có mức sai số không đồng đều. ID 2 và ID 4 gần như trùng với chiều dài thật, cho thấy hệ thống vẫn tách và đo được từng cá thể khi mask rõ. Ngược lại, ID 7 lệch 11.55 mm và ID 8 lệch 8.03 mm, phản ánh ảnh hưởng của vị trí cá thể trong ảnh hoặc vùng mask gần cá thể lân cận. Với tập này, sai số chủ yếu nghiêng về đo dư ở các cá thể khó hơn.

Bảng 7.5. Kết quả đo chi tiết tập nhiều con thứ hai

STT	Nguồn	ID	Đo được (mm)	Thực tế (mm)	Độ lệch (mm)	Phần trăm lệch (%)
1	IMG_0127	1	203.12	205.00	-1.88	-0.92
2	IMG_0127	2	163.52	155.00	8.52	5.50
3	IMG_0127	3	172.38	160.00	12.38	7.74
4	IMG_0127	4	169.02	160.00	9.02	5.64
5	IMG_0127	5	161.53	160.00	1.53	0.96
6	IMG_0127	6	198.74	190.00	8.74	4.60
7	IMG_0127	7	178.98	170.00	8.98	5.28

Ở Bảng 7.5, chỉ ID 1 và ID 5 có sai số nhỏ, còn phần lớn cá thể bị đo dư từ khoảng 8 mm đến hơn 12 mm. Mẫu ID 3 có sai số lớn nhất, đạt 12.38 mm. Xu hướng đo dư lặp lại trên nhiều ID cho thấy tập này khó hơn tập nhiều con thứ nhất, nhiều khả năng do biên giữa các cá thể chưa tách rõ hoặc skeleton bị kéo dài bởi phần mask dư.

Bảng 7.6. Kết quả đo chi tiết tập nhiều con thứ ba

STT	Nguồn	ID	Đo được (mm)	Thực tế (mm)	Độ lệch (mm)	Phần trăm lệch (%)
1	IMG_0129	1	163.10	155.00	8.10	5.23
2	IMG_0129	2	169.25	160.00	9.25	5.78
3	IMG_0129	3	163.84	155.00	8.84	5.70
4	IMG_0129	4	164.49	155.00	9.49	6.12

Ở Bảng 7.6, cả bốn cá thể đều bị đo dư, với độ lệch nằm trong khoảng 8.10 mm đến 9.49 mm. Vì số mẫu ít nhưng sai số cùng chiều, tập này cho thấy một tình huống khó có tính hệ thống hơn là sai lệch ngẫu nhiên. Khả năng cao vùng mask hoặc skeleton trong ảnh này có xu hướng dài hơn chiều dài thật, làm MAPE của tập nhiều con thứ ba cao nhất trong nhóm ảnh nhiều con.

Bảng 7.7. Kết quả đo chi tiết tập nhiều con thứ tư

STT	Nguồn	ID	Đo được (mm)	Thực tế (mm)	Độ lệch (mm)	Phần trăm lệch (%)
1	IMG_0238	1	189.20	190.00	-0.80	-0.42
2	IMG_0238	2	161.47	165.00	-3.53	-2.14
3	IMG_0238	3	184.56	185.00	-0.44	-0.24
4	IMG_0238	4	153.35	155.00	-1.65	-1.06
5	IMG_0238	5	181.44	180.00	1.44	0.80
6	IMG_0238	6	145.38	143.00	2.38	1.66
7	IMG_0238	7	183.09	178.00	5.09	2.86
8	IMG_0238	8	147.76	150.00	-2.24	-1.49
9	IMG_0238	9	167.65	165.00	2.65	1.61

Ở Bảng 7.7, tập nhiều con thứ tư có kết quả ổn định nhất trong nhóm ảnh nhiều con. Phần lớn mẫu lệch dưới 3 mm, chỉ ID 7 lệch 5.09 mm. Sai số có cả âm và dương nhưng biên độ nhỏ, cho thấy hệ thống tách từng cá thể tốt trong ảnh này. Đây là ví dụ cho thấy số lượng cá thể nhiều không nhất thiết làm sai số tăng nếu vùng mask của từng cá thể đủ rõ và ít chồng lấn.

Bảng 7.8. Kết quả đo chi tiết tập nhiều con thứ năm

STT	Nguồn	ID	Đo được (mm)	Thực tế (mm)	Độ lệch (mm)	Phần trăm lệch (%)
1	IMG_0239	1	166.92	165.00	1.92	1.16
2	IMG_0239	2	207.72	203.00	4.72	2.33
3	IMG_0239	3	210.98	203.00	7.98	3.93
4	IMG_0239	4	159.61	155.00	4.61	2.97
5	IMG_0239	5	163.50	165.00	-1.50	-0.91
6	IMG_0239	6	155.02	165.00	-9.98	-6.05
7	IMG_0239	7	161.64	160.00	1.64	1.02
8	IMG_0239	8	160.27	160.00	0.27	0.17

Ở Bảng 7.8, tập nhiều con thứ năm nhìn chung có sai số thấp, đặc biệt ID 8 gần như trùng với chiều dài thật. Sai số đáng chú ý nhất là ID 6 bị đo thiếu -9.98 mm, trái chiều với một số mẫu đo dư như ID 3. Điều này cho thấy trong cùng một ảnh nhiều con, từng cá thể có thể chịu lỗi khác nhau: có cá thể bị kéo dài do mask dư, nhưng cũng có cá thể bị rút ngắn nếu vùng đầu hoặc đuôi chưa được giữ đầy đủ.

Bảng 7.9. Kết quả đo chi tiết tập video thứ nhất

STT	Nguồn	ID	Đo được (mm)	Thực tế (mm)	Độ lệch (mm)	Phần trăm lệch (%)
1	IMG_0130	1	165.83	155.00	10.83	6.99
2	IMG_0130	2	199.17	195.00	4.17	2.14
3	IMG_0130	4	129.22	125.00	4.22	3.38
4	IMG_0130	5	183.68	170.00	13.68	8.05
5	IMG_0130	6	218.32	210.00	8.32	3.96
6	IMG_0130	7	160.34	155.00	5.34	3.45
7	IMG_0130	9	150.01	140.00	10.01	7.15
8	IMG_0130	10	202.20	195.00	7.20	3.69
9	IMG_0130	12	183.97	170.00	13.97	8.22
10	IMG_0130	14	167.08	160.00	7.08	4.43
11	IMG_0130	15	165.61	160.00	5.61	3.51

STT	Nguồn	ID	Đo được (mm)	Thực tế (mm)	Độ lệch (mm)	Phần trăm lệch (%)
12	IMG_0130	21	207.91	190.00	17.91	9.43
13	IMG_0130	22	223.39	205.00	18.39	8.97

Ở Bảng 7.9, tập video thứ nhất có xu hướng đo dư rõ rệt, chỉ số lệch của toàn bộ 13 mẫu đều dương. Các mẫu ID 21 và ID 22 có sai số lớn nhất, lần lượt 17.91 mm và 18.39 mm. Kết quả này cho thấy sai số video không chỉ phụ thuộc vào thuật toán đo trên một ảnh đơn, mà còn chịu ảnh hưởng của tracking và frame đại diện được chọn. Khi frame đại diện chưa tối ưu, vùng đưa sang segmentation có thể làm chiều dài skeleton bị kéo dài.

Bảng 7.10. Kết quả đo chi tiết tập video thứ hai

STT	Nguồn	ID	Đo được (mm)	Thực tế (mm)	Độ lệch (mm)	Phần trăm lệch (%)
1	IMG_0133	1	163.56	160.00	3.56	2.23
2	IMG_0133	2	176.45	165.00	11.45	6.94
3	IMG_0133	3	163.16	155.00	8.16	5.26
4	IMG_0133	4	158.09	155.00	3.09	1.99
5	IMG_0133	5	170.37	170.00	0.37	0.22
6	IMG_0133	9	134.57	130.00	4.57	3.52
7	IMG_0133	10	168.76	160.00	8.76	5.47
8	IMG_0133	11	165.07	165.00	0.07	0.04
9	IMG_0133	12	174.09	165.00	9.09	5.51
10	IMG_0133	15	163.14	155.00	8.14	5.25
11	IMG_0133	16	163.36	155.00	8.36	5.39
12	IMG_0133	17	161.69	155.00	6.69	4.32
13	IMG_0133	18	163.08	155.00	8.08	5.21
14	IMG_0133	19	163.67	160.00	3.67	2.29

Ở Bảng 7.10, tập video thứ hai cũng có xu hướng đo dư, nhưng mức sai số thấp hơn tập video thứ nhất. ID 11 gần như trùng với chiều dài thật, trong khi ID 2 có sai số lớn nhất là 11.45 mm. Các mẫu còn lại chủ yếu lệch trong khoảng 3 mm đến 9 mm. Điều này cho thấy cùng là dữ liệu video, kết quả vẫn thay đổi theo chất lượng frame, tư thế tôm trong frame đại diện và mức độ rõ của mask sau segmentation.

7.3.4 Nhận xét chung về hệ thống

Từ các bảng trên, hệ thống đo chiều dài ổn định nhất khi ảnh có một cá thể rõ ràng hoặc khi nhiều cá thể được tách biệt tốt. Các tập tôm thẳng một con, tôm cong một con, nhiều con thứ tư và nhiều con thứ năm đều có MAPE dưới 3.10%, cho thấy pipeline có khả năng đo ở mức khá tốt trong điều kiện ảnh thuận lợi. Sai số tăng rõ hơn ở các trường hợp nhiều cá thể sát nhau và video, nơi hệ thống phải xử lý thêm bài toán tách cá thể, tracking và chọn frame đại diện.

Nguồn sai số chính của hệ thống đến từ bốn điểm. Thứ nhất là mask segmentation chưa bao phủ đủ đầu hoặc đuôi tôm, làm skeleton bị ngắn. Thứ hai là mask có thể dính hoặc nhiều khi nhiều cá thể nằm gần nhau, làm skeleton bị kéo dài. Thứ ba là tôm cong mạnh khiến đường skeleton dễ xuất hiện nhánh phụ hoặc bị rút ngắn ở vùng cong. Thứ tư là dữ liệu video phụ thuộc vào track ID và frame đại diện, nên chỉ một frame chọn chưa tối ưu cũng có thể làm sai số cuối tăng. Vì vậy, khi triển khai vận hành, cần ưu tiên kiểm soát chất lượng ảnh đầu vào, hiệu chuẩn scale factor đúng bối cảnh chụp và kiểm tra lại các mẫu có độ lệch lớn.

7.4 KIỂM THỬ HIỆU NĂNG XỬ LÝ VIDEO

7.4.1 FPS xử lý của hai tập video

Hệ thống ghi nhận thời gian xử lý của hai tập video để đánh giá khả năng vận hành trên dữ liệu chuỗi frame. Video 1 dài 33 giây, độ phân giải 1080 x 1920, tốc độ 30 FPS, tương ứng khoảng 990 frame nguồn. Tổng thời gian xử lý là 43.65 giây, nên FPS xử lý tương đương đạt 22.68 FPS. Video 2 dài 26 giây, cùng độ phân giải và tốc độ 30 FPS, tương ứng khoảng 780 frame nguồn. Tổng thời gian xử lý là 33.67 giây, nên FPS xử lý tương đương đạt 23.17 FPS.

Bảng 7.11. Hiệu năng xử lý của hai tập video

Tên tập kiểm thử	Thời lượng video	Độ phân giải	FPS nguồn	Số frame ước lượng	Thời gian xử lý (s)	FPS xử lý tương đương
Video 1	33 s	1080 x 1920	30	990	43.65	22.68
Video 2	26 s	1080 x 1920	30	780	33.67	23.17

FPS xử lý tương đương trong Bảng 7.11 được tính theo số frame nguồn lý thuyết chia cho tổng thời gian xử lý. Đây không phải là FPS suy luận trực tiếp trên số frame thực sự được đưa vào mô hình, vì pipeline có thể lấy frame theo step cấu hình và chỉ đưa một phần frame vào các bước detection, tracking hoặc segmentation. Chỉ số này phù hợp để đánh giá tốc độ xử lý thực tế của cả pipeline trên một video đầu vào, bao gồm đọc frame, phát hiện, tracking, chọn frame đại diện, segmentation, skeleton và ghi kết quả.

7.4.2 So sánh giữa hai video

Video 2 có FPS xử lý tương đương 23.17, cao hơn nhẹ so với 22.68 của video 1. Chênh lệch này không lớn, cho thấy thời gian xử lý của pipeline khá ổn định khi hai video có cùng độ phân giải và FPS nguồn. Tuy nhiên, video 1 có MAE 9.748 mm và MAPE 5.64%, cao hơn video 2 với MAE 6.004 mm và MAPE 3.83%. Điều này cho thấy tốc độ xử lý và độ chính xác đo không luôn thay đổi cùng chiều: video có tốc độ xử lý tương đương vẫn có thể cho sai số khác nhau do nội dung frame, tư thế tôm, mức độ rõ của mask và thời điểm chọn frame đại diện.

So với ảnh tĩnh, video có luồng xử lý phức tạp hơn vì hệ thống phải duy trì track ID qua nhiều frame và chỉ chọn một frame đại diện trước khi đo. Cách thiết kế này giúp giảm số lần chạy YOLOv8 Segment so với việc segmentation mọi frame, nhưng làm chất lượng kết quả phụ thuộc nhiều vào logic chọn frame. Vì vậy, cải thiện video trong các phiên bản sau nên tập trung vào điều kiện chọn frame đại diện, kiểm tra track bị mất, và bổ sung cơ chế ưu tiên frame có mask rõ thay vì chỉ dựa trên diện tích bounding box.

CHƯƠNG 8. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

8.1 KẾT QUẢ ĐẠT ĐƯỢC

Đề tài đã xây dựng được hệ thống đo chiều dài và phân loại kích cỡ tôm từ ảnh và video. Hệ thống đã chuyển được dữ liệu hình ảnh đầu vào thành kết quả chiều dài theo milimet, đồng thời gán nhóm kích cỡ theo bảng phân loại cấu hình trước. Đây là kết quả chính của đề tài, thể hiện việc kết hợp được mô hình học sâu, xử lý ảnh và giao diện vận hành thành một pipeline hoàn chỉnh.

Về mặt xử lý, đề tài đã xây dựng pipeline gồm các bước đọc dữ liệu đầu vào, phát hiện tôm bằng YOLOv8 Detect, tracking đối với video bằng ByteTrack, chọn frame đại diện, segmentation vùng tôm bằng YOLOv8 Segment, tạo skeleton, đo chiều dài theo pixel, quy đổi sang milimet bằng scale factor và lưu kết quả. Việc tách pipeline thành các flow từ F1 đến F6 giúp từng bước xử lý có vai trò rõ ràng, thuận lợi cho kiểm tra lỗi, phân tích sai số và mở rộng sau này.

Về mặt ứng dụng, đề tài đã triển khai được giao diện web để người dùng thao tác mà không cần chạy lệnh thủ công. Giao diện hỗ trợ nạp ảnh hoặc video, điều chỉnh cấu hình xử lý, chạy pipeline, theo dõi log, xem ảnh minh họa trung gian, hiệu chuẩn scale factor, xem kết quả đo và export dữ liệu. Nhờ đó, hệ thống không chỉ dừng ở mức mã nguồn xử lý đơn lẻ mà đã có hình thức vận hành phù hợp với quá trình thử nghiệm thực tế.

Về mặt kiểm thử, đề tài đã kiểm thử được hệ thống trên các nhóm dữ liệu gồm tôm thẳng một con, tôm cong một con, nhiều con trong cùng ảnh và video. Kết quả cho thấy hệ thống đo ổn định hơn trong các ảnh một cá thể rõ ràng hoặc khi các cá thể được tách biệt tốt. Tập tôm thẳng một con đạt MAE 5.008 mm và MAPE 3.05%. Tập tôm cong một con đạt MAE 4.380 mm và MAPE 2.73%. Trong nhóm nhiều con, sai số thay đổi theo từng ảnh, trong đó tập nhiều con thứ tư đạt kết quả tốt nhất với MAE 2.247 mm và MAPE 1.37%, còn tập nhiều con thứ ba có sai số cao hơn với MAE 8.920 mm và MAPE 5.71%.

Đối với dữ liệu video, hệ thống đã chạy được chuỗi frame, duy trì tracking, chọn frame đại diện và đo chiều dài từng cá thể được ghi nhận. Tập video thứ nhất đạt MAE 9.748 mm và MAPE 5.64%, còn tập video thứ hai đạt MAE 6.004 mm và MAPE 3.83%. Về hiệu năng, tốc độ xử lý tương đương đạt 22.68 FPS với video thứ nhất và 23.17 FPS với video thứ hai. Các kết quả này cho thấy hệ thống hiện tại đã vận hành được trên cả ảnh tĩnh và video, đồng thời chỉ ra rõ các điều kiện làm sai số tăng để định hướng cải tiến.

8.2 HẠN CHẾ CỦA HỆ THỐNG HIỆN TẠI

Hạn chế lớn nhất của hệ thống hiện tại nằm ở bộ dữ liệu kiểm thử (dataset). Dữ liệu hiện tại còn nhỏ và được thu trong điều kiện có kiểm soát, chưa đại diện đầy đủ cho môi trường băng chuyền công nghiệp. Trong môi trường thực tế, ánh sáng, nền ảnh, độ ướt bề mặt, tốc độ di chuyển, tư thế tôm, mức độ chồng lấn và dải kích cỡ có thể thay đổi nhiều hơn so với dữ liệu đã dùng trong đề tài. Vì vậy, kết quả hiện tại mới phản ánh khả năng của hệ thống trong phạm vi thử nghiệm hiện tại, chưa đủ để khẳng định độ ổn định trong mọi điều kiện sản xuất.

Các tập nhiều con và video cũng còn ít mẫu. Nhóm nhiều con mới phản ánh một số ảnh đại diện, trong khi thực tế có thể xuất hiện nhiều kiểu chồng lấn, che khuất, tiếp xúc biên và mật độ cá thể khác nhau. Hai video kiểm thử giúp đánh giá được tracking và chọn frame đại diện, nhưng chưa đủ để bao phủ các tình huống video dài, tốc độ băng chuyền thay đổi, cá thể vào ra liên tục hoặc Track ID bị đứt trong thời gian dài.

Về thuật toán, kết quả đo vẫn phụ thuộc nhiều vào chất lượng segmentation mask và skeleton. Nếu mask thiếu vùng đầu hoặc đuôi, chiều dài skeleton có thể bị rút ngắn. Nếu mask dính sang cá thể khác hoặc có nhiễu ở biên, skeleton có thể bị kéo dài hoặc sinh nhánh phụ. Với tôm cong mạnh, đường skeleton cũng dễ không bám đúng trục thân chính, làm kết quả đo lệch so với chiều dài thật.

Với dữ liệu video, kết quả còn phụ thuộc vào tracking và frame đại diện. Khi Track ID không ổn định, cá thể di chuyển nhanh hoặc frame được chọn chưa thể hiện rõ toàn thân tôm, các bước segmentation và skeleton phía sau sẽ kế thừa sai lệch từ frame đầu vào. Cơ chế chọn frame hiện tại phù hợp với phạm vi thử nghiệm hiện tại, nhưng chưa đủ mạnh để xử lý mọi kiểu chuyển động, che khuất và thay đổi tư thế trong vận hành thực tế.

Scale factor cũng là một hạn chế quan trọng trong triển khai. Hệ số này phụ thuộc vào camera, khoảng cách chụp, tiêu cự, độ phân giải, góc nhìn và vùng quan sát. Khi thay đổi bố trí phần cứng mà không hiệu chuẩn lại, toàn bộ kết quả chiều dài theo milimet có thể bị lệch theo cùng một hướng. Do đó, hệ thống cần quy trình hiệu chuẩn chặt chẽ hơn nếu đưa vào môi trường sản xuất.

8.3 HƯỚNG PHÁT TRIỂN

Hướng phát triển đầu tiên là mở rộng bộ dữ liệu bằng dữ liệu thực tế từ băng chuyền. Bộ dữ liệu mới cần có nhiều điều kiện ánh sáng, nền ảnh, độ ướt bề mặt, tốc độ di chuyển, tư thế thân tôm, mức độ chồng lấn, dải kích cỡ và video dài hơn. Việc mở rộng dữ liệu sẽ giúp đánh giá lại toàn bộ pipeline trong bối cảnh gần sản xuất, đồng thời cung cấp thêm mẫu khó để cải thiện mô hình detection, segmentation và thuật toán hậu xử lý.

Hướng thứ hai là tích hợp hệ thống với Internet of Things (IoT) để tiến gần hơn đến vận hành thực tế. Một mô hình triển khai có thể gồm camera cố định phía trên băng chuyền, hệ thống đèn chiếu sáng ổn định, cảm biến tốc độ băng chuyền, cảm biến hiện diện để kích hoạt ghi hình, bộ xử lý biên để chạy pipeline tại chỗ và dashboard để giám sát kết quả đo. Khi kết hợp với IoT, dữ liệu chiều dài và nhóm kích cỡ có thể được lưu theo thời gian, gửi về máy chủ hoặc kết nối với hệ thống thống kê sản lượng.

Hướng thứ ba là bổ sung các phương pháp học máy để hiệu chỉnh hoặc dự đoán chiều dài thật từ nhiều đặc trưng của tôm. Thay vì chỉ dùng chiều dài skeleton nhân với scale factor, hệ thống có thể trích thêm các đặc trưng như diện tích mask, chu vi mask, kích thước bounding box, tỷ lệ dài rộng, độ rộng thân, độ cong, khoảng cách đầu đuôi, độ dài skeleton, confidence của detection và segmentation. Từ các đặc trưng này, có thể huấn luyện mô hình hồi quy tuyến tính, Random Forest, Gradient Boosting hoặc neural network nhỏ để hiệu chỉnh sai số đo, đặc biệt ở các trường hợp tôm cong, mask thiếu vùng đầu đuôi hoặc nhiều cá thể nằm sát nhau.

Hướng thứ tư là cải thiện xử lý video. Cơ chế chọn frame đại diện có thể bổ sung tiêu chí về độ rõ nét, mức che khuất, độ ổn định Track ID, tỷ lệ vùng tâm trong khung hình và chất lượng mask sau segmentation. Khi có nhiều frame ứng viên của cùng một cá thể, hệ thống có thể chọn frame không chỉ theo diện tích bounding box mà còn theo độ tin cậy tổng hợp, từ đó giảm sai số ở các video có chuyển động nhanh hoặc tư thế thay đổi.

Hướng thứ năm là cải thiện hậu xử lý mask và skeleton. Hệ thống có thể bổ sung bước lọc nhánh phụ, kiểm tra vùng đầu đuôi, loại bỏ nhiễu nhỏ, phát hiện mask bị dính giữa hai cá thể và cảnh báo các mẫu có hình dạng bất thường. Các cải tiến này cần được đánh giá trực tiếp bằng sai số chiều dài, vì mục tiêu cuối cùng không chỉ là mask đẹp mà là kết quả đo gần với chiều dài thật.

Hướng cuối cùng là tối ưu hiệu năng và quy trình triển khai. Có thể tiếp tục tối ưu mô hình OpenVINO, điều chỉnh kích thước ảnh đầu vào, frame step, số lượng ảnh minh họa được lưu và cách tổ chức queue giữa các flow. Khi triển khai cùng phần cứng thật, cần đánh giá tốc độ xử lý theo độ phân giải camera, tốc độ băng chuyền và số lượng cá thể đi qua vùng quan sát để chọn cấu hình phù hợp giữa độ chính xác và thời gian xử lý.

TÀI LIỆU THAM KHẢO

- [1] OpenCV, "Operations with images," [Online]. Available: https://docs.opencv.org/4.x/d5/d98/tutorial_mat_operations.html.
- [2] OpenCV, "Image Thresholding," [Online]. Available: https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html.
- [3] Y. LeCun, L. Bottou, Y. Bengio and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278-2324, 1998.
- [4] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *Proc. 32nd International Conference on Machine Learning*, 2015.
- [5] Ultralytics, "Explore Ultralytics YOLOv8," [Online]. Available: <https://docs.ultralytics.com/models/yolov8/>.
- [6] Y. Qiu, X. Zhou, J. Wan, T. Yang, L. Zhang, Y. Zhong, L. Shen and X. Ji, "Automated urban flood level detection based on flooded bus dataset using YOLOv8," *Natural Hazards and Earth System Sciences*, vol. 25, pp. 3525-3544, 2025.
- [7] J. Ma, Y. Zhao, W. Fan and J. Liu, "An Improved YOLOv8 Model for Lotus Seedpod Instance Segmentation in the Lotus Pond Environment," *Agronomy*, vol. 14, no. 6, 2024.
- [8] Ultralytics, "Performance Metrics Deep Dive," [Online]. Available: <https://docs.ultralytics.com/guides/yolo-performance-metrics/>.
- [9] A. Neubeck and L. V. Gool, "Efficient Non-Maximum Suppression," in *Proc. 18th International Conference on Pattern Recognition (ICPR 2006)*, 2006.
- [10] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu and X. Wang, "ByteTrack: Multi-Object Tracking by Associating Every Detection Box," in *Proc. European Conference on Computer Vision*, 2022.
- [11] OpenCV, "Structural Analysis and Shape Descriptors," [Online]. Available: https://docs.opencv.org/4.x/d3/dc0/group_imgproc_shape.html.
- [12] scikit-image, "skimage.morphology," [Online]. Available: <https://scikit-image.org/docs/stable/api/skimage.morphology.html>.
- [13] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, H. Cormen, C. E. Leiserson, R. L. Rivest và C. Stein, Introduction to Algorithms, tái bản lần thứ 4. Cambridge, MA, Hoa Kỳ: MIT Press, 2022.

- [14] National Institute of Standards and Technology, "Linear Least Squares Regression," [Online]. Available: <https://www.itl.nist.gov/div898/handbook/pmd/section1/pmd141.htm>.
- [15] R. J. Hyndman and A. B. Koehler, "Another Look at Measures of Forecast Accuracy," *International Journal of Forecasting*, vol. 22, no. 4, pp. 679-688, 2006.
- [16] conda, "Managing environments," [Online]. Available: <https://docs.conda.io/projects/conda/en/stable/user-guide/tasks/manage-environments.html>.
- [17] Roboflow, "Dataset Versions," [Online]. Available: <https://docs.roboflow.com/datasets/dataset-versions>.
- [18] Kaggle, "Notebooks Documentation," [Online]. Available: <https://www.kaggle.com/docs/notebooks>.
- [19] Ultralytics, "Intel OpenVINO Export," [Online]. Available: <https://docs.ultralytics.com/integrations/openvino/>.
- [20] Pallets, "Welcome to Flask," [Online]. Available: <https://flask.palletsprojects.com/en/stable/>.
- [21] Tailwind Labs, "Styling with utility classes," [Online]. Available: <https://tailwindcss.com/docs/utility-first>.
- [22] NumPy, "NumPy documentation," [Online]. Available: <https://numpy.org/doc/stable/>.
- [23] Supervision, "Object Tracking," [Online]. Available: <https://supervision.roboflow.com/develop/notebooks/object-tracking/>.
- [24] E. Gazoni and C. Clark, "openpyxl - A Python library to read/write Excel 2010 xlsx/xlsm files," 2010. [Online]. Available: <https://openpyxl.readthedocs.io/en/stable/>.
- [25] Python Software Foundation, "json - JSON encoder and decoder," [Online]. Available: <https://docs.python.org/3.14/library/json.html>.
- [26] Python Software Foundation, "csv - CSV File Reading and Writing," [Online]. Available: <https://docs.python.org/3.14/library/csv.html>.
- [27] Công ty TNHH Máy và Thiết bị Thực phẩm Hoàng Nam, "Máy phân cỡ tôm FJ-A-200G," [Online]. Available: <https://thietbihoangnam.vn/san-pham/may-phan-co-tom-fj-a-200g>.

- [28] Công ty TNHH Anh Phát, "Máy phân cỡ con lăn," [Online]. Available: <https://anhphat.com.vn/products/may-phan-co-con-lan>.
- [29] H. Hong, X. Yang, Z. You and F. Cheng, "Visual Quality Detection of Aquatic Products Using Machine Vision," *Aquacultural Engineering*, vol. 63, pp. 62-71, 2014.
- [30] P.-M. Pan, J.-P. Li, G.-L. Lv, H. Yang, S.-M. Zhu and J.-Z. Lou, "Prediction of Shelled Shrimp Weight by Machine Vision," *Journal of Zhejiang University-SCIENCE B*, vol. 10, no. 8, pp. 589-594, 2009.
- [31] Công ty TNHH Anh Phát, "Máy phân cỡ tôm thông minh," [Online]. Available: <https://anhphat.com.vn/products/may-phan-co-tom-thong-minh>.
- [32] M. H. Long, "Shrimp Object Detection," [Online]. Available: <https://universe.roboflow.com/mnhlongs-workspace/shrimp-object-detection>.
- [33] M. H. Long, "Shrimp Instance Segmentation," [Online]. Available: <https://universe.roboflow.com/mnhlongs-workspace/shrimp-instance-segmentation>.
- [34] M. H. Long, "Test Detection," [Online]. Available: <https://universe.roboflow.com/mnhlongs-workspace/test-detection-8lsvs>.
- [35] M. H. Long, "Test Segmentation," [Online]. Available: <https://universe.roboflow.com/mnhlongs-workspace/test-segmentation-r7ryq>.
- [36] M. H. Long, "Shrimp YOLOv8 Object Detection," [Online]. Available: <https://www.kaggle.com/code/longmaingchong/shrimp-yolov8-object-detection>.
- [37] M. H. Long, "Shrimp YOLOv8 Instance Segmentation," [Online]. Available: <https://www.kaggle.com/code/longmaingchong/shrimp-yolov8-instance-segmentation>.